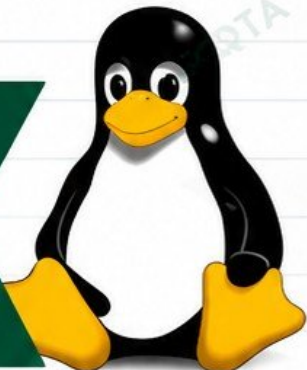


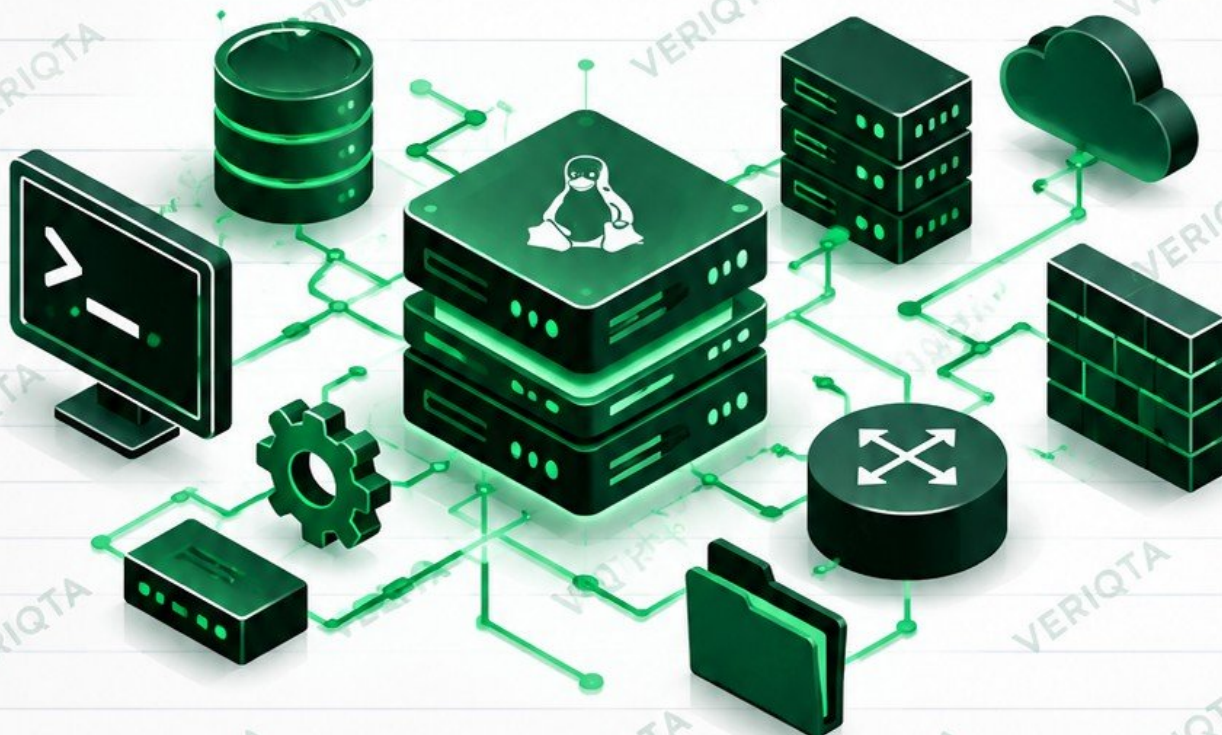
LINUX



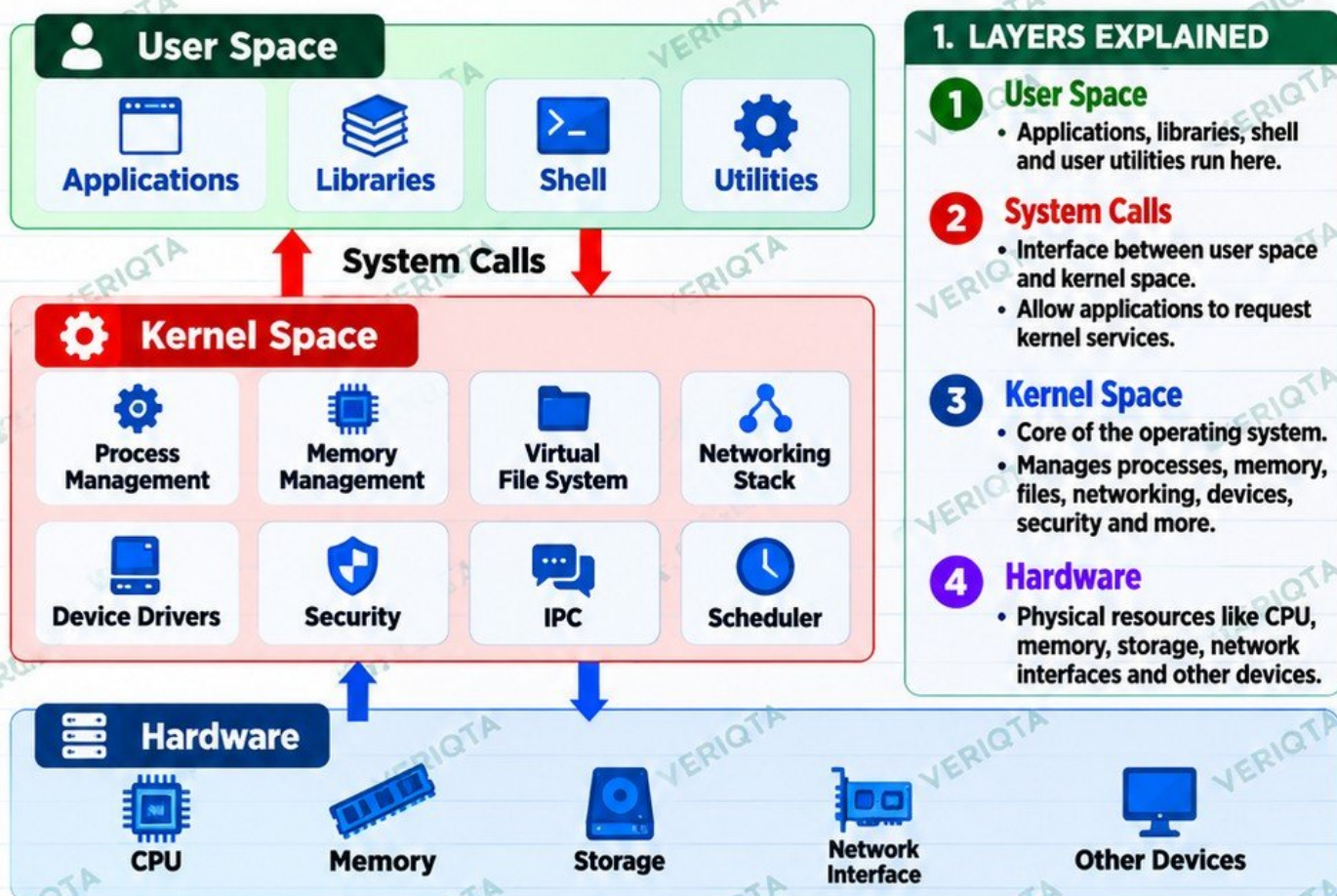
ENGINEERING

BIBLE

Broad Linux systems engineering reference



User space, kernel space, system calls, hardware boundary, major subsystems



1. LAYERS EXPLAINED

- 1 User Space**
 - Applications, libraries, shell and user utilities run here.
- 2 System Calls**
 - Interface between user space and kernel space.
 - Allow applications to request kernel services.
- 3 Kernel Space**
 - Core of the operating system.
 - Manages processes, memory, files, networking, devices, security and more.
- 4 Hardware**
 - Physical resources like CPU, memory, storage, network interfaces and other devices.

2. HOW IT WORKS

- 1** Application makes a request (e.g. read a file, open a socket).
- 2** Request goes to the kernel through a system call.
- 3** Kernel validates the request, checks permissions and performs the operation using the appropriate subsystem.
- 4** Kernel interacts with hardware through device drivers.
- 5** Result is returned back to the application in user space.

3. KEY POINTS

- User space is isolated from the kernel for security and stability.
- System calls provide a controlled interface to kernel services.
- The kernel manages CPU, memory, storage, networking, devices and security.
- Device drivers allow the kernel to communicate with hardware.
- Each kernel subsystem works together to provide a stable and efficient operating system.
- Understanding this architecture helps with troubleshooting, performance tuning, security hardening and system design.

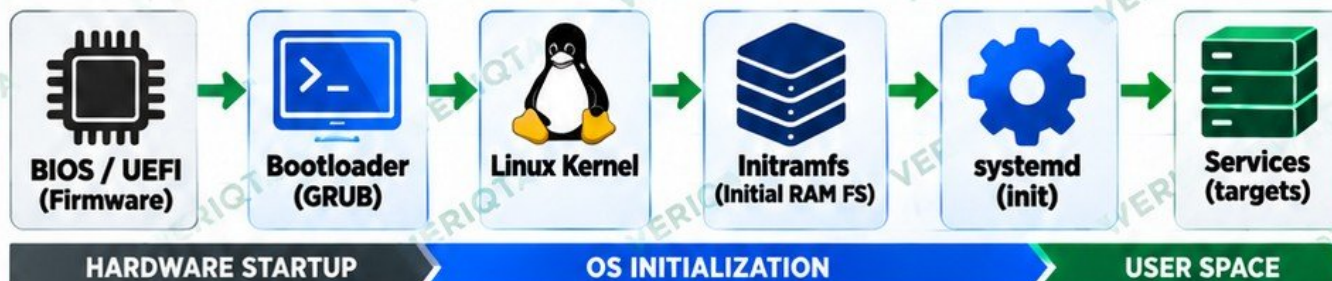
4. COMMON EXAMPLES

Operation	User Space Action	System Call	Kernel Subsystem	Hardware
Read a file	<code>cat /var/log/syslog</code>	<code>read()</code>	Virtual File System	Storage (Disk)
Create a process	<code>bash script.sh</code>	<code>fork()</code> , <code>execve()</code>	Process Management	CPU, Memory
Open a network connection	<code>curl https://example.com</code>	<code>socket()</code> , <code>connect()</code>	Networking Stack	Network Interface
Access a device	<code>ls /dev/sda</code>	<code>open()</code>	Device Drivers	Storage Device
Allocate memory	Application request	<code>mmap()</code> , <code>brk()</code>	Memory Management	Memory (RAM)



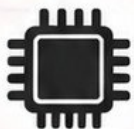
Firmware → bootloader → kernel → initramfs → systemd → services

1. BOOT SEQUENCE OVERVIEW



2. STAGES EXPLAINED

1 Firmware (BIOS / UEFI)



- Initializes hardware (CPU, memory, storage, devices).
- Performs POST (Power-On Self Test).
- Finds a boot device based on boot order.
- Loads the bootloader into memory.

2 Bootloader (GRUB)



- Provides a boot menu.
- Loads the selected Linux kernel and initramfs into memory.
- Passes kernel parameters (e.g. root device, options).
- Can boot multiple OS versions.

3 Linux Kernel



- Decompresses and initializes itself.
- Sets up memory management, device drivers, filesystem support and networking.
- Mounts the initramfs as a temporary root filesystem.
- Hands over control to initramfs.

4 Initramfs (Initial RAM FS)



- A minimal userspace environment.
- Loads required drivers (e.g. storage, LVM, RAID, encryption).
- Finds and mounts the real root filesystem (e.g. /dev/sda, LVM, network storage).
- Switches to the real root filesystem and starts systemd.

5 systemd (init)



- First process (PID 1) in user space.
- Reads unit files and starts target services.
- Manages service dependencies, parallel startup and recovery.
- Brings the system to the default target (e.g. multi-user.target).

6 Services (Targets)



- systemd starts all required services (e.g. networking, ssh, logging).
- Reaches the default target (e.g. multi-user.target for servers).
- System is ready for login and applications.

3. VIEW THE BOOT PROCESS

```

journalctl -b          # view logs from current boot
journalctl -b -1       # view logs from previous boot
systemd-analyze        # show boot time summary
systemd-analyze blame  # show slow services
systemd-analyze critical-chain # show boot dependency chain
dmesg | less           # view kernel messages
lsblk                  # check storage devices
  
```

4. COMMON BOOT ISSUES

Issue	Possible Cause
System stops at bootloader	Wrong disk, missing config, corrupted GRUB
Kernel panic	Driver issue, wrong kernel parameters
Initramfs drops to shell	Missing storage driver, wrong root device
systemd fails to start services	Service failure, dependency issue
Slow boot time	Slow storage, network delay, failing service
System reboots repeatedly	Kernel panic, hardware issue, misconfiguration



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

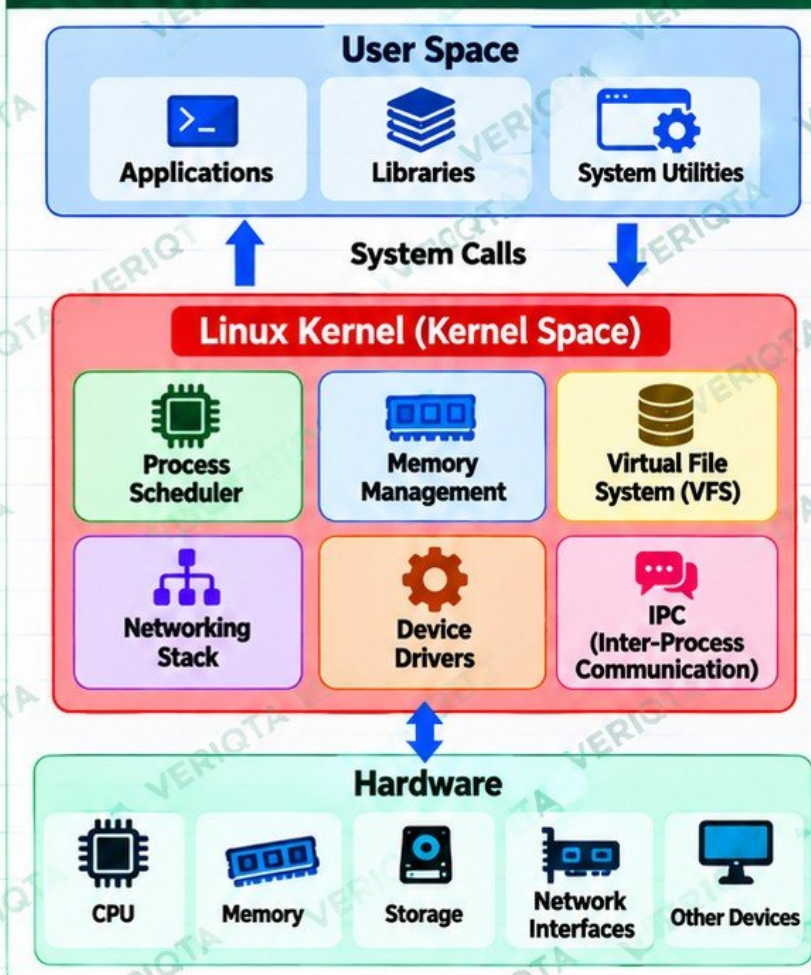


@veriqta

02

Scheduler, VFS, memory management, networking, drivers, IPC

1. BIG PICTURE



2. KEY SUBSYSTEMS

1 Process Scheduler



- Decides which process/thread runs on the CPU.
- Manages run queues, priorities, context switching and fairness.

2 Memory Management



- Manages virtual memory, physical memory, page cache and swap.
- Handles memory allocation, reclamation and protection.

3 Virtual File System (VFS)



- Provides a common interface for different filesystems.
- Manages files, directories, inodes, mounts and permissions.

4 Networking Stack



- Implements TCP/IP, routing, sockets and packet processing.
- Handles firewalling and network device interaction.

5 Device Drivers



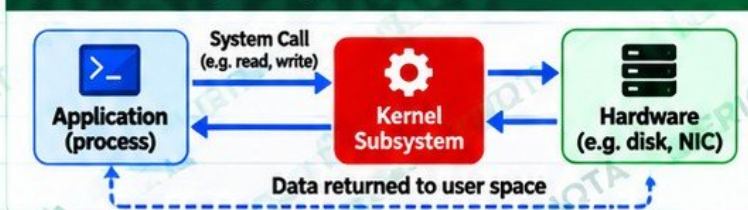
- Allow the kernel to communicate with hardware devices.
- Handle device initialization, I/O and interrupts.

6 IPC (Inter-Process Communication)



- Enables processes to communicate.
- Methods include pipes, message queues, shared memory, semaphores and sockets.

3. KERNEL COMPONENT FLOW



4. REAL KERNEL COMPONENTS

Component	Main Responsibility
Scheduler	CPU scheduling, process prioritization, context switching
Memory Management	Virtual memory, page cache, allocation, swap
VFS	Filesystem abstraction, inodes, dentries, mounts
Networking Stack	TCP/IP, routing, sockets, packet processing
Device Drivers	Hardware interaction, interrupts, I/O handling
IPC	Pipes, message queues, shared memory, semaphores
Security	Permissions, capabilities, LSM (e.g. SELinux, AppArmor)
Timers	Kernel timers, scheduling, timekeeping
Module System	Loadable kernel modules and device drivers

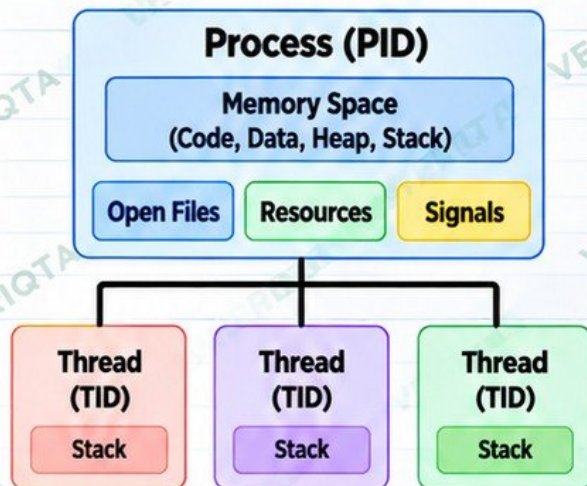
5. USEFUL COMMANDS

Area	Commands (Examples)
Kernel info	<code>uname -a</code> <code>cat /proc/version</code>
Scheduler	<code>cat /proc/sched_debug</code> <code>top</code> , <code>htop</code> , <code>ps -eo pid,cls,rtprio,comm</code>
Memory	<code>free -h</code> <code>cat /proc/meminfo</code> <code>vmstat 1</code> <code>cat /proc/pressure/memory</code>
VFS / Filesystem	<code>df -h</code> <code>mount column -t</code> <code>cat /proc/filesystems</code>
Networking	<code>ss -tulnp</code> <code>ip a</code> <code>ip route</code>
Drivers	<code>lsmod</code> <code>lspci</code> <code>dmesg grep -i driver</code>
IPC	<code>ipcs -a</code> <code>ls /dev/shm</code> <code>cat /proc/sys/kernel/ipc_*</code>

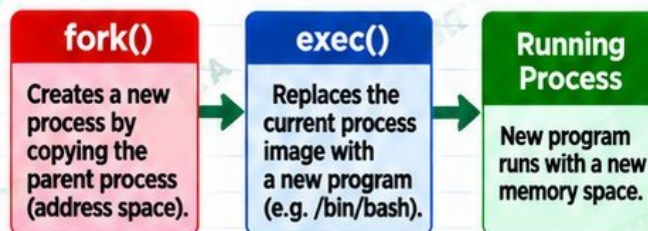


PID/TID, fork, exec, process states, context switching, zombies

1. PROCESS AND THREAD RELATIONSHIP



2. PROCESS CREATION AND EXECUTION



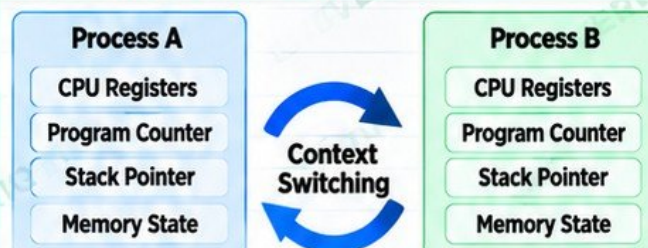
COMMON COMMANDS

<code>fork()</code>	# create a new process (called by shell, applications)
<code>exec()</code>	# replace process image with a new program
<code>ps tree</code>	# show process tree
<code>ps -ef</code>	# list processes
<code>ps -T -p <PID></code>	# show threads of a process

3. PROCESS STATES

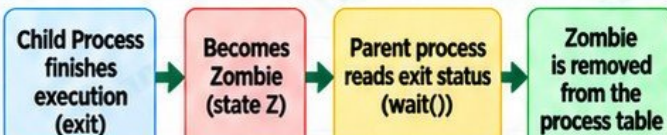
State	Code	Meaning
Running	R	Currently on CPU or ready to run
Sleeping	S	Waiting for an event (interruptible)
Uninterruptible Sleep	D	Waiting for I/O (not interruptible)
Stopped	T	Stopped (e.g. by signal)
Zombie	Z	Process finished, waiting for parent to collect exit status
Traced	t	Being debugged (e.g. strace)

4. CONTEXT SWITCHING



The kernel saves the current process state (registers, program counter, stack pointer) and loads the next process state so the CPU can run another process. This happens many times per second.

5. ZOMBIE PROCESSES



Why Zombies Happen

- Child process finishes but parent does not read its exit status.
- Entry remains in the process table until the parent calls `wait()`.



How to Check and Fix

- Check: `ps -ef | grep defunct`
- Find parent: `ps -o ppid= -p <zombie_pid>`
- Fix: restart the parent process or ensure it calls `wait()`.

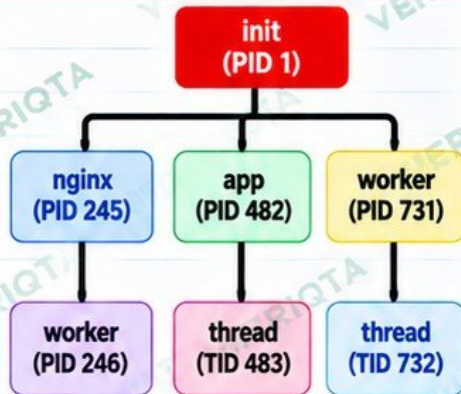
6. USEFUL COMMANDS AND EXAMPLES

Task	Command
List processes	<code>ps -ef</code>
Show threads of a process	<code>ps -T -p <PID></code>
Show process tree	<code>ps tree -p</code>
View process state	<code>ps -o pid,ppid,state,cmd</code>
Follow a process	<code>strace -p <PID></code>
Monitor in real time	<code>top # press H to show threads</code>
Check CPU context switches	<code>vmstat 1</code>
Check process details	<code>cat /proc/<PID>/status</code>
Check open files	<code>ls -l /proc/<PID>/fd</code>
Find zombie processes	<code>ps -ef grep defunct</code>



ps, pstree, top, /proc, strace, process relationships

1. LINUX PROCESS OVERVIEW



- Every process has a unique PID.
- A process can have multiple threads (TIDs).
- Processes form a tree (parent and child).
- PID 1 (systemd) is the root of all processes.
- Processes use system resources like CPU, memory, files and network.

2. ps - VIEW PROCESSES

basic process list

`ps aux`

specific columns

`ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu`

find a process

`ps aux | grep nginx`

Column	Meaning
PID	Process ID (unique)
PPID	Parent Process ID
CMD	Command that started the process
%CPU	CPU usage
%MEM	Memory usage

3. pstree - PROCESS TREE

show full process tree

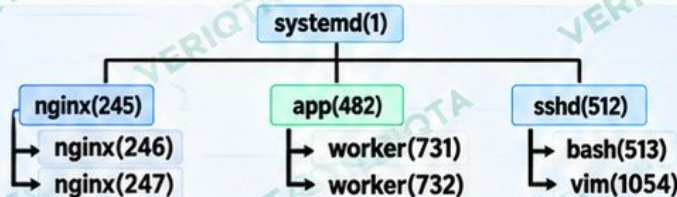
`pstree -p`

show tree for a specific process

`pstree -p 245`

show only processes of current user

`pstree -pu $USER`



4. top - REAL-TIME MONITORING

start top

`top`

sort by CPU usage (inside top press P)

sort by memory usage (inside top press M)

- Shows real-time CPU, memory and process activity.
- Press P to sort by CPU usage.
- Press M to sort by memory usage.
- Press k to kill a process (use with caution).
- Useful for identifying high resource usage processes.

5. /proc - PROCESS INFORMATION

view process details

`cat /proc/245/status`

view command line

`cat /proc/245/cmdline`

view open files

`ls -l /proc/<PID>/fd`

view environment variables

`cat /proc/245/environ`

- /proc provides real-time information about processes.
- Each process has a directory: /proc/PID.
- You can inspect status, open files, memory maps, environment variables and more.

6. strace - TRACE SYSTEM CALLS

trace a process

`strace -p 245`

trace and write to a file

`strace -o /tmp/trace.log -p 245`

trace a new command

`strace -f -o /tmp/trace.log nginx`

- Shows the system calls made by a process.
- Useful for debugging issues like file access, network connections and permission errors.
- Can be noisy, use filters for specific calls.

7. COMMON INVESTIGATION TASKS

Task	Command
Find a process by name	<code>ps aux grep <name></code>
View process tree	<code>pstree -p</code>
Check process resource usage	<code>top</code> or <code>ps -eo pid,cmd,%cpu,%mem</code>
View open files	<code>ls -l /proc/<PID>/fd</code>
Check process details	<code>cat /proc/<PID>/status</code>
Trace system calls	<code>strace -p <PID></code>
Find child processes	<code>ps --ppid <PID></code>
Kill a process (graceful)	<code>kill <PID></code>
Kill a process (force)	<code>kill -9 <PID></code>

8. KEY TAKEAWAYS

- 1 Use `ps` to quickly find and inspect processes.
- 2 Use `pstree` to understand process relationships.
- 3 Use `top` for real-time monitoring.
- 4 Use `/proc` for detailed process information.
- 5 Use `strace` to troubleshoot system call activity.
- 6 Understanding process relationships helps identify issues, resource usage and failure impact.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

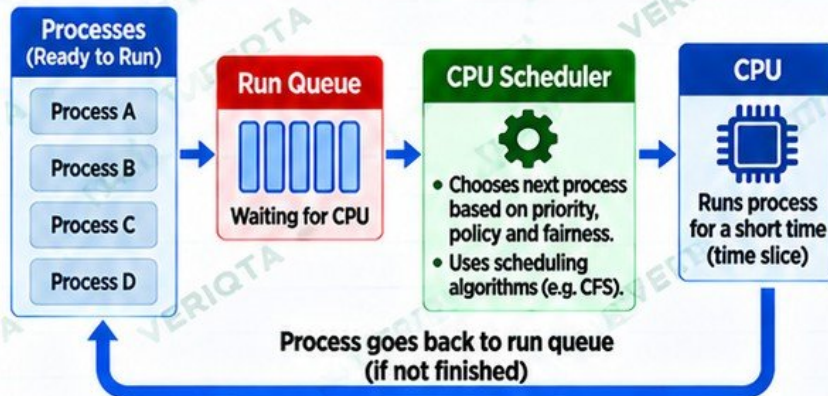


@veriqta

05

Run queues, load average, context switches, CPU saturation, steal time

1. HOW CPU SCHEDULING WORKS



2. KEY SCHEDULING CONCEPTS

- 1 Run Queue**
List of processes waiting for CPU.
- 2 Scheduler**
Decides which process runs next (Completely Fair Scheduler - CFS).
- 3 Time Slice**
Each process gets a small amount of CPU time.
- 4 Context Switch**
Saving current process state and loading another process state.
- 5 Preemption**
Scheduler can pause a running process to give CPU to another process.

3. CPU SCHEDULING METRICS

Metric	Meaning	High Value Indicates
Load Average	Average number of runnable processes over 1, 5, 15 minutes.	More processes waiting for CPU (possible CPU saturation).
Run Queue Length	Number of processes waiting for CPU right now.	CPU is busy, processes queued.
Context Switches	Number of switches between processes per second.	Very high values can indicate CPU pressure or many short-lived processes.
CPU Usage	Percentage of CPU time used (user, system, idle, iowait, etc).	High user or system usage indicates CPU saturation.
Steal Time	Time taken by hypervisor (in virtual machines).	High steal time means your VM is not getting enough CPU.

4. LOAD AVERAGE EXPLAINED

- Load average shows the average number of processes in the run queue (runnable or uninterruptible I/O).
- It is not a CPU percentage.
- It is reported for 1, 5 and 15 minutes.
- Compare load average with the number of CPU cores.

Example: 4 CPU Cores

Load Average			Interpretation
0.5	0.5	0.5	Low load (plenty of available CPU).
2.0	1.5	1.0	Moderate load (some waiting).
4.0	3.5	3.0	High load (CPU likely saturated).
8.0	7.0	6.0	Very high load (serious CPU pressure).

5. USEFUL COMMANDS AND EXAMPLES

Check load average

```
$ uptime
10:24:36 up 2 days, 3 users,
load average: 0.58, 0.42, 0.37
```

View CPU usage (real time)

```
$ top
%Cpu(s): 12.3 us, 5.2 sy, 0.0 ni,
80.1 id, 2.4 wa, 0.0 hi, 0.0 si,
0.0 st
```

Check detailed CPU stats

```
$ cat /proc/stat | head -n 1
cpu 3357 120 4310 1362393
98 0 78 0 0 0
```

View run queue length

```
$ cat /proc/loadavg
0.58 0.42 0.37 1/124 3095
```

Check context switches

```
$ vmstat 1 5
cs in r sy id wa st
1523 0 12 83 3 0
```

Check steal time (in VMs)

```
$ top
%Cpu(s): 10.1 us, 3.5 sy, 0.0 ni,
80.0 id, 1.2 wa, 0.0 hi, 5.2 st
```

6. IDENTIFYING CPU SATURATION

- Load average is consistently higher than the number of CPU cores.
- High %us (user) or %sy (system) in top.
- Long run queue length.
- High context switch rate.
- Processes are slow or unresponsive.

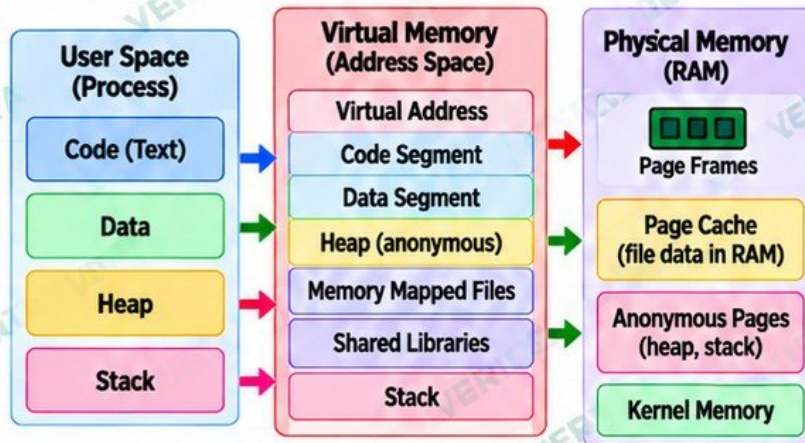
7. COMMON CAUSES OF HIGH CPU LOAD

- CPU-intensive applications (e.g. compression, video processing).
- Too many active processes or threads.
- Infinite loops or runaway processes.
- Inefficient code or queries.
- High context switching due to many short-lived processes.
- Noisy neighbors in virtualized environments (high steal time).



Virtual memory, pages, anonymous/file-backed memory, page cache

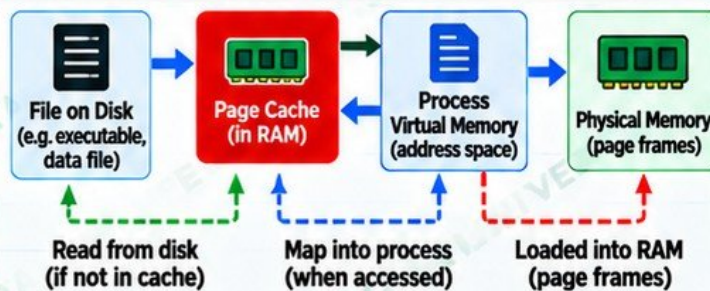
1. LINUX MEMORY ARCHITECTURE



2. KEY CONCEPTS

- 1 Virtual Memory**
Each process has its own virtual address space, independent of physical memory.
- 2 Memory Page**
Memory is divided into fixed-size pages (usually 4 KB on x86_64).
- 3 Anonymous Memory**
Memory not backed by a file (heap, stack).
- 4 File-Backed Memory**
Memory mapped to a file (executables, libraries, mmap).
- 5 Page Cache**
Keeps file data in RAM to speed up I/O.
- 6 Demand Paging**
Pages are loaded into RAM only when needed.

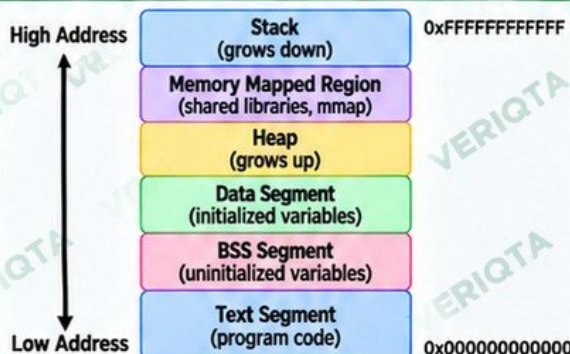
3. MEMORY PAGE FLOW



4. MEMORY TYPES COMPARISON

Memory Type	Backed By	Examples	Purpose
Anonymous Memory	Not a file	Heap, Stack	Process private memory
File-Backed Memory	A file on disk	Executables, shared libraries, mmap files	Shared and cacheable
Page Cache	File data in RAM	Recently read files	Speeds up file I/O, reused by multiple processes
Shared Memory	Not a file (special region)	POSIX shared memory, System V	IPC between processes

5. VIRTUAL MEMORY LAYOUT (EXAMPLE)



6. USEFUL COMMANDS

Task	Command
View memory usage	<code>free -h</code>
Detailed memory info	<code>cat /proc/meminfo</code>
View process memory	<code>cat /proc/<PID>/status</code>
View memory maps	<code>cat /proc/<PID>/maps</code>
Show page cache usage	<code>cat /proc/meminfo grep -i E "cached buff"</code>
View virtual memory areas	<code>pmmap -x <PID></code>
View shared libraries	<code>ldd <binary></code>
Monitor memory in real time	<code>htop</code>
Show slab memory (kernel)	<code>slabtop</code>
Show memory details	<code>vmstat 1</code>
Show page faults	<code>perf stat -e page-faults ...</code>

7. EXAMPLE: /proc/meminfo (KEY FIELDS)

```
$ cat /proc/meminfo | head -n 10
MemTotal: 16330708 kB # total physical memory
MemFree: 2345678 kB # free memory
MemAvailable: 4567890 kB # available memory
Buffers: 123456 kB # buffer cache
Cached: 3456789 kB # page cache
SwapTotal: 2097148 kB # total swap
SwapFree: 2097148 kB # free swap
```

8. PAGE CACHE DEMONSTRATION

```
# Clear page cache (as root)
sudo sync
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'

# Read a file and check cache
cat /var/log/syslog >/dev/null
cat /proc/meminfo | grep -i cached
```

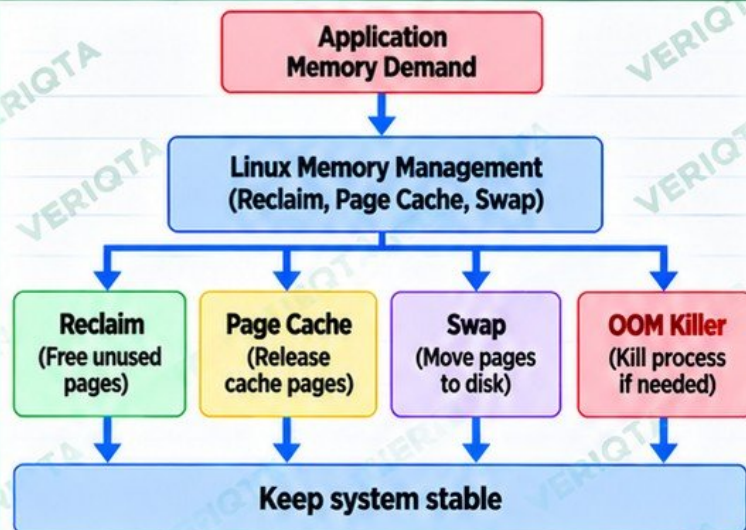
9. KEY TAKEAWAYS

- 1** Linux uses virtual memory to isolate processes.
- 2** Memory is managed in pages.
- 3** Anonymous memory is for heap and stack.
- 4** File-backed memory is shared and cached.
- 5** Page cache improves file I/O performance.
- 6** Pages are loaded on demand.
- 7** You can inspect memory using /proc and system tools.



Reclaim, swap, PSI, OOM killer, cgroups, diagnosing memory incidents

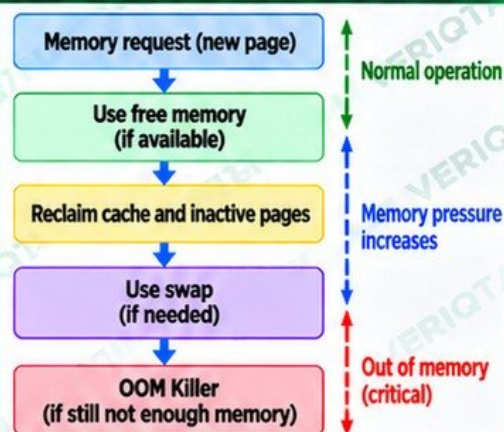
1. MEMORY PRESSURE OVERVIEW



2. KEY CONCEPTS

- 1 Reclaim**
Kernel frees unused pages (cache, buffers, inactive pages) to satisfy new memory requests.
- 2 Swap**
Moves less active pages to disk to free RAM.
- 3 PSI (Pressure Stall Information)**
Measures time tasks are stalled due to lack of CPU, memory or I/O resources.
- 4 OOM Killer**
Kills processes when the system cannot allocate memory.
- 5 cgroups (Memory Control)**
Limit and isolate memory usage per process/group.
- 6 Memory Incidents**
High memory usage, swap activity, OOM kills, application slowdowns.

3. LINUX MEMORY RECLAIM FLOW



4. PSI (PRESSURE STALL INFORMATION)

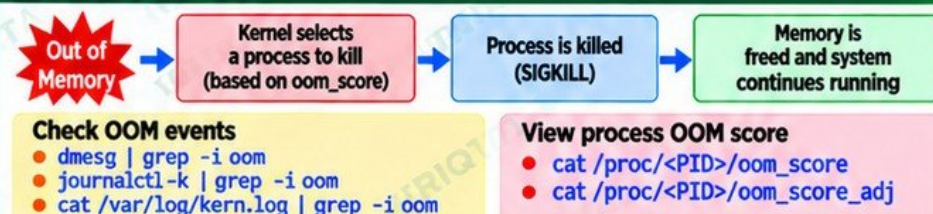
```
# view memory pressure
cat /proc/pressure/memory
```

Field	Meaning
some	Time tasks are partially stalled due to memory pressure.
full	Time tasks are completely stalled (direct reclaim or OOM).
avg10	Average over 10 seconds.
avg60	Average over 60 seconds.
avg300	Average over 300 seconds.

5. USEFUL COMMANDS

Task	Command
View memory usage	<code>free -h</code>
Real-time memory info	<code>top (press M)</code>
Detailed memory info	<code>cat /proc/meminfo</code>
View swap usage	<code>swapon --show</code>
View PSI	<code>cat /proc/pressure/memory</code>
Check OOM kills	<code>dmesg grep -i oom</code>
View cgroup memory	<code>cat /sys/fs/cgroup/memory.current</code>
Check process memory	<code>ps aux --sort=-%mem</code>
Monitor memory usage	<code>vmstat 1</code>
View slab memory	<code>slabtop</code>
Analyze memory maps	<code>pmap -x <PID></code>
Trace memory events	<code>perf stat -e page-faults ./app</code>

6. OOM KILLER



7. CGROUPS MEMORY CONTROL (V2)

```
# create a cgroup
mkdir /sys/fs/cgroup/myapp
echo 500M > /sys/fs/cgroup/myapp/memory.max
echo $PID > /sys/fs/cgroup/myapp/cgroup.procs
```

- memory.max sets the memory limit.
- memory.current shows current usage.
- Isolates and protects the system from runaway processes.

8. DIAGNOSING MEMORY INCIDENTS

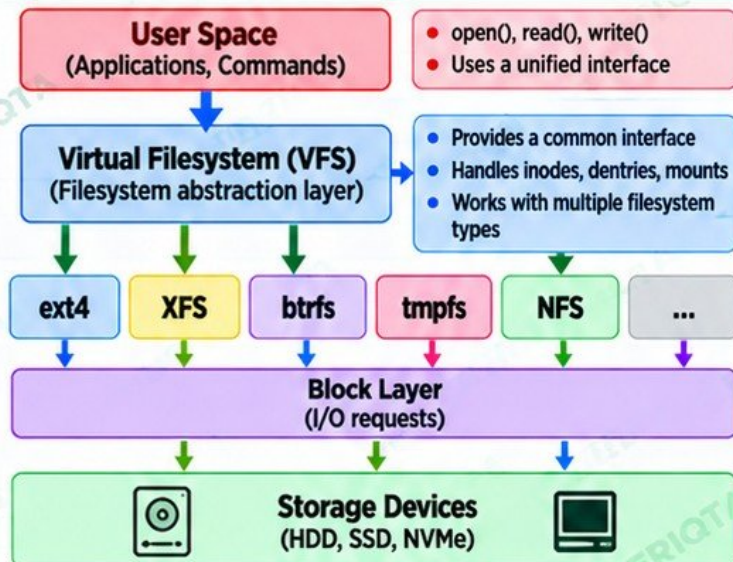
- 1 Check overall memory usage (free, top, vmstat).
- 2 Look for swap activity (swapon --show, vmstat).
- 3 Check PSI (cat /proc/pressure/memory).
- 4 Identify high memory processes (top, ps aux).
- 5 Check recent OOM kills (dmesg, journalctl).
- 6 Analyze application memory usage (pmap, smem).
- 7 Review cgroup limits (memory.current, memory.max).
- 8 Investigate memory leaks or abnormal growth.

9. KEY TAKEAWAYS

- Linux uses multiple techniques to manage memory (reclaim, swap, OOM).
- PSI shows real memory pressure.
- OOM killer protects the system but can terminate applications.
- cgroups help limit and isolate memory usage.
- Always investigate the root cause of memory incidents.

Inodes, dentries, mounts, filesystem abstraction, ext4/XFS concepts

1. LINUX FILESYSTEM LAYERS



2. KEY CONCEPTS

- 1 Inode**
Stores metadata about a file (permissions, owner, size, timestamps, block pointers).
- 2 Dentry**
Represents a filename in memory (directory entry).
- 3 VFS (Virtual Filesystem)**
Provides a unified interface for all filesystems.
- 4 Mount**
Attaches a filesystem to a directory in the directory tree.
- 5 Filesystem Type**
ext4, XFS, btrfs, tmpfs, NFS, etc.
- 6 Block Device**
Underlying storage device (e.g. /dev/sda1).
- 7 Filesystem Abstraction**
Applications use the same system calls regardless of filesystem type.

3. INODES AND DENTRIES

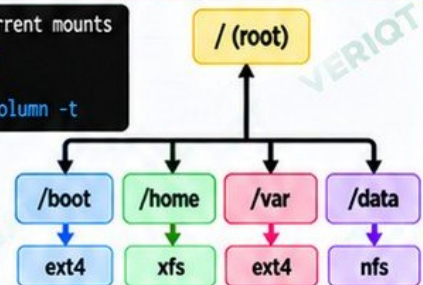
Inode (metadata)	
File type	regular file, directory, etc.
Permissions	rw-r--r--
Owner/Group	user and group IDs
Size	file size in bytes
Timestamps	atime, mtime, ctime
Block pointers	location of data on disk

Dentry (directory entry)	
Filename	name in directory
Inode reference	points to inode
Parent dentry	parent directory
Cache	kept in memory (dentry cache)



4. MOUNTS AND DIRECTORY TREE

```
# view current mounts
findmnt
# or
mount | column -t
```



- A filesystem is attached to a directory using a mount.
- Each mount can use a different filesystem type.
- All filesystems appear in a single unified directory tree through VFS.

5. COMMON FILESYSTEM TYPES

Filesystem	Key Features	Common Use Cases
ext4	Stable, widely used, good performance	General purpose (most Linux systems)
XFS	High performance, scalable, good for large files	Large files, databases, production systems
btrfs	Copy-on-write, snapshots, checksumming	Modern Linux, snapshots, containers
tmpfs	In-memory filesystem (RAM)	Temporary files, /tmp, high speed
NFS	Network filesystem	Shared storage across servers

6. USEFUL COMMANDS

Task	Command
View filesystems	lsblk -f
View mounts	findmnt
Show disk usage	df -h
Show inode usage	df -i
View filesystem type	stat -f -c %T /path
View inode details	stat /path/to/file
Show mount options	mount column -t
View all mounts	cat /proc/mounts
List open files on filesystem	lsof +D /path
Check disk and inode usage	du -sh /path
Show filesystem info	tune2fs -l /dev/sdX1 # ext4 xfs_info /dev/sdX1 # XFS

7. ext4 vs XFS (CONCEPTS)

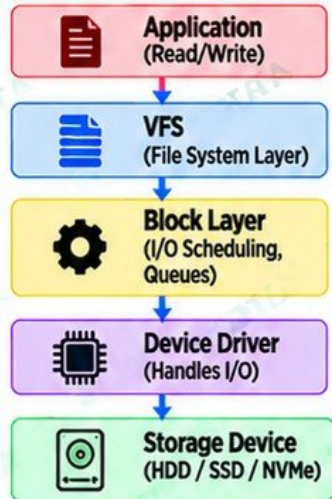
ext4	XFS
• Default on many distributions • Stable and reliable • Good general performance • Supports large files and volumes • Widely supported tools	• High performance and scalable • Good for large files and high I/O • Online grow capability • Parallel I/O and multitasking • Common in enterprise systems

8. KEY TAKEAWAYS

- Linux uses the VFS to provide a unified filesystem interface.
- Inodes store metadata, dentries store filenames.
- Different filesystems (ext4, XFS, etc.) can be mounted anywhere in the directory tree.
- Each filesystem type has different strengths and use cases.
- Understanding inodes, mounts and filesystem types is essential for troubleshooting.

IOPS, throughput, latency, queues, I/O wait, iostat, lsblk

1. DISK I/O OVERVIEW

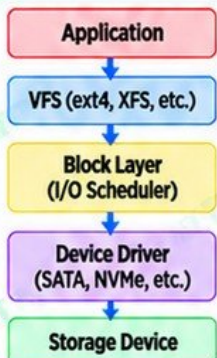


- Applications read and write data through the Linux filesystem.
- The block layer manages I/O requests using schedulers and queues.
- Device drivers send requests to the storage device (HDD, SSD, NVMe).
- Performance depends on IOPS, throughput, latency and queue depth.
- Slow disk I/O can cause high I/O wait, slow applications and system bottlenecks.

2. KEY STORAGE PERFORMANCE METRICS

Metric	Meaning
IOPS	Number of input/output operations per second. (e.g. 4K read/write operations)
Throughput	Amount of data transferred per second (e.g. MB/s or GB/s)
Latency	Time taken to complete a single I/O request (e.g. in milliseconds)
Queue Depth	Number of I/O requests waiting to be processed
I/O Wait	Time CPU spends waiting for disk I/O to complete
Read/Write	Amount of data read or written
Utilization	Percentage of time the device is busy handling I/O

3. LINUX STORAGE STACK



- VFS provides a common interface for all filesystems.
- Block layer manages I/O requests and scheduling.
- Different I/O schedulers (e.g. mq-deadline, none) affect performance.
- Hardware (HDD, SSD, NVMe) has different characteristics for IOPS, throughput and latency.

4. COMMON STORAGE DEVICES

HDD	SSD (SATA)	NVMe
• Lower IOPS • Higher latency • Good for large storage	• Higher IOPS • Lower latency • Common in servers	• Very high IOPS • Very low latency • Best performance

5. USEFUL COMMANDS

Task	Command
List block devices	<code>lsblk -f</code>
View disk usage	<code>df -h</code>
Detailed disk info	<code>fdisk -l</code>
View partitions	<code>lsblk</code>
Monitor I/O stats	<code>iostat -x 1</code>
View real-time I/O	<code>iotop</code>
Check device info	<code>cat /proc/diskstats</code>
View scheduler	<code>cat /sys/block/sdX/queue/scheduler</code>
Check queue depth	<code>cat /sys/block/sdX/queue/nr_requests</code>

6. EXAMPLE: lsblk OUTPUT

```

$ lsblk -f
NAME        FSTYPE  SIZE  MOUNTPOINT
sda          500G
├─sda1      ext4    100G  /
├─sda2      ext4    200G  /data
└─sda3      swap    200G  [SWAP]
nvme0n1      1T
└─nvme0n1p1 1T     /backup
  
```

7. EXAMPLE: iostat OUTPUT

```

$ iostat -x 1
Device  r/s    w/s    rMB/s  wMB/s  await  util%
sda     12.4   8.1    0.5    1.2    4.3    12.5
sdb      0.2    0.1    0.0    0.0    0.6     1.0
nvme0n1 120.5  95.3   5.2    4.8    0.8    35.6
  
```

- r/s, w/s = read/write operations per second (IOPS)
- rMB/s, wMB/s = read/write throughput
- await = average I/O latency (ms)
- util% = disk utilization (how busy the device is)

8. EXAMPLE: iotop OUTPUT

```

$ iotop -b -n 1
PID  USER  DISK  READ  DISK  WRITE  COMMAND
1234  root  12.5  M/s  0.0  B/s  mysqld
2345  app   0.0  B/s  8.3  M/s  java
3456  root  1.2  M/s  0.0  B/s  rsync
4567  www   0.0  B/s  4.1  M/s  nginx
  
```

- Shows real-time disk I/O per process.
- Useful for finding which process is using disk I/O.
- Can help identify I/O heavy applications.

9. COMMON PERFORMANCE ISSUES

Issue	Symptoms	Possible Cause
High I/O wait	System feels slow, high iowait in top	Slow disk, high latency, I/O queue full
Low IOPS	Applications slow	HDD limits, insufficient IOPS, wrong storage type
High latency	Slow read/write	Disk overloaded, hardware issues
High utilization	Disk at 100%	Too many I/O requests
Low throughput	Slow data transfer	Network storage, old disk, limits
Queue depth high	I/O backlog	Too many pending requests
Frequent errors	I/O errors in logs	Faulty disk, bad cables, hardware failure

10. TUNING TIPS

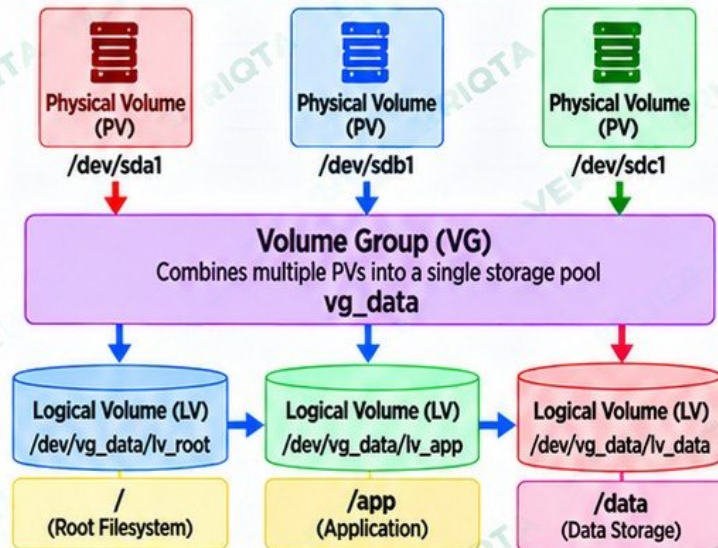
- Use SSD or NVMe for high IOPS workloads.
- Choose the right filesystem (ext4, XFS, etc.).
- Use appropriate I/O scheduler for your workload.
- Monitor disk I/O regularly with iostat and iotop.
- Check and optimize queue depth.
- Avoid running I/O heavy tasks on the same disk.
- Use separate disks for databases, logs and app data.
- Check for storage latency and errors.
- Use RAID or distributed storage for high availability.

11. KEY TAKEAWAYS

- 1 Disk performance is measured by IOPS, throughput and latency.
- 2 Linux uses the VFS and block layer to manage I/O.
- 3 Tools like lsblk, iostat and iotop help investigate disk issues.
- 4 High I/O wait indicates storage bottlenecks.
- 5 Different storage types (HDD, SSD, NVMe) have different performance characteristics.
- 6 Monitor and tune storage for better application performance.

PV → VG → LV, expansion, snapshots, operational trade-offs

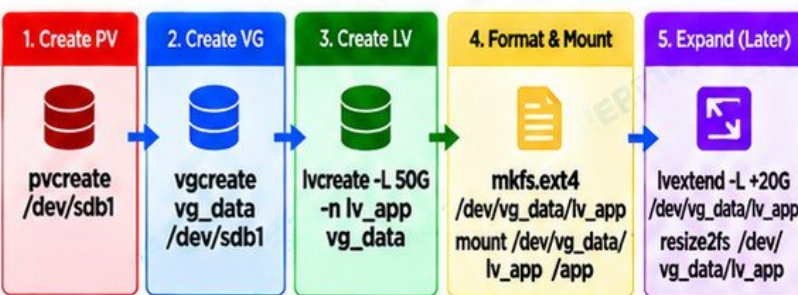
1. LVM ARCHITECTURE OVERVIEW



2. KEY CONCEPTS

- Physical Volume (PV)**
A disk or partition initialized for LVM (e.g. /dev/sda1).
- Volume Group (VG)**
Combines one or more PVs into a single storage pool.
- Logical Volume (LV)**
Virtual volume created from the VG (like a partition).
- Expansion**
Increase LV size and grow the filesystem without downtime.
- Snapshots**
Point-in-time copy of an LV for backup or testing.
- Operational Trade-offs**
Flexibility vs complexity, performance impact, snapshot space usage.

3. LVM WORKFLOW (COMMON OPERATIONS)



4. IMPORTANT COMMANDS

Task	Command
Create PV	<code>pvcreate /dev/sdb1</code>
View PVs	<code>pvs</code>
Create VG	<code>vgcreate vg_data /dev/sdb1</code>
View VGs	<code>vgs</code>
Create LV	<code>lvcreate -L 50G -n lv_app vg_data</code>
View LVs	<code>lvs</code>
Extend LV	<code>lvextend -L +20G /dev/vg_data/lv_app</code>
Reduce LV	<code>lvreduce -L -10G /dev/vg_data/lv_app</code>
Create snapshot	<code>lvcreate -s -L 10G -n lv_app_snap /dev/vg_data/lv_app</code>
Remove LV	<code>lvremove /dev/vg_data/lv_app</code>
View usage	<code>df -h</code>

5. LVM SNAPSHOTS



- Snapshot is a point-in-time copy (COW).
- Useful for backups, testing and rollbacks.
- Consumes space only when data changes.
- Can be removed when no longer needed.

6. EXPANDING A LOGICAL VOLUME (EXAMPLE)

```

# 1. Extend the LV by 20G
lvextend -L +20G /dev/vg_data/lv_app
# 2. Grow the filesystem (ext4)
resize2fs /dev/vg_data/lv_app
# 3. Verify new size
df -h /app
# 4. Check LV details
lvs -o +size,lv_path
  
```

Output (example)

```

Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/    78G  1.2G  66G   2% /app
vg_data-lv_app
  
```

7. LVM VS TRADITIONAL PARTITIONS

Feature	LVM	Traditional Partitions
Flexibility	High (easy to resize)	Low (fixed size)
Snapshots	Yes	No (requires other tools)
Manage multiple disks	Yes (volume groups)	No
Online resize	Yes (in most cases)	Limited / No
Complexity	Higher	Lower
Best use case	Servers, scalable storage	Simple systems

8. OPERATIONAL TRADE-OFFS

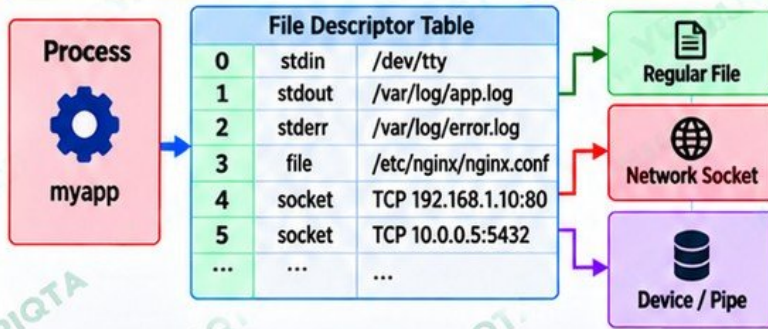
- **Pros:**
 - Flexible storage management
 - Easy expansion and resizing
 - Snapshots for backup and testing
 - Combine multiple disks
 - Ideal for servers and growing workloads
- **Cons:**
 - Slight performance overhead
 - More complex than traditional partitions
 - Snapshots consume extra space (COW)
 - Requires monitoring and management
 - Not always necessary for simple systems

9. KEY TAKEAWAYS

- LVM uses PV → VG → LV architecture.
- You can easily expand LVs and grow filesystems.
- Snapshots provide point-in-time backups.
- LVM offers flexibility but has some performance overhead.
- Choose LVM for scalable, production systems.

1. WHAT ARE FILE DESCRIPTORS?

- A file descriptor (FD) is a number that represents an open file or resource.
- Everything is a file in Linux: regular files, directories, sockets, pipes, devices.
- Each process has its own FD table that maps FD numbers to open files.
- Sockets (network connections) are also file descriptors.



2. KEY CONCEPTS

- File Descriptor (FD)**
A number that represents an open file, directory, socket, pipe or device.
- FD Table**
Each process has its own table mapping FD numbers to open files.
- Sockets as FDs**
Network connections are file descriptors.
- FD Limits**
The maximum number of open files per process and system.
- FD Exhaustion**
When a process or system runs out of available file descriptors.
- /proc/PID/fd**
Shows all open file descriptors for a process.

3. COMMON FD TYPES

FD Type	Example	Description
Regular File	/etc/passwd	Normal file on disk
Directory	/var/log	Directory handle
Socket	TCP/UDP	Network connection
Pipe	pipe:[12345]	Inter-process communication
Device	/dev/sda	Device file
Symbolic Link	/etc/hosts	Symlink
Anonymous	anon_inode: [eventfd]	Kernel internal resource

4. VIEW OPEN FILE DESCRIPTORS

```
# List open files for a process
ls -l /proc/1234/fd

# Show what each FD points to
readlink -f /proc/1234/fd/*

# View current shell FDs
ls -l /proc/$$/fd
```

Example output:

```
0 -> /dev/pts/0
1 -> /var/log/app.log
2 -> /var/log/error.log
3 -> /etc/nginx/nginx.conf
4 -> socket:[123456]
5 -> socket:[123457]
```

5. USEFUL COMMANDS

Task	Command
List open files	<code>lsof -p <PID></code>
List sockets	<code>lsof -i -p <PID></code>
Count open files	<code>ls /proc/<PID>/fd wc -l</code>
View FD details	<code>cat /proc/<PID>/fdinfo/<FD></code>
Show system limits	<code>cat /proc/sys/fs/file-max</code>
Show process limit	<code>ulimit -n</code>
Show all open files	<code>lsof</code>
List network FDs	<code>ss -p</code>
Find which process	<code>lsof /path/to/file</code>

6. FD LIMITS

Process Limit (ulimit)

```
# View current llimit
ulimit -n

# Set a higher limit
ulimit -n 65536
```

Controls the maximum open files for the current process.

System Limit

```
# View system-wide limit
cat /proc/sys/fs/file-max

# Increase temporarily
echo 2097152 > /proc/sys/fs/file-max
```

Controls the total number of open files on the system.

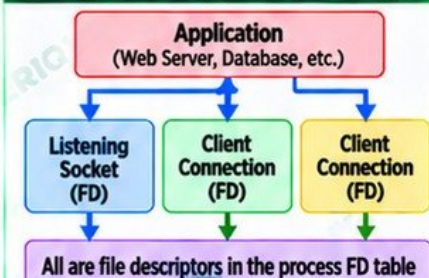
7. FD EXHAUSTION (EXAMPLE)

```
# Simulate opening many files
for i in {1..100000}; do
  exec {fd}>/dev/null
done
```

Common errors:

- too many open files
- cannot create socket
- accept: Too many open files
- application fails to start

8. SOCKETS AS FILE DESCRIPTORS



9. CHECK FD USAGE (REAL EXAMPLE)

```
# Check number of open files for nginx
lsof -p $(pgrep nginx) | wc -l

# View open network connections
lsof -i -p $(pgrep nginx)

# Show FD details
cat /proc/$(pgrep nginx)/fdinfo/3
```

10. BEST PRACTICES

- Set appropriate ulimit and system limits.
- Monitor open file descriptors for critical applications.
- Close files and sockets properly in applications.
- Use connection pooling for databases and services.
- Monitor with lsof, ss and /proc.
- Set alerts for high FD usage.
- Understand that sockets, pipes and files all count towards the same limit.

11. KEY TAKEAWAYS

- 1 File descriptors represent all open files, sockets, pipes and devices.
- 2 Each process has its own FD table.
- 3 Sockets are file descriptors.
- 4 Use ulimit and system limits to control maximum open files.
- 5 Monitor FD usage to prevent exhaustion.
- 6 Use lsof and /proc/PID/fd for investigation.



UID/GID, modes, ACLs, setuid/setgid, sticky bit, ownership boundaries

1. LINUX PERMISSION MODEL



Linux uses permissions to control who can access files and directories.

Owner
(User)
UID

Group
GID

Others
(Everyone else)

Permission	Symbol	Meaning
Read	r	View file contents / list directory
Write	w	Modify file contents / create, delete
Execute	x	Run a file / enter a directory

2. PERMISSION MODES (NUMERIC & SYMBOLIC)

Symbolic Representation

-rwxr-xr--

Owner
rwx
(7)

Group
r-x
(5)

Others
r--
(4)

Numeric Representation

754

7 = 4 + 2 + 1 (rwx)

5 = 4 + 0 + 1 (r-x)

4 = 4 + 0 + 0 (r--)

- Use chmod to change permissions.
- Use ls -l to view permissions.

3. FILE vs DIRECTORY PERMISSIONS

File Permissions

Control what you can do with the file content.



- r = read file content
- w = modify file content
- x = execute file

Directory Permissions

Control what you can do inside the directory.



- r = list files (ls)
- w = create, delete, rename
- x = enter directory (cd)

4. UID, GID AND OWNERSHIP

```
# View current user and group IDs
uid -u      # UID
id -g       # GID
id          # Both UID and GID
```

```
# View file ownership
ls -l file.txt
-rw-r--r-- 1 veriqta devops 1024 Sep 24 10:00 file.txt
```

- UID identifies the user.
- GID identifies the primary group.
- Use chown to change ownership.
- Use chgrp to change group.

5. ACCESS CONTROL LISTS (ACLs)

ACLs provide more flexible permissions than the standard owner/group/others model.

```
# View ACLs
getfacl file.txt

# Set ACL (give user read permission)
setfacl -m u:john:r file.txt

# Remove ACL
setfacl -x u:john file.txt

# Default ACL for new files in directory
setfacl -d -m u:john:r /shared
```

6. SPECIAL PERMISSIONS

Permission	Octal	Applies To	What It Does
setuid	4	Files	Run as file owner's user ID.
setgid	2	Files	Run as file owner's group ID.
setgid	2	Directories	New files inherit the directory's group.
sticky bit	1	Directories	Only file owner can delete or rename files (e.g. /tmp).

7. EXAMPLES

```
# Set setuid on a file
sudo chmod 4755 /usr/bin/myapp

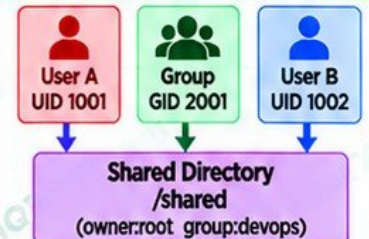
# Set setgid on a file
sudo chmod 2755 /usr/bin/myapp

# Set setgid on a directory
sudo chmod 2775 /shared

# Set sticky bit on a directory
sudo chmod 1777 /tmp

# View special permissions
ls -l /tmp
```

8. OWNERSHIP BOUNDARIES



- Ownership and group membership define access boundaries.
- Use groups and ACLs for controlled sharing.
- Avoid giving unnecessary root ownership.

9. USEFUL COMMANDS

Task	Command
View file permissions	ls -l
Change permissions (numeric)	chmod 755 file.txt
Change permissions (symbolic)	chmod u+x file.txt
Change ownership	chown user:group file.txt
Change group	chgrp devops file.txt
View ACLs	getfacl file.txt
Set ACL	setfacl -m u:john:r file.txt
Remove ACL	setfacl -x u:john file.txt
View user ID	id
Find files by permission	find / -perm 755 2>/dev/null

10. COMMON PERMISSION VALUES

Value	Symbolic	Meaning	Use Case
644	rw-r--r--	Read/write for owner, read for others	Regular files
755	rwxr-xr-x	Full access for owner, read/execute for others	Executable files/directories
700	rwx-----	Only owner has access	Private files
770	rwxrwx---	Owner and group only	Shared within team
777	rwxrwxrwx	Everyone has full access	Avoid (insecure)

11. KEY TAKEAWAYS

- 1 Linux uses UID, GID and permission modes to control access.
- 2 Directories and files use permissions differently.
- 3 ACLs allow more granular access control.
- 4 setuid, setgid and sticky bit provide special behaviors.
- 5 Ownership and group boundaries are critical for security.
- 6 Always apply the principle of least privilege.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



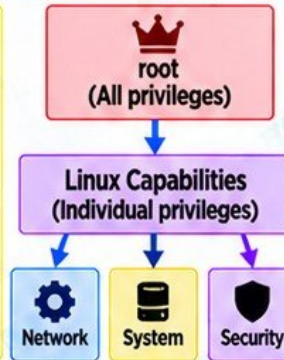
@veriqta

13

Breaking root privileges into capabilities, least privilege, containers

1. WHAT ARE LINUX CAPABILITIES?

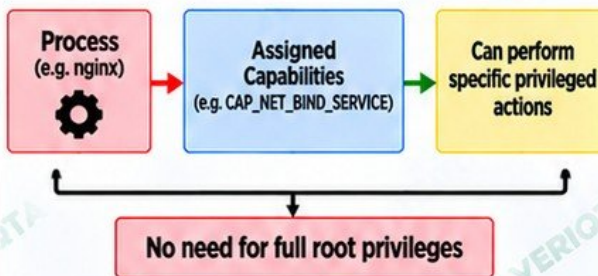
- Linux capabilities break the all-powerful root privileges into smaller, individual privileges.
- They allow a process to perform specific privileged actions without running as full root.
- This enables the principle of least privilege.
- Widely used in modern systems and containers to improve security.



2. KEY CONCEPTS

- Capability**
A single, specific privilege (e.g. CAP_NET_BIND_SERVICE).
- Process Capabilities**
Assigned to a process or executable instead of full root.
- Types of Capabilities**
Permitted, Effective, Inheritable, Bounding, Ambient.
- Least Privilege**
Grant only the capabilities an application needs.
- Security**
Reduces risk compared to giving full root access.
- Containers**
Used heavily in containers (Docker/Kubernetes) to run processes with minimal privileges.

3. HOW CAPABILITIES WORK



4. COMMON LINUX CAPABILITIES

Capability	Description	Common Use Case
CAP_NET_BIND_SERVICE	Allow binding to low ports (< 1024)	Web servers (port 80/443)
CAP_SYS_ADMIN	Perform system administration tasks	Mount filesystems, namespaces
CAP_SYS_TIME	Change system time	Time synchronization services
CAP_NET_ADMIN	Network configuration	Manage network interfaces, routing, firewall
CAP_DAC_OVERRIDE	Bypass file read/write permissions	Access restricted files
CAP_CHOWN	Change file ownership	Service management
CAP_SETUID	Set user ID	Manage process user privileges

5. VIEW PROCESS CAPABILITIES

```
# View capabilities of a running process
ps -o pid,comm,uid,gid,cap_eff,cap_prm --no-headers
# Or use capsh
capsh --print
# Check capabilities of a specific process
cat /proc/$(pidof nginx)/status | grep Cap
```

6. SET CAPABILITIES ON EXECUTABLES

```
# Give nginx the ability to bind to port 80
sudo setcap 'cap_net_bind_service=+ep' /usr/sbin/nginx
# Verify the capability
getcap /usr/sbin/nginx
# Remove the capability
sudo setcap -r /usr/sbin/nginx
```

7. CAPABILITY TYPES

Type	Description	When Used
Permitted (P)	Capabilities the process may use	Set by kernel or file
Effective (E)	Capabilities currently active	Used for privilege checks
Inheritable (I)	Capabilities passed to child processes	For exec and child processes
Bounding (B)	Maximum capabilities for the process	Cannot exceed this set
Ambient (A)	Capabilities preserved across exec	Used in modern systems

8. CAPABILITIES IN CONTAINERS



Docker

- Drop unnecessary capabilities using `--cap-drop`.
- Add only required capabilities using `--cap-add`.
- Run containers with least privilege.

```
# Run nginx with only the required capability
docker run --rm \
  --cap-add=NET_BIND_SERVICE \
  nginx
```



Kubernetes

- Use `securityContext` to control capabilities.
- Drop all capabilities by default and add only required ones.
- Helps improve container security.

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
```

9. BEST PRACTICES

- Use capabilities instead of full root access.
- Grant only the minimum required capabilities.
- Drop unnecessary capabilities in containers.
- Regularly audit capabilities assigned to processes.
- Combine with other security controls (namespaces, cgroups, seccomp).
- Follow the principle of least privilege.

10. KEY TAKEAWAYS

- Linux capabilities split root privileges into smaller, specific privileges.
- They allow least privilege access for processes and applications.
- Essential for modern systems and container security.
- Use tools like `getcap`, `setcap`, and `capsh` to work with capabilities.
- Always follow the principle of least privilege.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

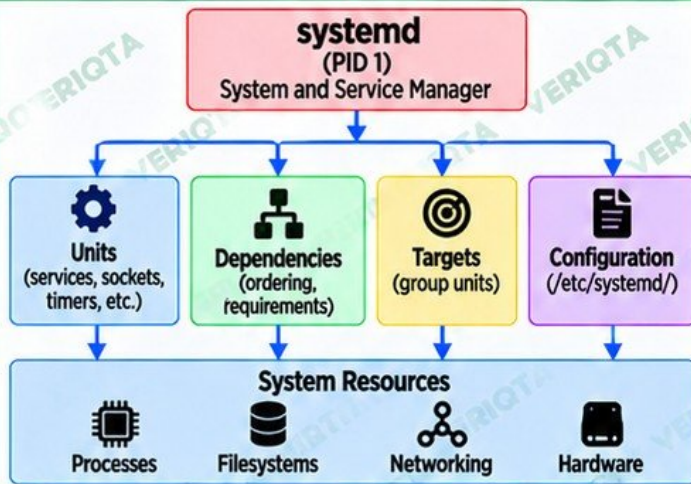


@veriqta

14

Units, dependencies, targets, ordering, service lifecycle

1. SYSTEMD ARCHITECTURE OVERVIEW



2. KEY COMPONENTS

Component	Description
systemd (PID 1)	The init system and service manager. First process that starts during boot.
Units	Configuration files that define resources (services, sockets, timers, mounts, etc.).
Dependencies	Define the order and requirements between units (Requires, Wants, Before, After).
Targets	Group of units that represent a system state (e.g. multi-user.target).
Journal	Logging component that collects logs from services.
systemctl	Command-line tool to manage systemd.
Configuration	Unit files in /etc/systemd/system/ or /lib/systemd/system/.

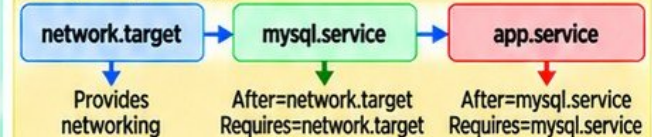
3. COMMON UNIT TYPES

Unit Type	File Extension	Purpose	Example
Service	.service	Manage a long-running process	nginx.service
Socket	.socket	Listen for incoming connections	docker.socket
Timer	.timer	Run a task on a schedule	backup.timer
Target	.target	Group multiple units	multi-user.target
Mount	.mount	Mount a filesystem	data.mount
Automount	.automount	Automatically mount on access	data.automount
Path	.path	Trigger actions on file system changes	monitor.path
Slice	.slice	Group and limit resources	app.slice
Scope	.scope	Manage externally started processes	user@1000.scope

4. UNIT DEPENDENCIES (ORDERING)

Common Directives	Meaning
Requires=	Unit must be started (strong dependency).
Wants=	Unit should be started (weak dependency).
Before=	Start this unit before the specified unit.
After=	Start this unit after the specified unit.
Conflicts=	Cannot be active at the same time.
PartOf=	Tie the lifecycle to another unit.
BindsTo=	Stop this unit when the other unit stops.

Example Dependency Flow



5. SERVICE LIFECYCLE



7. COMMON SYSTEMCTL COMMANDS

Task	Command
Start a service	systemctl start nginx
Stop a service	systemctl stop nginx
Restart a service	systemctl restart nginx
Enable at boot	systemctl enable nginx
Disable at boot	systemctl disable nginx
Check service status	systemctl status nginx
List all units	systemctl list-units --type=service
View dependencies	systemctl list-dependencies nginx
View unit file	systemctl cat nginx

6. EXAMPLE SERVICE UNIT FILE

```

/etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
After=network.target
Requires=network.target
[Service]
Type=simple
ExecStart=/usr/local/bin/myapp
Restart=always
User=apuser
Group=appgroup
[Install]
WantedBy=multi-user.target
  
```

Explanation

- 1 Description of the service.
- 2 Start after network is ready.
- 3 Command to start the application.
- 4 Restart automatically on failure.
- 5 Run as a non-root user.
- 6 Enable the service in the multi-user.target.

8. COMMON TARGETS

Target	Purpose
default.target	Default target (symlink)
multi-user.target	Normal multi-user system
graphical.target	Multi-user with GUI
rescue.target	Minimal system for repair
emergency.target	Very minimal system (root only)
reboot.target	Reboot the system
poweroff.target	Power off the system

9. BEST PRACTICES

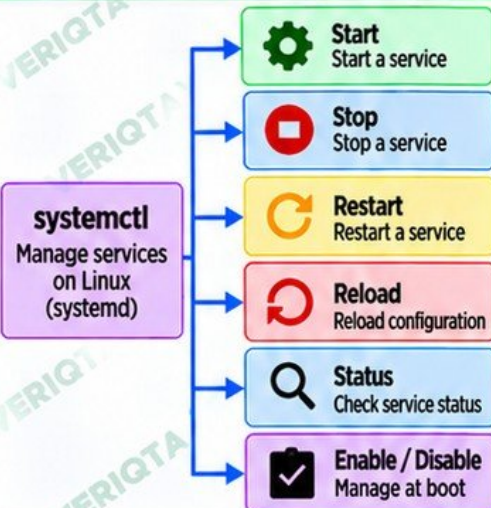
- ✓ Use proper dependencies (Requires/After).
- ✓ Keep services lightweight and focused.
- ✓ Use non-root users for security.
- ✓ Set appropriate restart policies.
- ✓ Use resource limits with slices.
- ✓ Monitor services using journal.
- ✓ Keep unit files in /etc/systemd/system/.

10. KEY TAKEAWAYS

- 1 systemd is PID 1 and manages the system.
- 2 Units define resources and services.
- 3 Upstream control startup order.
- 4 Targets group multiple units.
- 5 Use systemctl to manage services.
- 6 Understand the service lifecycle and logs.

systemctl, restart behavior, dependencies, failed units, service recovery

1. SERVICE MANAGEMENT OVERVIEW



2. COMMON SYSTEMCTL COMMANDS

Task	Command
Start a service	<code>systemctl start nginx</code>
Stop a service	<code>systemctl stop nginx</code>
Restart a service	<code>systemctl restart nginx</code>
Reload configuration	<code>systemctl reload nginx</code>
Check status	<code>systemctl status nginx</code>
Enable at boot	<code>systemctl enable nginx</code>
Disable at boot	<code>systemctl disable nginx</code>
List all services	<code>systemctl list-units --type=service</code>
List running services	<code>systemctl list-units --type=service --state=running</code>
List failed services	<code>systemctl list-units --type=service --state=failed</code>

3. SERVICE STATES

- Active (running)**
Service is running normally.
- Active (exited)**
Service ran and exited successfully (e.g. one-time tasks).
- Inactive (dead)**
Service is not running.
- Failed**
Service failed to start or crashed.
- Activating**
Service is starting.
- Deactivating**
Service is stopping.

4. RESTART BEHAVIOR

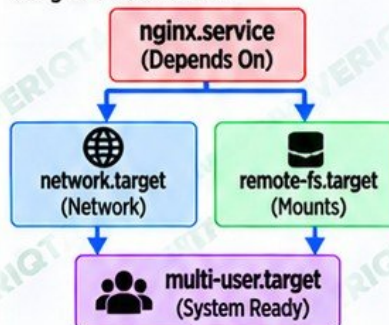
systemd can automatically restart failed services based on the unit file configuration.

```
[Service]
Restart=on-failure
RestartSec=5
StartLimitIntervalSec=60
StartLimitBurst=3
```

Option	Meaning
no	Do not restart.
on-success	Restart only on clean exit.
on-failure	Restart when service fails.
on-abnormal	Restart on abnormal exit.
always	Always restart.

5. SERVICE DEPENDENCIES

Services can depend on other services using unit file directives.



Common Dependency Directives

- **Requires=** (Must start)
- **Wants=** (Should start)
- **After=** (Start after)
- **Before=** (Start before)
- **Conflicts=** (Cannot run together)

6. CHECKING AND TROUBLESHOOTING

- Check service status**
`systemctl status nginx`
- View logs for a service**
`journalctl -u nginx -f`
- Check why a service failed**
`systemctl --failed`
`systemctl status nginx`
- View service dependencies**
`systemctl list-dependencies nginx`
- Check recent service failures**
`journalctl -u nginx --since "1 hour ago"`
- Verify unit file**
`systemctl cat nginx`
- Check service configuration**
`systemctl show nginx`
- Test configuration (if applicable)**
`nginx -t`

7. RECOVERING FAILED SERVICES

- 1 Check Status**
`systemctl status nginx`
- 2 View Logs**
`journalctl -u nginx`
- 3 Identify Cause**
Check config, dependencies, permissions
- 4 Fix Issue**
Update config, restore files, fix permissions
- 5 Restart Service**
`systemctl restart nginx`

Common Causes of Service Failure

- Invalid configuration file
- Missing dependencies
- Port already in use
- Permission issues
- Resource limits (memory, file descriptors)
- Application errors
- Incorrect user or group
- Network or filesystem not available

8. REAL EXAMPLE: NGINX SERVICE

- # 1. Check status**
`systemctl status nginx`
- # 2. View logs**
`journalctl -u nginx -f`
- # 3. Test configuration**
`nginx -t`
- # 4. Restart service**
`systemctl restart nginx`
- # 5. Enable at boot**
`systemctl enable nginx`
- # 6. Check dependencies**
`systemctl list-dependencies nginx`

9. BEST PRACTICES

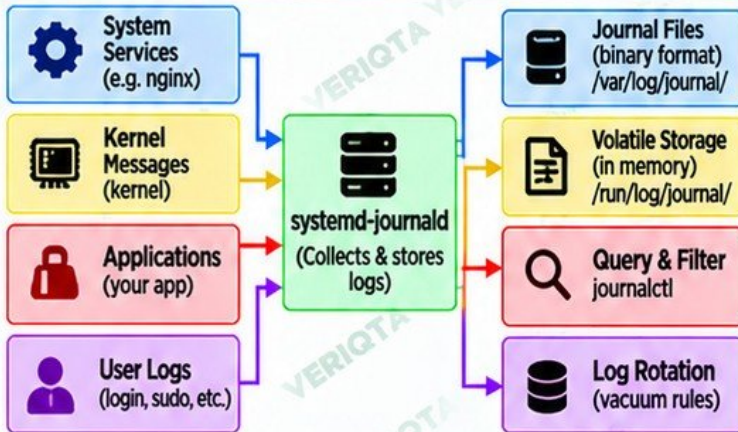
- ✓ Use meaningful service names.
- ✓ Configure proper restart policies.
- ✓ Monitor service logs regularly.
- ✓ Set resource limits (CPU, memory, etc.).
- ✓ Use non-root users where possible.
- ✓ Ensure proper dependencies.
- ✓ Use health checks for critical services.
- ✓ Keep unit files in `/etc/systemd/system/`.
- ✓ Document custom configurations.
- ✓ Monitor and alert on failed services.

10. KEY TAKEAWAYS

- 1** Use systemctl to manage services.
- 2** Understand service states and logs.
- 3** Configure restart behavior for resilience.
- 4** Use dependencies to control startup order.
- 5** Investigate and fix failed units quickly.
- 6** Follow best practices for production.

Journal architecture, persistent logs, filtering, rotation, incident evidence

1. JOURNALD ARCHITECTURE



2. KEY CONCEPTS

- systemd-journald**
Central logging service in modern Linux systems.
- Journal Files**
Logs stored in binary format for fast, indexed access.
- Persistent Logs**
Stored in `/var/log/journal/` for long-term retention.
- Volatile Logs**
Stored in memory at `/run/log/journal/` (lost on reboot).
- Filtering**
Use `journalctl` to filter logs by time, unit, priority, etc.
- Rotation**
Logs are automatically rotated and vacuumed.
- Incident Evidence**
Provides reliable, tamper-resistant logs for troubleshooting.

3. JOURNALD VS TRADITIONAL LOGGING

Feature	Journald	Traditional (rsyslog)
Log Format	Binary (indexed)	Plain text
Storage Location	<code>/var/log/journal/</code>	<code>/var/log/*.log</code>
Query Tool	<code>journalctl</code>	<code>grep</code> , <code>less</code> , <code>cat</code>
Structured Logs	Yes (fields)	No (text only)
Log Rotation	Automatic (vacuum)	<code>logrotate</code>
Persistence	Optional (can be volatile)	Always on disk
Performance	High (indexed)	Good
Best For	Modern Linux (systemd)	Legacy systems

4. USEFUL JOURNALCTL COMMANDS

Task	Command
View all logs	<code>journalctl</code>
View logs in real time	<code>journalctl -f</code>
View logs for a service	<code>journalctl -u nginx</code>
View logs since a time	<code>journalctl --since "1 hour ago"</code>
View logs until a time	<code>journalctl --until "2024-09-24 10:00"</code>
Filter by priority (error)	<code>journalctl -p err</code>
Filter by boot	<code>journalctl -b</code>
View kernel logs	<code>journalctl -k</code>
Show logs with full details	<code>journalctl -o verbose</code>
Follow logs for multiple units	<code>journalctl -u nginx -u mysql -f</code>

5. LOG PRIORITY LEVELS

Priority	Name	Description
0	emerg	System is unusable
1	alert	Action must be taken immediately
2	crit	Critical conditions
3	err	Error conditions
4	warning	Warning conditions
5	notice	Normal but significant
6	info	Informational messages
7	debug	Debug-level messages

6. LOG STORAGE AND ROTATION

Storage Locations

- `/var/log/journal/` (persistent)
- `/run/log/journal/` (volatile, in memory)
- Automatically created per machine ID.
- Each boot has its own log files.

Vacuum Commands (Rotation)

- `journalctl --vacuum-time=7d` # keep 7 days
- `journalctl --vacuum-size=1G` # keep max 1GB
- `journalctl --vacuum-files=10` # keep 10 files

Configuration File

`/etc/systemd/journald.conf`

Important Settings

- `Storage=persistent`
- `SystemMaxUse=1G`
- `SystemMaxFileSize=100M`
- `MaxRetentionSec=7day`

7. REAL EXAMPLES

```

# 1. View recent logs for nginx
journalctl -u nginx --since "1 hour ago"

# 2. View only error logs
journalctl -p err

# 3. Follow logs in real time
journalctl -f

# 4. View logs for current boot
journalctl -b

# 5. Filter logs by keyword
journalctl | grep -i "failed"

# 6. View logs with full details
journalctl -u sshd -o verbose
  
```

8. INCIDENT INVESTIGATION

- Identify the affected service.
- Check recent logs with `journalctl -u <service>`.
- Look for errors, warnings or failed units.
- Check related services and dependencies.
- Use time filters to focus on the incident window.
- Collect logs as evidence.

9. PERSISTENT LOGS SETUP (STEP BY STEP)

```

# 1. Create journal directory (if not exists)
sudo mkdir -p /var/log/journal

# 2. Set permissions
sudo chmod 2755 /var/log/journal

# 3. Edit configuration
sudo nano /etc/systemd/journald.conf

# 4. Set Storage=persistent

# 5. Restart systemd-journald
sudo systemctl restart systemd-journald
  
```

10. BEST PRACTICES

- ✓ Enable persistent logging on servers.
- ✓ Set reasonable retention limits.
- ✓ Use structured logging with `journalctl`.
- ✓ Monitor disk usage of journal logs.
- ✓ Filter logs effectively during incidents.
- ✓ Keep important logs for compliance.
- ✓ Combine with other logs (application, audit logs).
- ✓ Regularly vacuum old logs.
- ✓ Use log forwarding for centralized logging (ELK, Loki, etc.).

11. KEY TAKEAWAYS

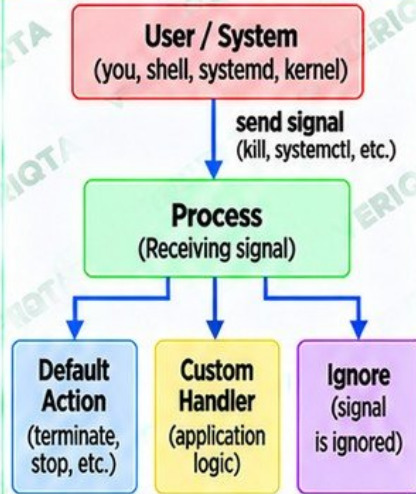
- Journald is the default logging system in modern Linux.
- Logs are stored in binary format and indexed for fast searching.
- Use `journalctl` to view, filter and diagnose issues.
- Configure persistent storage for long-term retention.
- Logs rotate automatically (vacuum).
- Journald provides valuable evidence during incidents.

SIGTERM, SIGKILL, SIGHUP, handlers, graceful shutdown, failure consequences

1. WHAT ARE LINUX SIGNALS?

- Signals are software interrupts sent to a process to notify it of an event or request an action.
- They allow you to control processes, handle errors and perform graceful shutdowns.
- Each signal has a numeric ID and a name (e.g. 15 = SIGTERM).
- Processes can catch, ignore, or take default action on signals.

2. SIGNAL FLOW



3. COMMON SIGNALS

Signal	Number	Default Action	Common Use Case
SIGHUP	1	Terminate	Reload config (e.g. nginx)
SIGINT	2	Terminate	Interrupt from keyboard (Ctrl+C)
SIGQUIT	3	Terminate	Quit and dump core
SIGTERM	15	Terminate	Graceful shutdown (request to stop)
SIGKILL	9	Terminate	Immediately (forced kill)
SIGUSR1	10	Terminate	Custom application use
SIGUSR2	12	Terminate	Custom application use
SIGCHLD	17	Ignore	Notify parent of child state change
SIGSTOP	19	Always stop	Stop (cannot be caught or ignored)
SIGCONT	18	Continue	Resume a stopped process

4. SENDING SIGNALS

Task	Command
Send SIGTERM (graceful)	<code>kill -15 <PID></code> <code>kill <PID></code>
Send SIGKILL (force kill)	<code>kill -9 <PID></code>
Send SIGHUP (reload)	<code>kill -1 <PID></code>
Send signal by name	<code>kill -SIGTERM <PID></code>
Send to all processes (by name)	<code>pkill -SIGTERM nginx</code>
Send to process group	<code>kill -15 -<PGID></code>
List available signals	<code>kill -l</code>

5. PROCESS SIGNAL STATES

Running	Process is running normally.
Stopped	Process is paused (e.g. SIGSTOP).
Terminated	Process has exited (e.g. SIGTERM or SIGKILL).
Zombie	Process has finished but parent has not collected its exit status.
Hung	Process not responding (may need SIGKILL).

6. SIGNAL HANDLERS (EXAMPLE)

```

# Python example: handle SIGTERM
import signal
import time

def handle_sigterm(signum, frame):
    print(f"Received SIGTERM ({signum}). Cleaning up...")
    # Add cleanup logic here
    exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)

print("Process running. Press Ctrl+C or send SIGTERM.")
while True:
    time.sleep(1)
  
```

7. GRACEFUL SHUTDOWN VS FORCE KILL

SIGTERM (Graceful Shutdown)	SIGKILL (Force Kill)
✓ Allows the process to clean up (resources, files, connections). ✓ Recommended for normal shutdown. ✓ Can be caught and handled. ✓ Used by systemd when stopping services.	• Terminates the process immediately. • Cannot be caught or ignored. • Does not allow cleanup. • May cause data loss or corruption. • Use only when the process does not respond.

8. REAL EXAMPLES

```

# Find process ID
ps aux | grep nginx

# Send graceful shutdown
kill -15 1234

# Force kill (if not responding)
kill -9 1234

# Reload nginx config (SIGHUP)
kill -1 1234

# Check signal names
kill -l | head -20
  
```

9. SYSTEMD SERVICE SIGNALS

Action	Command / Signal
Stop service (systemd)	<code>systemctl stop nginx</code> (SIGTERM)
Force stop	<code>systemctl kill nginx</code> (SIGKILL)
Reload service	<code>systemctl reload nginx</code> (SIGHUP)
Restart service	<code>systemctl restart nginx</code> (SIGTERM + start again)

10. FAILURE CONSEQUENCES

Signal / Action	Potential Consequences
SIGKILL	Data loss, incomplete writes, corrupted files, unclean shutdown.
SIGTERM (unhandled)	Same as SIGKILL (if process does not handle it).
SIGHUP (not handled)	Process may terminate unexpectedly.
SIGSTOP	Process stops and may cause service outage.
Repeated restarts	Service instability and downtime.

11. BEST PRACTICES

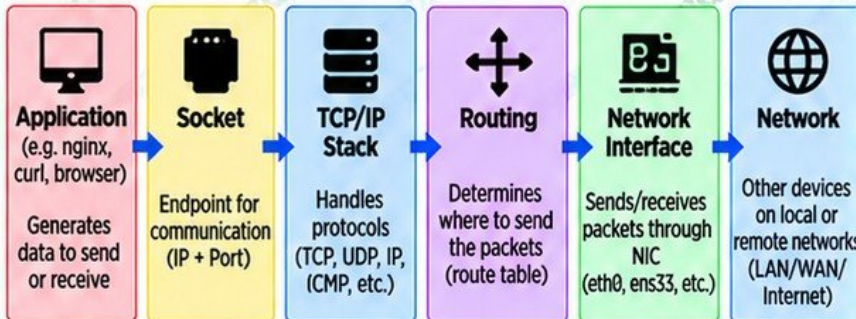
- ✓ Use SIGTERM for normal shutdowns.
- ✓ Implement signal handlers in your applications.
- ✓ Ensure cleanup of files, databases and connections on shutdown.
- ✓ Use systemd to manage service signals.
- ✓ Avoid using SIGKILL unless necessary.
- ✓ Test your application's behavior with different signals.
- ✓ Monitor logs to confirm graceful shutdown.
- ✓ Document your signal handling logic.

12. KEY TAKEAWAYS

- 1 Signals allow you to control and manage processes.
- 2 SIGTERM requests graceful shutdown.
- 3 SIGKILL forces immediate termination.
- 4 Processes can catch, ignore or take default action on signals.
- 5 Proper signal handling prevents data loss and ensures stability.
- 6 systemd uses signals to manage services.
- 7 Always test signal behavior in production scenarios.

Application → socket → TCP/IP → routing → interface → network

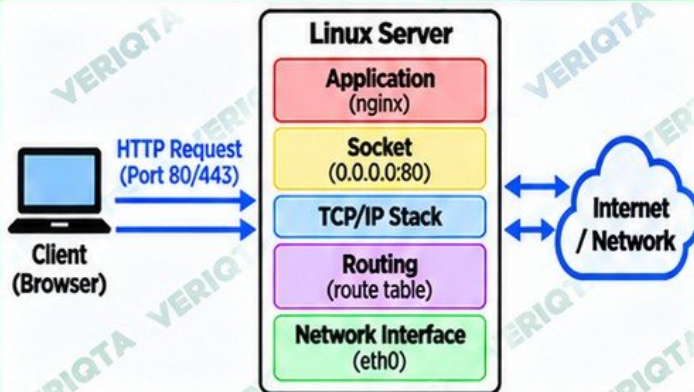
1. LINUX NETWORKING ARCHITECTURE OVERVIEW



2. KEY LAYERS (TCP/IP MODEL)

Layer	Protocols	Purpose
Application	HTTP, HTTPS, SSH, DNS, FTP, SMTP	User applications and services
Transport	TCP, UDP	End-to-end communication (ports, reliability)
Network	IP, ICMP	Logical addressing and routing
Link (Data Link)	Ethernet, ARP	Framing and local network delivery
Physical	NIC, cables, Wi-Fi	Actual transmission of bits

3. DATA FLOW EXAMPLE (WEB REQUEST)



4. IMPORTANT LINUX NETWORKING COMPONENTS

Component	Description
Socket	Endpoint for communication (IP address + port).
Ports	Identify the application (e.g. 80, 443, 22).
TCP	Reliable, connection-oriented protocol.
UDP	Faster, connectionless protocol.
IP Address	Identifies the device (IPv4 or IPv6).
Routing Table	Decides where to send packets (ip route).
Network Interface	Physical or virtual interface (eth0, ens33, lo).
Firewall	Controls traffic (iptables, nftables, firewalld).
DNS	Resolves domain names to IP addresses.
ARP	Maps IP addresses to MAC addresses (local network).
Gateway	Routes traffic to other networks.

5. COMMON LINUX NETWORKING COMMANDS

Task	Command
View network interfaces	<code>ip addr</code>
View IP routing table	<code>ip route</code>
View open sockets and ports	<code>ss -tln</code>
Check active connections	<code>ss -tunp</code>
Test network connectivity	<code>ping <host></code>
Trace packet path	<code>traceroute <host></code>
Check DNS resolution	<code>nslookup <domain></code> or <code>dig <domain></code>
View ARP table	<code>ip neigh</code>
View network interface stats	<code>ip -s link</code>
Capture packets	<code>tcpdump -i <interface></code>
Check firewall rules	<code>sudo iptables -L</code> or <code>sudo nft list ruleset</code>

6. ROUTING EXAMPLE

Linux uses a routing table to decide where to send packets.

```
# View routing table
ip route
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link
10.0.0.0/24 via 192.168.1.2 dev eth0
```



7. CHECKING OPEN PORTS (SOCKETS)

```
# Show listening ports
ss -tln
```

Netid	State	Local Address:Port	Peer Address:Port
tcp	LISTEN	0.0.0.0:80	0.0.0.0:*
tcp	LISTEN	0.0.0.0:22	0.0.0.0:*
udp	UNCONN	0.0.0.0:53	0.0.0.0:*

8. TROUBLESHOOTING FLOW

- 1 Check application (is service running?)
- 2 Check socket (is port open?)
- 3 Check firewall (is traffic allowed?)
- 4 Check routing (is route correct?)
- 5 Check DNS (is name resolving?)
- 6 Test connectivity (ping, traceroute)
- 7 Capture packets (tcpdump)
- 8 Check logs (application and system)

9. KEY TAKEAWAYS

- ✓ Applications use sockets to communicate using IP and ports.
- ✓ TCP/UDP handle transport, IP handles routing.
- ✓ The routing table decides where packets go.
- ✓ Network interfaces send and receive packets.
- ✓ Use tools like ip, ss, ping, traceroute, tcpdump to troubleshoot.
- ✓ Understanding the flow helps you quickly identify and fix network issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

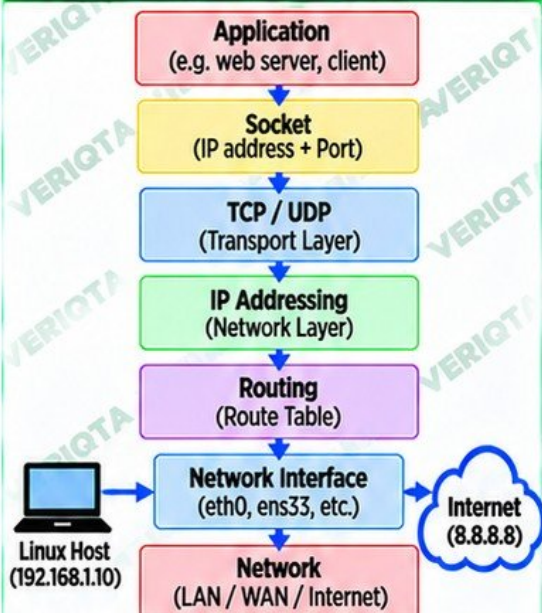


@veriqta

IP Addressing, Interfaces & Routes

Interfaces, routes, gateways, policy routing, ip tooling

1. OVERVIEW: HOW LINUX NETWORKING WORKS



2. IP ADDRESSING CONCEPTS

IP Address	Identifies a device on a network (IPv4 or IPv6).
Subnet Mask	Defines the network portion (e.g. 255.255.255.0 or /24).
Network	Group of devices in the same address range.
Broadcast	Sends data to all devices in the network.
Gateway	Routes traffic to other networks (e.g. 192.168.1.1).
DNS	Converts domain names to IP addresses (e.g. 8.8.8.8).
Private IPs	Used in local networks (e.g. 10.0.0.0/8, 192.168.0.0/16).
Public IPs	Used on the internet (assigned by ISP or cloud provider).
IPv6	Larger address space for modern networks.

3. COMMON IP ADDRESS RANGES

Type	Range
Private (Class A)	10.0.0.0 – 10.255.255.255 (10.0.0.0/8)
Private (Class B)	172.16.0.0 – 172.31.255.255 (172.16.0.0/12)
Private (Class C)	192.168.0.0 – 192.168.255.255 (192.168.0.0/16)
Loopback	127.0.0.0 – 127.255.255.255 (127.0.0.0/8)
Link Local	169.254.0.0 – 169.254.255.255 (169.254.0.0/16)
Multicast	224.0.0.0 – 239.255.255.255 (224.0.0.0/4)
IPv6 Local	fe80::/10 (Link Local)
IPv6 Unique Local	fc00::/7 (Private IPv6)
IPv6 Global	2000::/3 (Public IPv6)

4. NETWORK INTERFACES

Task	Command
List all interfaces	<code>ip addr</code>
Show interface details	<code>ip addr show eth0</code>
Show short output	<code>ip -br addr</code>
Bring interface up	<code>sudo ip link set eth0 up</code>
Bring interface down	<code>sudo ip link set eth0 down</code>
Assign IP address	<code>sudo ip addr add 192.168.1.10/24 dev eth0</code>
Remove IP address	<code>sudo ip addr del 192.168.1.10/24 dev eth0</code>
Show link status	<code>ip link show</code>
View interface stats	<code>ip -s link show eth0</code>

5. ROUTING BASICS

Task	Command
View routing table	<code>ip route</code>
Show detailed route	<code>ip route show</code>
Add a route	<code>sudo ip route add 10.0.0.0/24 via 192.168.1.1</code>
Delete a route	<code>sudo ip route del 10.0.0.0/24</code>
Add default gateway	<code>sudo ip route add default via 192.168.1.1</code>
Show route to host	<code>ip route get 8.8.8.8</code>
Flush route table	<code>sudo ip route flush table main</code>
Show multiple tables	<code>ip route show table all</code>

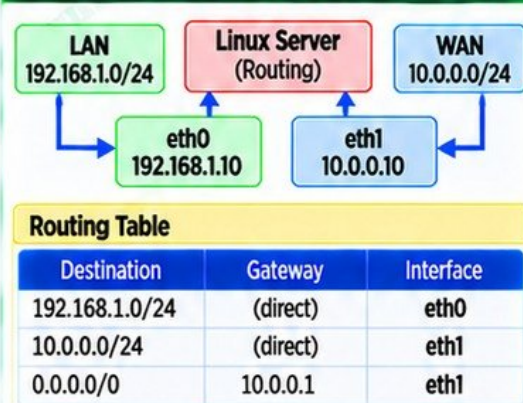
6. POLICY ROUTING (MULTIPLE TABLES)

Policy routing allows different traffic to use different routes based on rules.

- 1 Create a custom table in `/etc/iproute2/rt_tables`
- 2 Add routes to the custom table.
- 3 Create a rule to match traffic (source IP, destination, mark, etc.).
- 4 Test and verify.

```
# 1. Add custom table (edit file)
sudo nano /etc/iproute2/rt_tables
# 2. Add route to table 100
sudo ip route add 0.0.0.0/0 via 192.168.2.1 table 100
# 3. Add rule for traffic from 192.168.1.10
sudo ip rule add from 192.168.1.10 table 100
# 4. Show rules
ip rule show
```

7. REAL EXAMPLE: TWO INTERFACES + ROUTES



8. USEFUL DIAGNOSTIC COMMANDS

Task	Command
Check connectivity	<code>ping 8.8.8.8</code>
Trace path	<code>tracert 8.8.8.8</code>
Show DNS resolution	<code>dig google.com</code>
Show open ports	<code>ss -tln</code>
Check ARP table	<code>ip neigh</code>
View network stats	<code>ip -s link</code>
Check firewall rules	<code>sudo iptables -L</code> or <code>sudo nft list ruleset</code>
View connection details	<code>ss -tunp</code>
Capture packets	<code>sudo tcpdump -i eth0</code>
Check interface MTU	<code>ip link show eth0</code>
Check route to host	<code>ip route get 1.1.1.1</code>

9. BEST PRACTICES

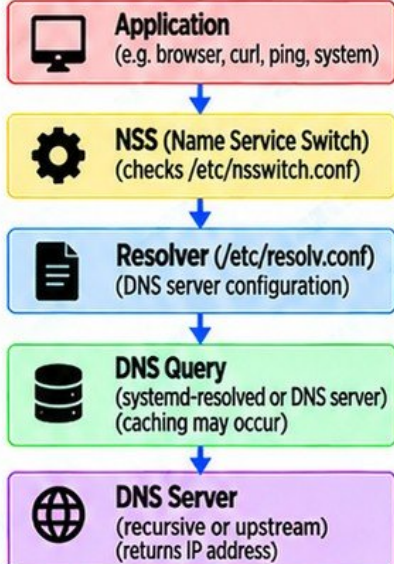
- ✓ Use meaningful interface names.
- ✓ Keep your routing table clean and documented.
- ✓ Use static IPs for servers when required.
- ✓ Validate gateway and DNS configuration.
- ✓ Use policy routing for multi-homed systems.
- ✓ Monitor interface status and network traffic.
- ✓ Secure with firewall rules (iptables/nftables).
- ✓ Test failover and routing changes.
- ✓ Document network configuration changes.

10. KEY TAKEAWAYS

- 1 IP addresses identify devices on a network.
- 2 Interfaces send and receive network traffic.
- 3 The routing table decides where packets go.
- 4 Gateways connect your network to other networks.
- 5 Policy routing allows advanced traffic control.
- 6 Use ip tools to configure, troubleshoot and monitor.

Resolver flow, /etc/resolv.conf, NSS, systemd-resolved, caching, failures

1. DNS RESOLUTION FLOW



2. KEY COMPONENTS

Component	Description
Application	Makes a request for a domain name (e.g. www.google.com).
NSS	Determines how name resolution is performed using /etc/nsswitch.conf.
/etc/resolv.conf	Lists DNS servers and options for the resolver.
systemd-resolved	Local DNS resolver, provides caching and manages DNS configuration.
DNS Server	Resolves domain names to IP addresses (recursive/upstream).
Cache	Stores recent DNS responses to speed up lookups.

3. /etc/resolv.conf EXAMPLE

This file specifies the DNS servers and options used for name resolution.

```
# /etc/resolv.conf
nameserver 8.8.8.8
nameserver 1.1.1.1
search veriqta.com
options edns0 trust-ad
options attempts:2 timeout:5
```

Option	Meaning
nameserver	IP address of DNS server.
search	Domain to append to short names.
options	Resolver options (timeout, attempts, EDNS, etc.).

4. NSS CONFIGURATION

/etc/nsswitch.conf determines the order used for name resolution (DNS, files, etc.).

```
# /etc/nsswitch.conf (relevant section)
hosts: files mdns4_minimal [NOTFOUND=return]
      dns myhostname
```

Source	Meaning
files	Checks /etc/hosts first.
dns	Uses DNS servers from /etc/resolv.conf or systemd-resolved.
myhostname	Resolves local system hostname.

5. SYSTEMD-RESOLVED

systemd-resolved is the default DNS resolver on many modern Linux distributions.

Feature	Description
Local resolver	Listens on 127.0.0.53.
Caching	Caches DNS responses for faster lookups.
Automatic configuration	Integrates with NetworkManager and DHCP.
Stub resolver	Uses /run/systemd/resolve/stub-resolv.conf.
Management	Use resolvectl to view and manage DNS settings.

Useful commands

```
resolvectl status
resolvectl dns
resolvectl domain
resolvectl query
systemctl status
systemd-resolved
```

6. DNS CACHING

Caching improves performance by storing recent DNS responses.

Location

- systemd-resolved (default)
- nscd (on some older systems)
- Application-level caching (e.g. browsers, databases)

Check cache (systemd-resolved)

```
resolvectl statistics
resolvectl show-cache
resolvectl flush-caches
```

7. USEFUL DNS COMMANDS

Task	Command
Resolve a domain name	dig example.com
Simple lookup	nslookup example.com
Check DNS from system	host example.com
Watch real-time queries	resolvectl query example.com
Check current DNS servers	resolvectl status
Test using specific server	dig @8.8.8.8 example.com
Check /etc/resolv.conf	cat /etc/resolv.conf
Check NSS configuration	cat /etc/nsswitch.conf
Check network DNS (NetworkManager)	nmcli device show
Flush local cache	resolvectl flush-caches

8. COMMON DNS FAILURES

Issue	Possible Cause
Domain not resolving	Wrong DNS server, network issue, firewall, typo.
Slow DNS resolution	Slow DNS server, no caching, network latency.
Works with IP but not with domain	DNS misconfiguration or server unreachable.
Intermittent failures	Unreliable DNS server or network connectivity issues.
NXDOMAIN error	Domain does not exist or wrong domain name.
Timeout	DNS server not responding or blocked by firewall.
Incorrect IP returned	DNS cache issue or DNS poisoning.

9. TROUBLESHOOTING FLOW

- 1 Check if the domain name is correct.
- 2 Verify /etc/resolv.conf and DNS servers.
- 3 Test connectivity to DNS server (ping, dig).
- 4 Check NSS configuration (/etc/nsswitch.conf).
- 5 Check systemd-resolved status (resolvectl status).
- 6 Look for firewall rules blocking DNS (port 53).
- 7 Check logs for errors (journalctl).
- 8 Flush cache and test again.

10. KEY TAKEAWAYS

- ✓ DNS converts domain names to IP addresses.
- ✓ /etc/resolv.conf specifies DNS servers and options.
- ✓ NSS controls the order of name resolution.
- ✓ systemd-resolved provides local caching and management.

- ✓ Use dig, nslookup, host and resolvectl for troubleshooting.
- ✓ Check network connectivity and firewall rules.
- ✓ Caching improves performance but can cause stale results.
- ✓ Understand common failure scenarios and how to diagnose them.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

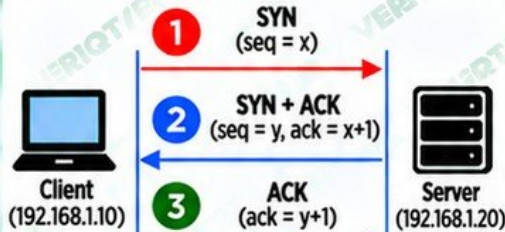


@veriqta

Handshake, connection states, TIME_WAIT, retransmissions, backlog

1. TCP THREE-WAY HANDSHAKE

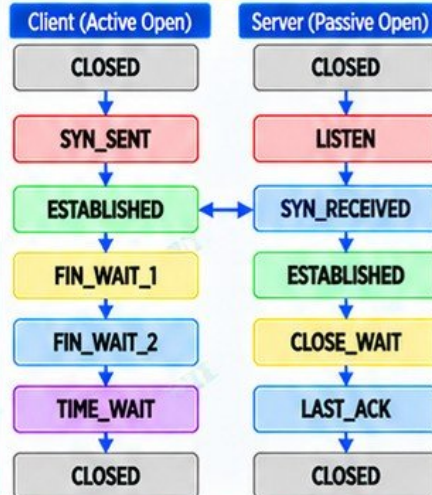
TCP uses a three-way handshake to establish a reliable connection between a client and a server.



- 1 Client sends SYN to request a connection.
- 2 Server responds with SYN + ACK.
- 3 Client sends ACK. Connection is established and data can now be transferred.

2. TCP CONNECTION STATES

TCP uses different states to manage the lifecycle of a connection.



3. TIME_WAIT EXPLAINED

TIME_WAIT is a state where a closed connection remains for a period to ensure reliability and prevent old packets from affecting new connections.

Key Points

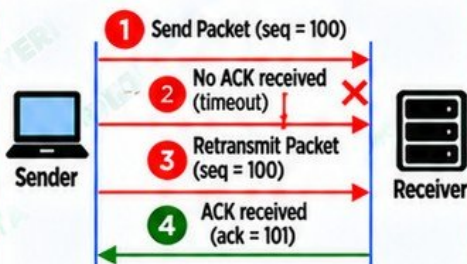
- Occurs on the side that initiates the close (active closer).
- Lasts $2 \times \text{MSL}$ (usually 60 seconds).
- Prevents delayed packets from previous connections from interfering with new connections.
- Helps ensure reliable connection termination.

Check TIME_WAIT connections

```
ss -tan state time-wait
netstat -tan | grep TIME_WAIT
```

4. TCP RETRANSMISSIONS

If an acknowledgment is not received, TCP retransmits the lost packet after a timeout.

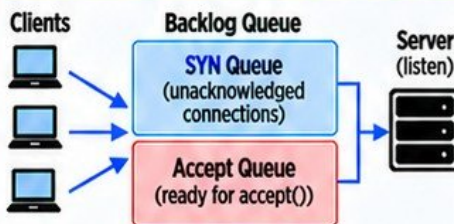


Key Points

- Retransmissions happen when ACK is not received within the timeout period (RTO).
- Caused by network congestion, packet loss, or high latency.
- Too many retransmissions can lead to slow connections or connection failure.

5. TCP LISTEN BACKLOG

When a server accepts TCP connections, incoming connection requests are placed in a backlog queue.



Important Settings

Parameter	Description
backlog (listen())	Maximum number of pending connections in the accept queue.
net.core.somaxconn	Maximum accept queue size (default 4096).
net.ipv4.tcp_max_syn_backlog	Maximum SYN queue size (default 256).
net.ipv4.tcp_synack_retries	Number of SYN+ACK retries (default 5).

6. USEFUL TCP COMMANDS

Task	Command
View TCP connections	ss -tan
View connection states	ss -tan state all
Filter by state	ss -tan state established
View listening ports	ss -ltn
View socket details	ss -ti
Check backlog (LISTEN)	ss -ltn sport = :80
Check retransmissions	netstat -s grep retrans
Monitor in real time	watch -n 1 'ss -tan'
Check kernel settings	sysctl net.ipv4.tcp_max_syn_backlog
Packet capture (handshake)	tcpdump -nn -i eth0 tcp port 80
Show TIME_WAIT	ss -tan state time-wait

7. COMMON TCP CONNECTION ISSUES

Issue	Symptoms	Possible Cause
Connection timeout	Cannot connect to server.	Firewall, routing, server down, packet loss.
SYN not received	No response from server.	Firewall blocking, server not listening.
High TIME_WAIT	Many connections in TIME_WAIT.	High connection churn, short-lived connections.
Connection reset	Connection closed immediately.	Application or firewall resetting connection.
Slow connection	High latency, slow response.	Network congestion, retransmissions.
Connection refused	Immediate failure.	Service not running or not listening on port.

8. TROUBLESHOOTING FLOW

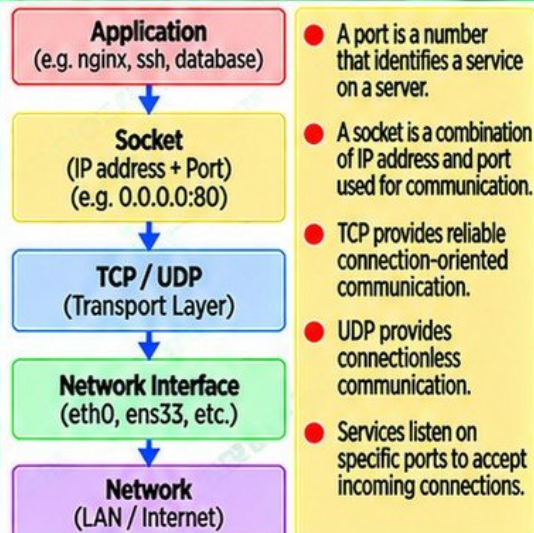
- 1 Check if the server is listening (ss -ltn).
- 2 Verify network connectivity (ping, traceroute).
- 3 Check firewall rules (iptables/nftables).
- 4 Check TCP connection states (ss -tan).
- 5 Look for retransmissions (netstat -s).
- 6 Check backlog and kernel settings.
- 7 Capture packets with tcpdump.
- 8 Analyze logs on client and server.

9. BEST PRACTICES

- ✓ Use appropriate backlog sizes.
- ✓ Monitor connection states (ESTABLISHED, TIME_WAIT, etc.).
- ✓ Tune kernel parameters for high connection environments.
- ✓ Use connection pooling for applications.
- ✓ Monitor retransmissions and packet loss.
- ✓ Keep firewall rules consistent.
- ✓ Use monitoring and alerting for unusual connection behavior.
- ✓ Understand TIME_WAIT and its impact on high traffic systems.
- ✓ Test under load to ensure stability.

ss, socket ownership, listening services, refused versus timed-out connections

1. PORTS, SOCKETS AND THE BIG PICTURE



2. COMMON PORTS

Port	Service	Protocol	Common Use
22	SSH	TCP	Remote login
80	HTTP	TCP	Web traffic
443	HTTPS	TCP	Secure web traffic
3306	MySQL	TCP	Database
5432	PostgreSQL	TCP	Database
6379	Redis	TCP	Cache
8080	HTTP-Alt	TCP	Web application
9090	Prometheus	TCP	Monitoring
3000	Application	TCP	Dev application

3. VIEW LISTENING SERVICES (ss)

The ss command shows open sockets, listening ports and established connections.

```
# Show listening ports (TCP and UDP)
ss -tln

# Show listening ports with process info
ss -tlnp

# Show all connections
ss -tanp
```

Example Output (ss -tln)

Netid	State	Local Address:Port	Process
tcp	LISTEN	0.0.0.0:22	(sshd)
tcp	LISTEN	0.0.0.0:80	(nginx)
tcp	LISTEN	127.0.0.1:3306	(mysqld)
udp	UNCONN	0.0.0.0:53	(systemd-resolved)

4. SOCKET OWNERSHIP (WHICH PROCESS USES THE PORT?)

You can see which process owns a listening port using ss, lsof or netstat.

Task	Command
Show listening ports with process	ss -tlnp
Find process using a specific port	lsof -i :80
Find process by name	pgrep -a nginx
Show process details	ps -fp <PID>
Using netstat (if available)	netstat -tlnp

5. CONNECTION STATES

State	Description
LISTEN	Service is waiting for connections.
ESTABLISHED	Connection is active.
SYN-SENT	Client sent SYN, waiting for response.
SYN-RECEIVED	Server received SYN, waiting for ACK.
TIME_WAIT	Connection closed, waiting before reuse.
CLOSE_WAIT	Remote closed, waiting for local close.
FIN_WAIT	Connection is closing.
CLOSED	No active connection.

6. TESTING CONNECTIONS

Task	Command
Test if port is open	nc -zv IP PORT
Test with telnet	telnet IP PORT
Test with curl (HTTP)	curl -I http://IP:PORT
Test with curl (HTTPS)	curl -I https://IP:PORT
Check connection timeout	nc -vz -w 5 IP PORT
Check route to host	tracert IP
Check if service responds	ss -tan grep :PORT

7. REFUSED VS TIMED-OUT CONNECTIONS

CONNECTION REFUSED

Means the host is reachable but nothing is listening on that port.



Common causes:

- Service not running.
- Wrong port number.
- Firewall actively rejecting connection.
- Service listening only on localhost (127.0.0.1).

CONNECTION TIMED OUT

Means the connection request did not receive a response.



Common causes:

- Host is unreachable.
- Firewall is blocking the connection (silently dropping).
- Network issue or routing problem.
- Service is down or overloaded.

8. DEBUGGING FLOW

- 1 Check if the host is reachable (ping, traceroute).
- 2 Check if the port is open (nc, telnet, ss).
- 3 Check if the service is running (systemctl, ss, ps).
- 4 Check firewall rules (iptables, ufw, firewalld).
- 5 Check if the service is listening on the correct IP and port (ss -tlnp).
- 6 Check application logs (e.g. /var/log/nginx/).
- 7 Test from local and remote hosts.
- 8 Identify if the issue is refused or timed out and fix accordingly.

9. USEFUL COMMANDS (QUICK REF)

Task	Command
Show listening ports	ss -tln
Show port with process	ss -tlnp
Find process by port	lsof -i :80
Check TCP connections	ss -tan
Check UDP sockets	ss -uan
Check socket ownership	ps -fp <PID>
Test port (TCP)	nc -zv IP PORT
Test port (HTTP)	curl -I http://IP:PORT
Trace network path	tracert IP
Check firewall rules	sudo iptables -L or sudo nft list ruleset

10. KEY TAKEAWAYS

- Ports identify services, sockets use IP + port for communication.
- Use ss to view listening ports and established connections.
- Check which process owns a port using ss or lsof.
- Connection refused means host is reachable but no service is listening.
- Connection timed out means no response (network, firewall or service down).
- Understand TCP connection states for troubleshooting.
- Use the right tools (ss, nc, curl, traceroute, iptables) to debug.
- Verify service configuration and logs.
- Always test from both local and remote hosts.
- Properly configure firewall rules to allow required ports.
- Document open ports and services in your environment.

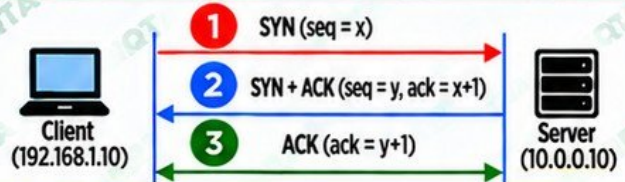
tcpdump, packet path, SYN failures, retransmissions, DNS/TLS evidence

1. PACKET PATH OVERVIEW



- 1** **Packet created**
(SRC IP: 192.168.1.10)
(DST IP: 10.0.0.10)
(Protocol: TCP/UDP)
- 2** **Packet routed**
(Through gateway and network infrastructure)
- 3** **Packet filtered**
(Firewall, ACL, security groups)
- 4** **Packet delivered**
(If allowed, reaches destination and service port)

2. TCP HANDSHAKE CAPTURE (SUCCESSFUL CONNECTION)



Capture with tcpdump

```
sudo tcpdump -nn -i eth0 host 10.0.0.10 and tcp port 80
IP 192.168.1.10.52344 > 10.0.0.10.80: Flags [S], seq 1000
IP 10.0.0.10.80 > 192.168.1.10.52344: Flags [S.], seq 2000, ack 1001
IP 192.168.1.10.52344 > 10.0.0.10.80: Flags [.], ack 2001
```

3. SYN FAILURE SCENARIOS

No Response (Timeout)



- SYN sent, no response.
- Usually network, firewall or routing issue.
- Connection times out.

Connection Refused



- Immediate RST response.
- Host is reachable but no service is listening.
- Usually port closed or service down.

Successful Connection



- SYN → SYN+ACK → ACK
- Connection established.
- Packets can now flow in both directions.

4. COMMON TCPDUMP FILTERS

Task	Command
Capture traffic to a host	<code>sudo tcpdump -nn host 10.0.0.10</code>
Capture a specific port	<code>sudo tcpdump -nn port 80</code>
Capture TCP traffic	<code>sudo tcpdump -nn tcp</code>
Capture UDP traffic	<code>sudo tcpdump -nn udp</code>
Capture SYN packets	<code>sudo tcpdump -nn 'tcp[tcpflags] & tcp-syn != 0'</code>
Capture DNS traffic	<code>sudo tcpdump -nn port 53</code>
Capture TLS/HTTPS	<code>sudo tcpdump -nn port 443</code>
Capture on specific interface	<code>sudo tcpdump -nn -i eth0</code>
Capture and save to file	<code>sudo tcpdump -nn -i eth0 -w capture.pcap</code>
Read from saved file	<code>tcpdump -nn -r capture.pcap</code>

5. DNS PACKET EXAMPLE

Capture DNS queries and responses.

```
sudo tcpdump -nn -i eth0 port 53
IP 192.168.1.10.55321 > 8.8.8.8.53:
12345+ A? example.com. (28)
IP 8.8.8.8.53 > 192.168.1.10.55321:
12345 1/0/0 A 93.184.216.34 (44)
```

Key Points

- Look for DNS queries (A, AAAA, CNAME).
- Check if you receive a response.
- Timeout means DNS server not reachable.
- NXDOMAIN means domain does not exist.

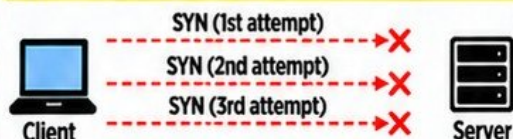
6. TLS/HTTPS PACKET EXAMPLE

Capture TLS handshake (encrypted traffic).

```
sudo tcpdump -nn -i eth0 port 443
IP 192.168.1.10.52344 > 10.0.0.10.443:
Client Hello
IP 10.0.0.10.443 > 192.168.1.10.52344:
Server Hello, Certificate, ...
```

7. RETRANSMISSIONS

Retransmissions happen when an ACK is not received. This can be caused by network congestion, packet loss or high latency.



Example tcpdump output

```
IP 192.168.1.10.52344 > 10.0.0.10.80: Flags [S], seq 1000 (retransmission)
IP 192.168.1.10.52344 > 10.0.0.10.80: Flags [S], seq 1000 (retransmission)
IP 192.168.1.10.52344 > 10.0.0.10.80: Flags [S], seq 1000 (retransmission)
```

8. USEFUL ANALYSIS TOOLS

Task	Command
Live packet capture	<code>sudo tcpdump -nn -i eth0</code>
Analyze packets visually	<code>wireshark capture.pcap</code>
Show TCP streams	<code>tshark -r capture.pcap -z follow,tcp,ascii,0</code>
Filter in Wireshark	<code>tcp, dns, tls, http</code>
Check packet path (MTU issues)	<code>ping -M do -s 1472 10.0.0.10</code>
Trace route and capture	<code>sudo tcpdump -nn -i eth0 host 10.0.0.10 and (icmp or tcp)</code>
Check for packet loss	<code>mtr 10.0.0.10</code>
Analyze DNS packets	<code>tshark -r capture.pcap -Y dns</code>
Analyze TLS handshake	<code>tshark -r capture.pcap -Y tls</code>

9. TROUBLESHOOTING FLOW

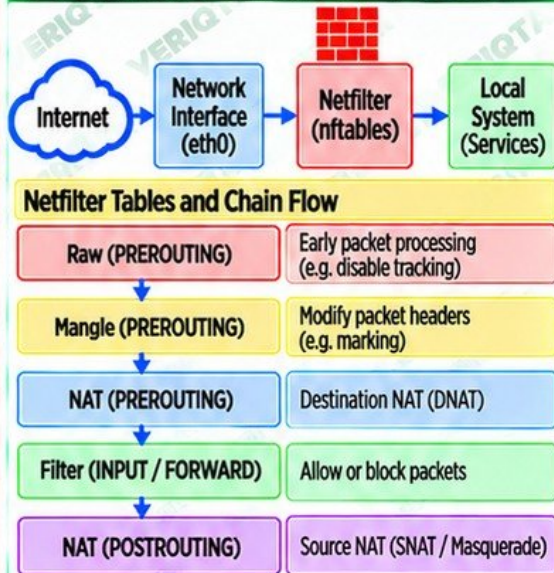
- 1** Reproduce the issue (command or application).
- 2** Capture packets with tcpdump.
- 3** Check if packets leave the client.
- 4** Check if packets reach the server.
- 5** Look for responses (SYN+ACK, DNS reply, TLS handshake, etc.).
- 6** Identify retransmissions or errors.
- 7** Verify firewall rules and network path.
- 8** Use Wireshark for deep analysis if needed.

10. KEY TAKEAWAYS

- ✓ Use tcpdump to see what is really happening on the network.
- ✓ SYN failures can be timeout (no response) or refused (RST).
- ✓ Retransmissions indicate packet loss, high latency or congestion.
- ✓ Capture DNS and TLS traffic to troubleshoot name resolution and encrypted connections.
- ✓ Always check both client and server side.
- ✓ Use appropriate filters to reduce noise.
- ✓ Saved packet captures can be analyzed later with Wireshark.
- ✓ Packet-level analysis provides real evidence for root cause.

Netfilter, nftables, filtering, state tracking, safe investigation

1. LINUX PACKET FILTERING ARCHITECTURE



2. COMMON NFTABLES RULES

Task	Command (nftables)
List rules	<code>sudo nft list ruleset</code>
Flush all rules	<code>sudo nft flush ruleset</code>
Allow SSH (port 22)	<code>sudo nft add rule inet filter input tcp dport 22 accept</code>
Allow HTTP/HTTPS	<code>sudo nft add rule inet filter input tcp dport {80,443} accept</code>
Allow established connections	<code>sudo nft add rule inet filter input ct state established, related accept</code>
Drop invalid packets	<code>sudo nft add rule inet filter input ct state invalid drop</code>
Default drop policy	<code>sudo nft add chain inet filter input { type filter hook input priority 0; policy drop; }</code>

3. STATE TRACKING (CONNTRACK)

Netfilter tracks connection states to allow related traffic (e.g. response packets).

State	Description
NEW	New incoming connection request.
ESTABLISHED	Connection is established (packets are part of a known connection).
RELATED	Related to an existing connection (e.g. FTP data).
INVALID	Packet is not part of any valid connection (often dropped).

```
# Show connection tracking table
sudo conntrack -L
# Show only established connections
sudo conntrack -L | grep ESTABLISHED
```

4. EXAMPLE: BASIC FIREWALL RULESET

```
# Flush existing rules
sudo nft flush ruleset
# Create a simple ruleset
sudo nft -f - <<'EOF'
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept # allow existing
    ct state invalid drop # drop invalid
    iif lo accept # allow localhost
    tcp dport 22 accept # allow SSH
    tcp dport {80,443} accept # allow HTTP/HTTPS
  }
}
EOF
```

5. COMMON FIREWALL COMMANDS

Task	Command
Check nftables status	<code>sudo nft list ruleset</code>
List rules (short)	<code>sudo nft list ruleset</code>
Check packet counters	<code>sudo nft list chain inet filter input --a</code>
Check with iptables (legacy)	<code>sudo iptables -L -n -v</code>
Check open ports	<code>sudo ss -tulnp</code>
Monitor dropped packets	<code>sudo dmesg grep -i drop</code>
Check conntrack entries	<code>sudo conntrack -L</code>
Check firewall service	<code>sudo systemctl status nftables</code>

6. SAFE INVESTIGATION PRACTICES

- ✓ Always verify current rules before making changes.
- ✓ Use logging rules to monitor before blocking.
- ✓ Test rules in a controlled environment.
- ✓ Avoid locking yourself out (keep SSH access).
- ✓ Use connection tracking instead of only source IP rules.
- ✓ Document all changes.
- ✓ Monitor logs for unexpected drops.
- ✓ Use specific rules instead of allowing all traffic.
- ✓ Regularly review and clean up unused rules.

7. LOGGING AND MONITORING

Log dropped packets to understand what traffic is being blocked.

```
# Add a logging rule (before drop)
sudo nft add rule inet filter input \
  counter log prefix "NFT DROP: "\
  drop
```

Log Location	Command
View kernel logs	<code>sudo dmesg grep "NFT DROP"</code>
View system logs	<code>sudo journalctl -k grep "NFT DROP"</code>
Real-time monitoring	<code>sudo journalctl -kf</code>
Monitor with tcpdump	<code>sudo tcpdump -i eth0</code>

8. COMMON USE CASES

	Allow specific services	Allow only required ports (e.g. 22, 80, 443).
	Block specific IP addresses	Drop traffic from known malicious IPs.
	Allow internal network	Allow trusted internal networks.
	Prevent port scanning	Use rate limiting and drop invalid packets.
	NAT / Masquerading	Allow internal network to access the internet.
	Log and investigate	Log suspicious traffic for analysis.

9. TROUBLESHOOTING FLOW

- 1 Check if the service is running (`ss -tulnp`).
- 2 Verify firewall rules (`nft list ruleset`).
- 3 Check packet counters (`nft list chain -a`).
- 4 Look for dropped packets in logs.
- 5 Test connectivity (`nc`, `telnet`, `curl`).
- 6 Check connection state (`conntrack -L`).
- 7 Use `tcpdump` to capture packets.
- 8 Identify whether traffic is blocked by firewall, network or service.

10. KEY TAKEAWAYS

- ✓ Netfilter and nftables control packet filtering, NAT and state tracking.
- ✓ Use stateful filtering to allow legitimate traffic and block unwanted traffic.
- ✓ Always log and monitor firewall activity.
- ✓ Make changes carefully to avoid losing access.
- ✓ Use standard troubleshooting tools (`ss`, `conntrack`, `tcpdump`, `logs`).
- ✓ Keep firewall rules documented and reviewed regularly.

PID, network, mount, user, IPC, UTS namespaces and container foundations

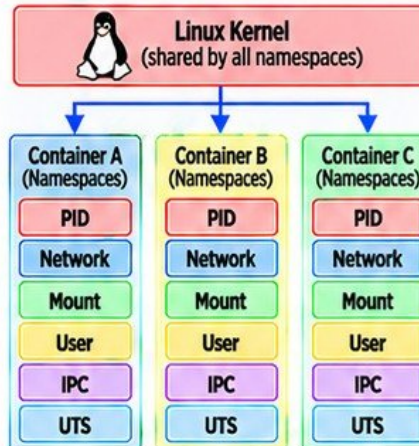
1. LINUX NAMESPACES OVERVIEW

Namespaces provide isolation by giving a process its own view of system resources while sharing the same kernel.

PID	Process isolation (own process tree)
Network	Network stack isolation (interfaces, routes, ports)
Mount	Filesystem isolation (own mount points)
User	User and group isolation (UID/GID mapping)
IPC	Inter-process communication isolation (message queues, shared memory, semaphores)
UTS	System identity isolation (hostname, domain name)

2. HOW NAMESPACES ISOLATE RESOURCES

Each namespace gives a process its own isolated view of specific system resources.



3. REAL EXAMPLE: PID NAMESPACE

Processes inside a namespace see their own process IDs starting from 1.

Host (PID namespace)	Inside new PID namespace
<pre>\$ ps -ef head -3</pre>	<pre>\$ unshare -pf --mount-proc bash \$ ps -ef head -3</pre>
<pre> UID PID CMD root 1 /sbin/init root 734 systemd user 1021 bash </pre>	<pre> UID PID CMD root 1 bash root 8 ps root 9 head </pre>

Key Points

- PID 1 inside the namespace is not the host's PID 1.
- Processes are isolated and cannot see host processes by default.
- Used by containers to isolate application processes.

4. NETWORK NAMESPACE EXAMPLE

Each network namespace has its own network interfaces, IP addresses, routes and firewall rules.

Host network namespace	New network namespace
<pre>\$ ip a</pre>	<pre>\$ ip netns add testns \$ ip netns exec testns ip a</pre>
<pre> 1: lo: <LOOPBACK> 2: ens33: <BROADCAST> inet 192.168.1.10/24 </pre>	<pre> 1: lo: <LOOPBACK> inet 127.0.0.1/8 </pre>

Key Points

- Isolated network stack with its own interfaces.
- Can have separate IP addresses, routes and firewall rules.
- Used by containers to provide network isolation.

5. MOUNT NAMESPACE EXAMPLE

Each mount namespace has its own view of the filesystem and mount points.

Host mount namespace	New mount namespace
<pre>\$ mount head -3</pre>	<pre>\$ unshare -m bash \$ mount head -3</pre>
<pre> /dev/sda1 on / type ext4 proc on /proc type proc tmpfs on /run type tmpfs </pre>	<pre> /dev/sda1 on / type ext4 proc on /proc type proc tmpfs on /tmp type tmpfs </pre>

Key Points

- Can have different mount points and filesystems.
- Changes inside the namespace do not affect the host.
- Used by containers to provide filesystem isolation (e.g. container root filesystem).

6. USER NAMESPACE EXAMPLE

User namespaces map user IDs inside the namespace to different IDs on the host.

Host user namespace	New user namespace
<pre>\$ id</pre>	<pre>\$ unshare -Ur bash \$ id</pre>
<pre> uid=1000(user) gid=1000(user) groups=1000(user) </pre>	<pre> uid=0(root) gid=0(root) groups=0(root) </pre>

Key Points

- Process can be root inside the namespace without being root on the host.
- Provides security by limiting privileges.
- Used by rootless containers (e.g. rootless Docker).

7. IPC NAMESPACE EXAMPLE

Each IPC namespace has its own message queues, shared memory and semaphores.

Host IPC namespace	New IPC namespace
<pre>\$ ipcs -q</pre>	<pre>\$ unshare -i bash \$ ipcs -q</pre>
<pre> ----- Message Queues ----- key msqid owner perms </pre>	<pre> ----- Message Queues ----- key msqid owner perms </pre>

Key Points

- Isolates System V IPC and POSIX message queues.
- Processes in different IPC namespaces cannot communicate using these IPC mechanisms.
- Used by containers to prevent cross-application IPC.

8. UTS NAMESPACE EXAMPLE

UTS namespace isolates system identity information such as hostname and domain name.

Host UTS namespace	New UTS namespace
<pre>\$ hostname</pre>	<pre>\$ unshare -u bash \$ hostname veriqa-container</pre>
<pre> veriqa-host \$ domainname (none) </pre>	<pre> veriqa-container </pre>

Key Points

- Can have a different hostname and domain name.
- Changes inside the namespace do not affect the host.
- Used by containers to give each container its own identity.

9. USEFUL COMMANDS (QUICK REFERENCE)

Task	Command
Create new PID namespace	<code>unshare -p bash</code>
Create new network namespace	<code>unshare -n bash</code>
Create new mount namespace	<code>unshare -m bash</code>
Create new user namespace	<code>unshare -Ur bash</code>
Create new IPC namespace	<code>unshare -i bash</code>
Create new UTS namespace	<code>unshare -u bash</code>
Run with multiple namespaces	<code>unshare -p -n -m -u -i bash</code>
List network namespaces	<code>ip netns list</code>
Execute command in netns	<code>ip netns exec <ns> <cmd></code>
Show namespace of a process	<code>ls -l /proc/<pid>/ns/</code>

10. KEY TAKEAWAYS

- ✓ Namespaces provide isolation while sharing the same kernel.
- ✓ PID, network, mount, user, IPC and UTS namespaces isolate different resources.
- ✓ Used by containers to create isolated environments.
- ✓ Changes inside a namespace do not affect the host.
- ✓ Understand namespace use cases and limitations.
- ✓ Use the right tools (unshare, ip netns, nsenter) for investigation.
- ✓ Namespaces are a foundation for container technologies like Docker and Kubernetes.
- ✓ Combine namespaces with cgroups for complete container isolation.



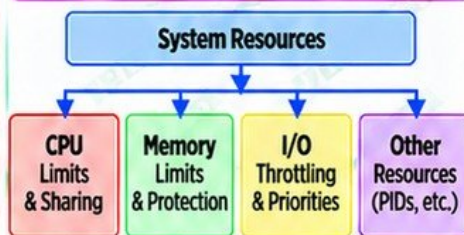
CPU, memory, I/O controls, pressure, limits, container resource behavior

1. WHAT ARE CGROUPS?

Control groups (cgroups) limit, monitor and manage the resources (CPU, memory, I/O, etc.) used by a group of processes.

Benefits

- Prevent resource exhaustion
- Ensure fair resource usage
- Improve system stability
- Used by containers (Docker, Kubernetes)



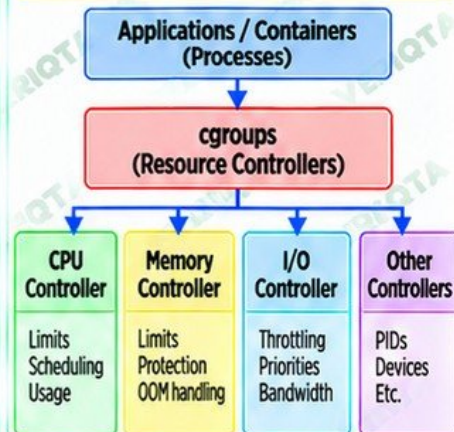
2. CGROUPS V1 vs V2 (MODERN)

cgroups v2 provides a unified hierarchy and simpler management compared to v1.

cgroups v1	cgroups v2
Multiple hierarchies (per resource)	Unified hierarchy (all resources)
More complex configuration	Simpler and cleaner
Older systems	Modern Linux (recommended)
Separate mount points	Single mount point /sys/fs/cgroup
Common in older distributions	Default in most recent distributions (Ubuntu 22.04+, RHEL 9+)

3. CGROUPS ARCHITECTURE

Processes are placed in cgroups and resource controllers enforce limits and track usage.



4. CPU RESOURCE CONTROL

Control how much CPU a process or container can use.

```
# Limit CPU to 1 core (cgroups v2)
mkdir -p /sys/fs/cgroup/demo
cd /sys/fs/cgroup/demo
echo 100000 > cpu.max # 100ms of 1 CPU
echo $$ > cgroup.procs
```

Parameter (v2)	Description
cpu.max	Limit CPU usage (quota)
cpu.weight	Relative CPU weight (1-10000)
cpu.stat	View CPU usage statistics

5. MEMORY RESOURCE CONTROL

Set memory limits and prevent the system from running out of memory.

```
# Limit memory to 512MB (cgroups v2)
mkdir -p /sys/fs/cgroup/demo
echo 536870912 > /sys/fs/cgroup/demo/
memory.max
echo $$ > /sys/fs/cgroup/demo/cgroup.procs
```

Parameter (v2)	Description
memory.max	Set memory limit (bytes)
memory.high	Soft limit (throttle before limit)
memory.current	Current memory usage
memory.events	View events (oom, high, max)

6. I/O RESOURCE CONTROL

Limit disk I/O bandwidth and I/O operations.

```
# Limit write speed to 10 MB/s (cgroups v2)
echo "10M wbps" > /sys/fs/cgroup/demo/
io.max
echo "8:0 10M" > /sys/fs/cgroup/demo/
io.max # device major:minor
```

Parameter (v2)	Description
io.max	Limit I/O bandwidth (rbps/wbps)
io.weight	Set I/O weight (1-10000)
io.stat	View I/O statistics

7. OBSERVABILITY & PRESSURE

Monitor resource usage and pressure to detect resource contention.

```
# View cgroup resource usage
cat /sys/fs/cgroup/cpu.stat
cat /sys/fs/cgroup/memory.current
cat /sys/fs/cgroup/io.stat
# Check pressure (PSI)
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

File	Description
cpu.stat	CPU usage and throttling stats
memory.current	Current memory usage
io.stat	I/O usage statistics
/proc/pressure/*	Pressure Stall Information (PSI)

8. CONTAINER RESOURCE EXAMPLE

Set resource limits for a Docker container (using cgroups).

```
# Run a container with resource limits
docker run -d --name app \
--cpus="1.0" \
--memory="512m" \
--memory=swap="512m" \
--blkio-weight=500
nginx
```

Option	Description
--cpus	Limit CPU cores
--memory	Set memory limit
--memory=swap	Set memory + swap limit
--blkio-weight	Set I/O weight
--pids-limit	Limit number of processes

9. COMMON USE CASES

- ⚙️ Limit container resources in production environments.
- 🛡️ Prevent noisy neighbors from consuming all resources.
- 📊 Ensure fair CPU, memory and I/O usage.
- 💾 Protect the system from OOM (Out of Memory).
- 👥 Used by Docker, Kubernetes and systemd.
- 🔍 Monitor and troubleshoot resource issues.

10. TROUBLESHOOTING FLOW

- 1 Check current usage (cpu.stat, memory.current).
- 2 Look for throttling or pressure (PSI).
- 3 Verify cgroup limits and configuration.
- 4 Check container runtime settings (Docker/K8s).
- 5 Adjust limits and monitor again.

11. KEY TAKEAWAYS

- ✓ cgroups limit, monitor and manage system resources.
- ✓ Control CPU, memory, I/O and other resources.
- ✓ cgroups v2 provides a unified and simpler interface.
- ✓ Used by containers to enforce resource limits.

- ✓ Always monitor resource usage and pressure.
- ✓ Set appropriate limits to prevent resource exhaustion.
- ✓ Understand container runtime options for resource governance.
- ✓ Use cgroups to improve stability, performance and multi-tenancy.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



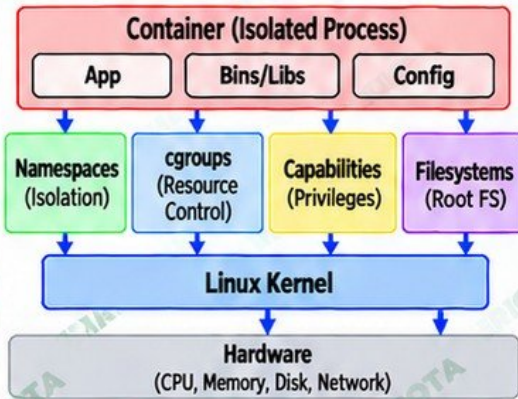
@veriqta

27

Namespaces + cgroups + capabilities + filesystems, what containers really are

1. WHAT CONTAINERS REALLY ARE

A container is a set of Linux kernel features that isolate processes and control resources, not a lightweight virtual machine.



2. LINUX KERNEL FEATURES USED

Namespaces	Isolate process view of the system (PID, network, mount, user, IPC, UTS).
cgroups	Control and limit resource usage (CPU, memory, I/O, pids).
Capabilities	Fine-grained privileges instead of full root access.
Filesystems	Use a separate root filesystem (layers, read-only + writable).
Other Features	Seccomp, LSM (AppArmor/SELinux), namespaces /proc and /sys.

3. CONTAINER VS VIRTUAL MACHINE

Container	Virtual Machine
✓ Shares host kernel	• Own guest kernel
✓ Uses namespaces	• Full OS isolation
✓ Uses cgroups	• Heavier resource usage
✓ Lightweight	• Slower to start
✓ Starts in seconds	• Larger size
✓ Smaller size	• Stronger isolation
✓ Process level isolation	• Hardware virtualisation (e.g. KVM, VMware)

4. NAMESPACES OVERVIEW

Each namespace gives a process its own isolated view of specific system resources.

Namespace	Isolates	What it Means
PID	Process IDs	Processes see their own PID 1 and process tree.
Network	Network stack	Own interfaces, routes, ports, iptables.
Mount	Filesystem mounts	Own mount points and filesystem view.
User	User and group IDs	Own UID/GID (root inside container can be non-root on host).
IPC	IPC resources	Own message queues, shared memory, semaphores.
UTS	Hostname and domain	Own hostname and domain name.

5. CGROUPS OVERVIEW

cgroups limit, monitor and manage resources used by a container.

Resource	cgroup Controller	Examples
CPU	cpu, cpuset	Limit CPU usage and assign CPUs.
Memory	memory	Set memory limits and monitor usage.
I/O	io, blkio	Control disk I/O bandwidth and IOPS.
PIDs	pids	Limit number of processes.
Devices	devices	Restrict access to device files.
Other	hugetlb, freezer, perf_event	Specialized resource controls.

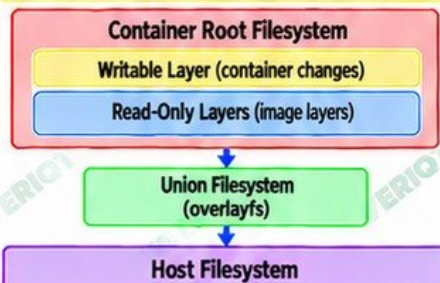
6. CAPABILITIES OVERVIEW

Capabilities split the root privileges into smaller permissions.

Capability	Purpose
CAP_NET_ADMIN	Manage network devices, routes, firewall rules.
CAP_SYS_ADMIN	Perform many administrative operations.
CAP_SYS_CHROOT	Use chroot and change root directory.
CAP_NET_BIND_SERVICE	Bind to privileged ports (less than 1024).
CAP_DAC_OVERRIDE	Bypass file read, write and execute permissions.
CAP_SETUID	Set user IDs.
CAP_SETGID	Set group IDs.
CAP_SYS_TIME	Change system time.

7. FILESYSTEMS IN CONTAINERS

Containers use a separate root filesystem with multiple layers and writable layer.



8. SIMPLE CONTAINER EXAMPLE

Run a container and observe namespaces and cgroups.

```
# Run a container
docker run -it --name demo nginx

# Check namespaces (inside container)
ls -l /proc/1/ns

# Check cgroup info
cat /proc/1/cgroup

# Check hostname (UTS namespace)
hostname

# Check network interfaces
ip a

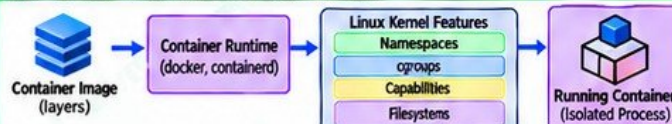
# Check mounts
mount | head -n 10
```

9. INSIDE VS OUTSIDE VIEW

The same process looks different inside and outside the container.

On Host	Inside Container
<pre>\$ docker ps CONTAINER ID NAME IMAGE 8f3e2d1a7c9b demo nginx</pre>	<pre>\$ ps -ef PID CMD 1 nginx 7 bash</pre>
<pre>\$ ps -ef grep nginx root 1234 ? 00:00 nginx</pre>	<pre>\$ ip a grep eth0 2: eth0: <...> inet 172.17.0.2/16</pre>
<pre>\$ ip a grep eth0 2: eth0: <...> inet 192.168.1.10/24</pre>	<pre>\$ hostname demo</pre>

10. REAL CONTAINER ARCHITECTURE



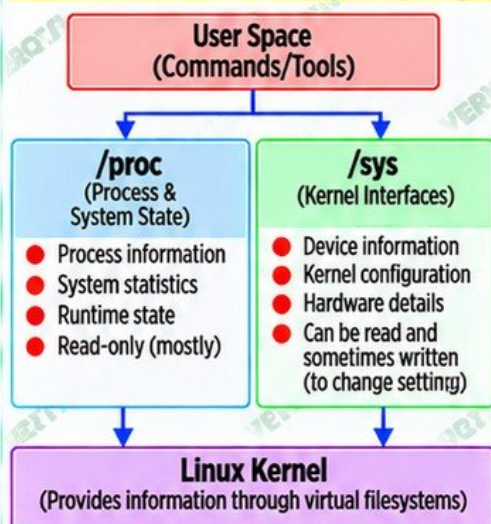
11. KEY TAKEAWAYS

- ✓ Containers are isolated processes, not virtual machines.
- ✓ Namespaces provide isolation for processes, network, mounts, users, IPC and hostname.
- ✓ cgroups control CPU, memory, I/O and other resources.
- ✓ Capabilities give fine-grained privileges instead of full root access.
- ✓ Containers use a layered filesystem with a writable layer.
- ✓ All these features are built into the Linux kernel.
- ✓ Understand these components to troubleshoot and secure containers effectively.

Runtime state, kernel interfaces, process evidence, safe inspection

1. OVERVIEW: WHAT ARE /proc AND /sys?

/proc and /sys are virtual filesystems that expose kernel information and runtime state. They are not regular files stored on disk.



2. COMMON /proc PATHS

/proc shows process information and system runtime state.

Path	Purpose
/proc/[pid]/	Information about a process
/proc/[pid]/status	Process status (UID, GID, etc.)
/proc/[pid]/cmdline	Process command line
/proc/[pid]/environ	Process environment variables
/proc/[pid]/fd/	Open file descriptors
/proc/[pid]/maps	Memory mappings
/proc/cpuinfo	CPU information
/proc/meminfo	Memory information
/proc/loadavg	System load averages
/proc/uptime	System uptime
/proc/partitions	Block device partitions
/proc/net/	Network stack information
/proc/sys/	Kernel parameters (same as /sys for many)

3. COMMON /sys PATHS

/sys exposes kernel objects, devices and configuration interfaces.

Path	Purpose
/sys/class/	Device classes (block, net, etc.)
/sys/class/net/	Network interface details
/sys/class/block/	Block devices (disks)
/sys/devices/	Kernel device tree
/sys/fs/	Filesystem information
/sys/module/	Loaded kernel modules
/sys/kernel/	Kernel features and settings
/sys/power/	Power management settings
/sys/bus/	Device bus information (PCI, USB, etc.)
/sys/block/	Block device details
/sys/firmware/	Firmware information

4. PROCESS INFORMATION EXAMPLES

Inspect a running process using /proc.

```
# Check process status
cat /proc/1234/status

# View command line
cat /proc/1234/cmdline

# View environment variables
tr '\0' '\n' < /proc/1234/environ

# List open files
ls -l /proc/1234/fd/

# View memory mappings
cat /proc/1234/maps | head -n 10
```

5. SYSTEM INFORMATION EXAMPLES

Get system runtime information from /proc.

```
# CPU information
cat /proc/cpuinfo | head -n 10

# Memory information
cat /proc/meminfo

# System load
cat /proc/loadavg

# Uptime
cat /proc/uptime

# Kernel version
cat /proc/version

# Mounted filesystems
cat /proc/mounts
```

6. DEVICE AND NETWORK EXAMPLES

Inspect devices and network interfaces using /sys.

```
# List network interfaces
ls -l /sys/class/net/

# View network interface details
cat /sys/class/net/eth0/operstate
cat /sys/class/net/eth0/speed

# List block devices
ls -l /sys/class/block/

# View device information
cat /sys/block/sda/queue/scheduler
cat /sys/block/sda/size

# List kernel modules
ls -l /sys/module/
```

7. REAL EXAMPLES

Practical use cases for troubleshooting.

Scenario	Command
Find why a process is using CPU	cat /proc/[pid]/stat
Check memory usage of a process	cat /proc/[pid]/status
See open network connections	cat /proc/net/tcp
Check interface state	cat /sys/class/net/eth0/operstate
Find disk scheduler	cat /sys/block/sda/queue/scheduler
Verify loaded kernel modules	ls /sys/module/
Check system boot time	cat /proc/stat grep btime

8. IMPORTANT NOTES

- 1 /proc and /sys are virtual filesystems provided by the kernel.
- 2 Information is real-time and reflects the current state.
- 3 Most files in /proc are read-only.
- 4 Some files in /sys can be written to change kernel behavior (use with care).
- 5 Not all information is available to non-root users.
- 6 Contents can vary between kernel versions and distributions.
- 7 Use these interfaces for troubleshooting, monitoring and automation.

9. SAFE INSPECTION PRACTICES

- ✓ Use read-only commands first (cat, ls).
- ✓ Avoid writing to /sys unless you understand the impact.
- ✓ Do not modify kernel parameters on production systems without testing.
- ✓ Use appropriate permissions (sudo when required).
- ✓ Check documentation (man proc, man sysfs).
- ✓ Compare output with expected values.
- ✓ Use these interfaces for troubleshooting, monitoring and learning.

10. KEY TAKEAWAYS

- ✓ /proc provides process and system runtime information.
- ✓ /sys provides kernel interfaces, devices and configuration.
- ✓ Both are virtual filesystems, not regular disk files.
- ✓ They are essential for troubleshooting, monitoring and understanding the Linux kernel.
- ✓ Use read-only inspection commands safely.
- ✓ Some /sys files can change kernel behavior.
- ✓ Information may vary by kernel version and distribution.
- ✓ These interfaces give powerful visibility into how Linux works.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



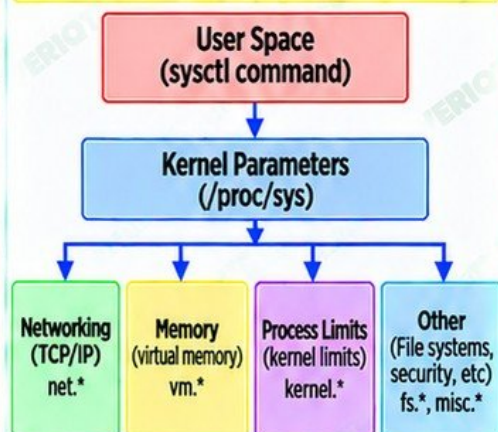
@veriqta

29

Networking, memory, limits, runtime tuning, persistent configuration risks

1. WHAT ARE KERNEL PARAMETERS?

Kernel parameters control how the Linux kernel behaves (networking, memory, limits, security, etc). You can view and change them using sysctl.



2. COMMON sysctl COMMANDS

Use sysctl to view, search and change kernel parameters at runtime.

Task	Command
View all parameters	<code>sysctl -a</code>
View a specific parameter	<code>sysctl net.ipv4.ip_forward</code>
Search for parameters	<code>sysctl -a grep keyword</code>
View current value	<code>sysctl vm.swappiness</code>
Change value (runtime)	<code>sudo sysctl -w vm.swappiness=10</code>
Load from a file	<code>sudo sysctl -p /etc/sysctl.conf</code>
Show only non-defaults	<code>sysctl -a --ignore-kernel grep -v "="</code>
Save to a file	<code>sysctl -a > sysctl-backup.conf</code>

3. CONFIGURATION FILES

Make changes persistent using configuration files.

File	Purpose
<code>/etc/sysctl.conf</code>	Main configuration file (global settings)
<code>/etc/sysctl.d/*.conf</code>	Additional configuration files (recommended)
<code>/usr/lib/sysctl.d/</code>	Default settings (distribution provided)
<code>/run/sysctl.d/</code>	Runtime configuration (temporary)

Example

```
# Make a change persistent
sudo nano /etc/sysctl.d/99-custom.conf
# Example
net.ipv4.ip_forward = 1
vm.swappiness = 10
# Apply changes
sudo sysctl -p /etc/sysctl.d/99-custom.conf
```

4. COMMON NETWORKING PARAMETERS

Important sysctl settings for TCP/IP networking.

Parameter	Default	Purpose
<code>net.ipv4.ip_forward</code>	0	Enable IP forwarding (routing)
<code>net.ipv4.tcp_syncookies</code>	1	Protect against SYN floods
<code>net.ipv4.tcp_max_syn_backlog</code>	256	SYN backlog queue size
<code>net.core.somaxconn</code>	4096	Max pending connections
<code>net.ipv4.tcp_fin_timeout</code>	60	TIME_WAIT duration
<code>net.ipv4.tcp_keepalive_time</code>	7200	Keepalive time (seconds)

Example

```
# Enable IP forwarding (runtime)
sudo sysctl -w net.ipv4.ip_forward=1
# Check value
sysctl net.ipv4.ip_forward
```

5. COMMON MEMORY PARAMETERS

Control virtual memory behavior and memory management.

Parameter	Default	Purpose
<code>vm.swappiness</code>	60	Tendency to use swap (0 = less, 100 = more)
<code>vm.dirty_ratio</code>	20	Percentage of memory for dirty pages
<code>vm.dirty_background_ratio</code>	10	Background write threshold
<code>vm.overcommit_memory</code>	0	Memory overcommit behavior
<code>vm.overcommit_ratio</code>	50	Overcommit limit (when mode = 2)
<code>vm.min_free_kbytes</code>	67584	Minimum free memory

Example

```
# Reduce swapping (runtime)
sudo sysctl -w vm.swappiness=10
# Check value
sysctl vm.swappiness
```

6. PROCESS AND SYSTEM LIMITS

Kernel parameters that affect process limits and system behavior.

Parameter	Default	Purpose
<code>kernel.pid_max</code>	4194304	Maximum process IDs
<code>kernel.threads-max</code>	6700416	Maximum threads
<code>kernel.sem</code>	32000 1024000 500 32000	Semaphore limits
<code>kernel.shmmax</code>	1844674407...	Maximum shared memory segment size
<code>fs.file-max</code>	9223372036...	Maximum open files (system-wide)
<code>fs.nr_open</code>	1048576	Maximum open files (per process)

Example

```
# Increase maximum open files (runtime)
sudo sysctl -w fs.file-max=2097152
# Check value
sysctl fs.file-max
```

7. RUNTIME VS PERSISTENT CHANGES

Understand the difference between temporary and permanent configuration.

Runtime Change (Temporary)	Persistent Change (Permanent)
- Use <code>sysctl -w</code> - Applies immediately - Lost after reboot - Good for testing - Does not modify config files	- Add to <code>/etc/sysctl.conf</code> or <code>/etc/sysctl.d/*.conf</code> - Apply with <code>sysctl -p</code> - Survives reboot - Use for production - Keep configuration files version controlled

8. RISKS AND BEST PRACTICES

Changing kernel parameters can improve performance but also cause issues.

Potential Risks

- Incorrect values can cause system instability.
- Networking changes can expose the system.
- Memory settings can affect application behavior.
- Some parameters may not be supported on all distributions.
- Changes can be overwritten by updates.

Best Practices

- Test changes in a non-production environment.
- Document all changes.
- Use configuration files with version control.
- Understand the impact of each parameter.
- Monitor system behavior after changes.

9. TROUBLESHOOTING

How to verify, troubleshoot and rollback changes.

Scenario	Command
Check current value	<code>sysctl parameter.name</code>
Verify all settings	<code>sysctl -a grep keyword</code>
Check loaded config files	<code>sysctl --system</code>
Find parameter location	<code>sysctl -a grep parameter</code>
Revert a runtime change	<code>sudo sysctl -w parameter=default_value</code>
Check kernel messages	<code>dmesg grep -i sysctl</code>
Troubleshoot issues	<code>journalctl -k grep parameter</code>

10. KEY TAKEAWAYS

- Kernel parameters control networking, memory, limits and other kernel behavior.
- Use `sysctl` to view and change parameters.
- Changes can be temporary (runtime) or persistent (configuration files).
- Common parameters include `net.*`, `vm.*`, `kernel.*` and `fs.*`.

- Test changes before applying in production.
- Incorrect values can cause system instability or security issues.
- Monitor system behavior after changes.
- Keep configuration files documented and version controlled.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Least privilege, PAM, SSH, sudo, auditing, hardening, attack surface

1. LINUX SECURITY OVERVIEW

Linux security engineering is about reducing risk by controlling access, hardening the system, monitoring activity and minimizing the attack surface.



2. LEAST PRIVILEGE PRINCIPLE

Give users and services only the minimum permissions required to do their job.

- ✓ Use regular users, avoid root.
- ✓ Use sudo for administrative tasks.
- ✓ Restrict file and directory permissions.
- ✓ Limit service accounts.
- ✓ Remove unnecessary privileges.
- ✓ Use role-based access.
- ✓ Review permissions regularly.

3. PAM (PLUGABLE AUTHENTICATION)

PAM controls how users authenticate and what happens during login (local and remote).

Common PAM Components

pam_unix	Local password authentication
pam_ldap	Integration with LDAP/AD
pam_faildelay	Delay after failed login
pam_limits	User resource limits
pam_tally2	Account lockout after failures
pam_faillock	Track failed login attempts

Example: View PAM configuration

```
$ cat /etc/pam.d/common-auth
auth required pam_unix.so
auth required pam_faillock.so
```

4. SSH SECURITY

Secure SSH to prevent unauthorized access.

Key SSH Hardening Settings (/etc/ssh/sshd_config)

```
Port 2222
PermitRootLogin no
PasswordAuthentication no
AllowUsers devops admin
ClientAliveInterval 300
MaxAuthTries 3
```

Useful Commands

```
$ systemctl restart sshd
$ ss -tlnp | grep :2222
$ journalctl -u ssh -f
```

5. SUDO AND PRIVILEGE MANAGEMENT

Use sudo to allow specific administrative access with control and auditing.

Sudo Best Practices

- Do not give full root access to everyone.
- Use sudo with specific commands.
- Use groups for role-based access.
- Enable logging and review usage.
- Remove access when no longer needed.

Example: Allow a user to restart nginx only

```
$ visudo
devops ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx
```

Check sudo logs

```
$ journalctl -u sudo
$ grep 'sudo' /var/log/auth.log
```

6. AUDITING AND LOGGING

Monitor and record important security events.

Key Logs to Monitor

/var/log/auth.log	Authentication events
/var/log/secure	Login and sudo activity (RHEL)
/var/log/audit/audit.log	Audit framework logs
/var/log/syslog	General system activity

Audit with auditd

```
$ sudo apt install auditd
$ sudo systemctl enable --now auditd
$ sudo auditctl -w /etc/passwd -p wa -k identity
```

7. SYSTEM HARDENING

Apply secure configurations to reduce risk.

Hardening Checklist

- 1 Keep the system updated.
- 2 Disable unnecessary services.
- 3 Use a firewall (UFW or firewalld).
- 4 Configure strong password policies.
- 5 Set correct file permissions.
- 6 Disable unused kernel modules.
- 7 Use secure SSH configuration.
- 8 Implement logging and monitoring.

8. REDUCE ATTACK SURFACE

Minimize exposed services, open ports and unnecessary software.

Steps

- ✓ List and disable unused services.
- ✓ Check open ports.
- ✓ Remove unnecessary packages.
- ✓ Disable unused kernel modules.
- ✓ Restrict network access with firewall rules.
- ✓ Limit services to required interfaces only.

Find open ports and services

```
$ ss -tlnp
$ lsof -i -P -n
$ systemctl list-unit-files --type=service
```

9. SECURITY MONITORING TOOLS

Useful tools for security monitoring and detection.

Tool	Purpose
fail2ban	Block suspicious login attempts
rkhunter	Detect rootkits
clamav	Malware scanning
aide	File integrity monitoring
ufw/firewalld	Firewall management
lynis	Security auditing and hardening
auditd	System auditing
osquery	Host monitoring and forensics

Install and enable fail2ban

```
$ sudo apt install fail2ban
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status
```

10. KEY TAKEAWAYS

- ✓ Use least privilege for users and services.
- ✓ Secure SSH, sudo and PAM configurations.
- ✓ Apply system hardening and reduce the attack surface.
- ✓ Enable auditing and monitor security logs.
- ✓ Keep the system updated and remove unnecessary software.
- ✓ Use the right tools for monitoring and threat detection.
- ✓ Regularly review access, permissions and configurations.
- ✓ Design security as a continuous process.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Authentication flow, keys, agents, host verification, bastions, debugging

1. SSH AUTHENTICATION FLOW

SSH uses key or password authentication to securely connect to a remote server.



2. SSH KEY AUTHENTICATION

Use SSH keys for secure, passwordless authentication.

Generate a key pair (on client)

```
$ ssh-keygen -t ed25519 -C "your_email"
# Private key: ~/.ssh/id_ed25519
# Public key: ~/.ssh/id_ed25519.pub
```

Copy public key to server

```
$ ssh-copy-id user@server
# or manually:
$ cat ~/.ssh/id_ed25519.pub | \
ssh user@server "mkdir -p ~/.ssh && \
cat >> ~/.ssh/authorized_keys"
```

3. SSH AGENT (KEY MANAGEMENT)

Use ssh-agent to manage keys without repeatedly entering passphrases.

```
$ Start the agent
$ eval "$(ssh-agent -s)"

# Add your key
$ ssh-add ~/.ssh/id_ed25519

# List loaded keys
$ ssh-add -l

# Remove a key
$ ssh-add -d ~/.ssh/id_ed25519
```

4. HOST VERIFICATION

SSH verifies the server's identity using a host key to prevent man-in-the-middle attacks.

First time connection

```
$ ssh user@server
# You will see a fingerprint and be asked to confirm.
```

View stored host keys

```
$ cat ~/.ssh/known_hosts
```

Manually verify a host key

```
$ ssh-keyscan -t ed25519 server | ssh-keygen -lf -
# Compare the fingerprint with the server.
```

5. COMMON SSH CONFIGURATION

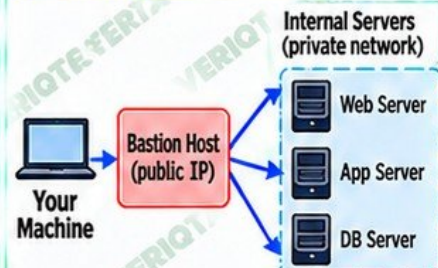
Use ~/.ssh/config to simplify and secure connections.

```
# ~/.ssh/config
Host dev
  HostName 192.168.10.10
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  Port 22
  ServerAliveInterval 60

Host prod
  HostName prod.example.com
  User ec2-user
  IdentityFile ~/.ssh/prod_key
  Port 22
  ProxyJump bastion
  ServerAliveInterval 60
```

6. BASTION HOST (JUMP SERVER)

Use a bastion host to securely access internal servers.



Connect using ProxyJump

```
$ ssh -J ubuntu@bastion user@internal-server
```

7. DEBUGGING SSH CONNECTIONS

Use verbose mode to troubleshoot connection issues.

```
$ ssh -vvv user@server
# Check key being used
# Look for authentication errors
# Identify network or host key issues
# Useful for production troubleshooting
```

8. COMMON SSH ISSUES & FIXES

Understand common problems and how to resolve them.

Issue	Cause	Fix
Permission denied	Wrong key or user	Check user and key, ssh -vvv
Connection timed out	Network or firewall	Check connectivity, port 22
Host key changed	Server rebuilt	Remove old key from known_hosts
Too many authentication failures	Multiple keys tried	Use correct key, adjust ssh config
Permission denied (publickey)	Key not in authorized_keys	Add public key to server
Connection closed	SSH policy or timeout	Check server logs, adjust timeouts

9. PRODUCTION BEST PRACTICES

- ✓ Use key-based authentication (avoid passwords).
- ✓ Use strong key types (ed25519 or RSA 4096).
- ✓ Protect private keys with passphrases.
- ✓ Use ssh-agent for key management.
- ✓ Verify server host keys.
- ✓ Use bastion hosts for internal servers.
- ✓ Restrict SSH access (firewall, security groups).
- ✓ Disable root login and use sudo.
- ✓ Enable logging and monitoring.
- ✓ Regularly rotate and audit keys.

10. KEY TAKEAWAYS

- 1 SSH securely encrypts communication between client and server.
- 2 Use key-based authentication and ssh-agent in production.
- 3 Always verify server host keys to prevent attacks.
- 4 Use bastion hosts to access internal servers.
- 5 Use SSH configuration files to simplify and secure connections.
- 6 Debug with verbose mode and check logs for issues.
- 7 Follow security best practices to reduce risk and protect systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



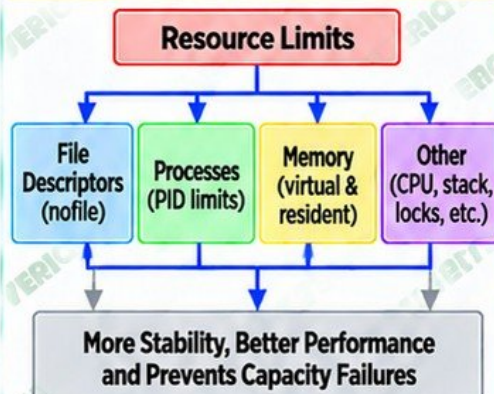
@veriqta

32

ulimit, nfile, PID limits, sockets, memory, capacity exhaustion

1. OVERVIEW: WHAT ARE RESOURCE LIMITS?

Resource limits restrict how many system resources a process or user can use. They prevent abuse, improve stability and help the system stay available.



2. CHECK CURRENT LIMITS

Use ulimit and system files to check current limits.

User limits (current shell)

```
$ ulimit -a
```

View specific limits

```
$ ulimit -n # open files
$ ulimit -u # max user processes
$ ulimit -v # virtual memory (KB)
$ ulimit -m # resident memory (KB)
```

System-wide limits

```
$ cat /proc/sys/fs/file-max
$ cat /proc/sys/kernel/pid_max
$ sysctl fs.nr_open
$ sysctl kernel.pid_max
```

3. COMMON RESOURCE LIMITS

Key limits and what they control.

Limit	Command / File	Purpose
nofile	ulimit -n	Maximum open file descriptors
nproc	ulimit -u	Maximum user processes (PIDs)
stack	ulimit -s	Stack size (KB)
memlock	ulimit -l	Locked memory (KB)
as	ulimit -v	Virtual memory (KB)
rss	ulimit -m	Resident memory (KB)
file-max	/proc/sys/fs/file-max	System-wide max open files
pid_max	/proc/sys/kernel/pid_max	Maximum PID number

4. FILE DESCRIPTORS (NOFILE)

Each open file, socket or connection uses a file descriptor. Hitting the limit can cause service failures.

Check current nofile limit

```
$ ulimit -n
```

Check system-wide file limits

```
$ cat /proc/sys/fs/file-max
$ cat /proc/sys/fs/nr_open
$ cat /proc/sys/fs/file-nr
```

Increase nofile (temporary)

```
$ ulimit -n 65535
```

Persistent configuration (systemd)

```
# /etc/systemd/system.conf
DefaultLimitNOFILE=65535
```

5. PROCESS LIMITS (PIDS)

Each process uses a PID. When PID limits are reached, new processes cannot start.

Check current user process limit

```
$ ulimit -u
```

Check system PID limits

```
$ cat /proc/sys/kernel/pid_max
$ pgrep -u $(whoami) | wc -l
```

Change PID limit (temporary)

```
$ ulimit -u 4096
```

Persistent configuration (/etc/security/limits.conf)

```
# Add for a user or group
user soft nproc 4096
user hard nproc 8192
```

6. MEMORY LIMITS

Memory limits control how much memory a process can use. Exceeding limits can cause failures or OOM kills.

Check memory limits

```
$ ulimit -v # virtual memory (KB)
$ ulimit -m # resident memory (KB)
```

View memory usage

```
$ free -h
$ cat /proc/meminfo
$ top
$ htop
```

Example: Limit memory for a process

```
$ ulimit -v 1048576 # 1 GB (KB)
$ some_application
```

7. SOCKET AND NETWORK CAPACITY

Each network connection uses file descriptors and kernel memory. High connections can exhaust resources.

Check current connections

```
$ ss -s
$ ss -ant | wc -l
$ netstat -an | wc -l
```

Key network-related limits

```
$ cat /proc/sys/net/core/somaxconn
$ cat /proc/sys/net/ipv4/tcp_max_syn_backlog
$ cat /proc/sys/net/ipv4/ip_local_port_range
```

8. SIGNS OF CAPACITY EXHAUSTION

Common symptoms when resource limits are reached.

Symptom	Possible Cause
Too many open files	nofile limit reached
Cannot fork	PID limit reached
Out of memory (OOM)	Memory exhausted
Connection refused	Socket or file limit reached
High load, processes failing	Process limit or memory issue
Services not starting	Limits too low
Slow performance	Resource exhaustion

9. TROUBLESHOOTING EXAMPLES

Real examples and how to investigate.

Example 1: Too many open files

```
$ tail -n 5 /var/log/syslog
myapp[1234]: Too many open files
```

Example 2: Cannot fork new process

```
$ dmesg | tail -n 5
fork: retry: Resource temporarily
unavailable
```

Example 3: Service failing due to memory

```
$ journalctl -u myapp -n 20
Out of memory: Kill process 1234 (myapp)
```

10. BEST PRACTICES

- Set appropriate limits based on workload requirements.
- Monitor resource usage regularly.
- Use systemd or /etc/security/limits.conf for persistent configuration.
- Test changes in a non-production environment.
- Set alerts for high usage (files, PIDs, memory).
- Tune kernel parameters for high-connection environments.
- Document all limit changes.

11. KEY TAKEAWAYS

- Resource limits protect the system and prevent abuse.
- Important limits include nofile, nproc, memory and network sockets.
- Check current limits using ulimit, /proc and sysctl.
- Increase limits carefully and make persistent changes for production.
- Monitor for signs of capacity exhaustion.
- Proper limits improve stability and prevent service failures.
- Always test changes and monitor the impact.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

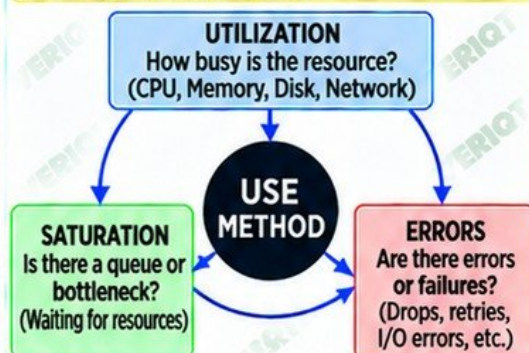


@veriqta

USE method, CPU, memory, disk, network, latency, saturation, evidence

1. OVERVIEW: WHAT IS LINUX PERFORMANCE INVESTIGATION?

Performance investigation is the process of finding what is slow, why it is slow, and how to fix it using real evidence from the system.



Find the real cause using metrics, logs and system evidence. Do not guess.

2. SYSTEM OVERVIEW CHECK

Start with a high-level view of system health.

```
# System information
uname -a
hostnamectl
uptime

# Overall resource usage
top
htop

# Quick summary
free -h
df -h
vmstat 1 5

# Check for recent issues
dmesg | tail -n 50
journalctl -p err -n 50
```

3. CPU INVESTIGATION

Check CPU usage, load, processes and context switching.

```
# CPU usage
top
htop
mpstat -P ALL 1

# Load average
uptime
cat /proc/loadavg

# Find top CPU processes
ps aux --sort=-%cpu | head -n 10

# CPU details
lscpu
cat /proc/cpuinfo

# Context switches and interrupts
vmstat 1 5
pidstat -w 1 5
```

4. MEMORY INVESTIGATION

Check memory usage, pressure and identify processes using memory.

```
# Memory usage
free -h
cat /proc/meminfo

# Real-time view
top
htop

# Find top memory processes
ps aux --sort=-%mem | head -n 10

# Check for memory pressure
cat /proc/pressure/memory

# Check for swapping
swapon --show
vmstat 1 5
```

5. DISK INVESTIGATION

Check disk usage, I/O, latency and file system performance.

```
# Disk space usage
df -h

# Disk I/O usage
iostat -x 1 5

# Disk latency
iotop

# Find processes using disk
lsof | grep deleted

# Check file system details
lsblk

# Check I/O errors
dmesg | grep -i "error"

# Monitor disk activity
pidstat -d 1 5
```

6. NETWORK INVESTIGATION

Check network usage, connections, latency and errors.

```
# Network interfaces
ip a

# Network statistics
sar -n DEV 1 5

# Active connections
ss -tulnp

# Network latency
ping -c 4 google.com

# Check for packet loss
mtr google.com

# Find processes using network
lsof -i -n -P

# Check network errors
dmesg | grep -i network
```

7. LATENCY AND RESPONSE TIME

Measure system and application latency.

```
# System latency
time command

# Network latency
ping -c 4 target.com

# Trace latency to a host
traceroute target.com

# Check DNS resolution time
dig target.com

# Monitor real-time latency
mtr -rw target.com
```

8. SIGNS OF RESOURCE SATURATION

Common indicators that a resource is saturated.

Resource	Signs of Saturation	Commands to Check
CPU	High CPU usage High load average	top, mpstat, uptime
Memory	High memory usage Swap usage	free, vmstat, cat /proc/pressure/memory
Disk	High I/O wait Slow read/write	iostat, iotop, df, vmstat
Network	High network usage Packet loss	sar -n DEV, ss, mtr
System	Process queue High latency	vmstat, dmesg, journalctl

9. REAL EXAMPLE: INVESTIGATE SLOW SERVER

Step-by-step investigation flow using USE method.

- 1 Check overall system health (uptime, top, free, df).
- 2 Identify which resource is slow (USE method).
- 3 Collect detailed metrics for that resource.
- 4 Find the process or application causing the issue.
- 5 Check logs (dmesg, journalctl, application logs).
- 6 Look for errors, throttling or resource limits.
- 7 Make a change, test again and verify improvement.

10. BEST PRACTICES

- Use the USE method to investigate performance issues.
- Always use real metrics and logs, not guesses.
- Check all major resources (CPU, memory, disk, network, latency).
- Look for signs of saturation and errors.
- Monitor resource usage regularly.
- Document findings and solutions.
- Set up monitoring and alerts for early detection.
- Test changes and verify improvement.

11. KEY TAKEAWAYS

- 1 USE method: Utilization, Saturation, Errors.
- 2 Check CPU, memory, disk, network and latency.
- 3 Use the right tools to collect evidence.
- 4 Identify the real cause, not just the symptom.
- 5 Look for saturation, bottlenecks and errors.
- 6 Document and fix the issue.
- 7 Monitoring and alerts help prevent future problems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Symptom → blast radius → evidence → hypothesis → mitigation → verification

1. OVERVIEW: THE TROUBLESHOOTING FLOW

A structured approach to find and fix problems in production. It helps you stay focused, reduce MTTR and prevent repeat incidents.



5. STEP 4: FORM A HYPOTHESIS

Use the evidence to identify possible root cause(s).

Common Hypotheses

- High CPU or memory usage.
- Disk full or high I/O.
- Network latency or packet loss.
- A recent deployment caused the issue.
- A configuration change or restart.
- A dependency service is down.
- Resource limits (ulimit, nfile, PID).

2. STEP 1: IDENTIFY THE SYMPTOM

Clearly understand what is wrong and when it started.

Key Questions

- What is the exact issue?
- When did it start?
- Is it a complete outage or degradation?
- Are there error messages?
- Is it user-facing or internal?

Example

Users report: "The website is slow and times out occasionally."

4. STEP 3: COLLECT EVIDENCE

Gather data from multiple sources.

Sources of Evidence

- System metrics (CPU, memory, disk, network)
- Logs (system, application, service)
- Traces (if available)
- Configuration changes
- Recent deployments or updates
- System events and alerts
- User reports and error messages

6. STEP 5: MITIGATE THE ISSUE

Apply a fix or workaround to reduce the impact.

Mitigation Examples

- Restart a failed service.
- Scale up resources (CPU, memory).
- Clear disk space.
- Roll back a recent deployment.
- Fix a misconfiguration.
- Use a temporary workaround.
- Enable rate limiting to protect the system.

3. STEP 2: FIND THE BLAST RADIUS

Determine the scope and impact of the issue.

Check

- Which services are affected?
- Is it one server, a service or multiple regions?
- Are all users affected or only some?
- What business functions are impacted?
- Is there data loss or security impact?

Example

Only the API service in us-east-1 is affected. Authentication and the web frontend are working.

Useful Commands

```
# System overview
uptime
htop
# Check logs
journalctl -xe
# Check disk and memory
df -h
free -h
# Check network
ss -tulnp
# View recent changes
cat /var/log/auth.log | tail -n 50
```

8. REAL-WORLD EXAMPLE: HIGH CPU ON A WEB SERVER

Follow the troubleshooting flow for a real case.

- Symptom** Website is slow.
- Blast Radius** Only one web server is affected.
- Evidence** CPU at 100%, nginx processes using high CPU.
- Hypothesis** A surge in traffic or a runaway process.
- Mitigation** Restart the process and enable rate limiting.
- Verification** CPU back to normal, website responsive.

Key Commands Used

```
# Check CPU usage
htop
# Check top processes
ps aux --sort=-%cpu | head
# Check nginx logs
tail -n 50 /var/log/nginx/access.log
# Restart service
sudo systemctl restart nginx
# Verify
curl -I http://localhost
```

9. COMMON TOOLS FOR TROUBLESHOOTING

Tool	Purpose
top / htop	View running processes and resource usage
df / du	Check disk space and usage
free	Check memory usage
ss / netstat	Check network connections
journalctl	View system and service logs
dmesg	View kernel messages
iostat / vmstat	Check disk and system performance
sar	Historical performance data
lsof	List open files and connections
tcpdump	Capture and analyze network traffic
strace	Trace system calls for a process

10. BEST PRACTICES

- Stay calm and follow a structured approach.
- Gather evidence before making changes.
- Make small, safe changes and observe the results.
- Document everything (commands, findings, fixes).
- Use monitoring and alerts to detect issues early.
- Implement preventive measures to reduce future incidents.

11. KEY TAKEAWAYS

- 1 Use a structured troubleshooting framework.
- 2 Always confirm the blast radius.
- 3 Collect evidence from multiple sources.
- 4 Test and validate your hypothesis.
- 5 Mitigate the issue with a safe and controlled approach.
- 6 Verify the resolution and prevent recurrence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Run queues, hot processes, threads, throttling, evidence and recovery

1. OVERVIEW: CPU SATURATION

CPU saturation happens when the CPU is consistently busy and cannot handle more work. This causes high load, slow response times and degraded performance.

Common Symptoms

- High CPU usage (close to 100%)
- High load average
- Slow applications and timeouts
- Increased run queue length
- Users reporting slowness
- Some services not responding

2. HOW TO CHECK CPU USAGE

Use these commands to quickly check CPU usage, load and system health.

Key Commands

```
$ top
$ htop
$ uptime
$ mpstat -P ALL 1
$ vmstat 1 5
$ sar -u 1 5
$ cat /proc/loadavg
$ nproc
$ lscpu
```

Check load average

```
$ cat /proc/loadavg
5.32 4.91 4.15 3/456 12345
```

1 min

5 min

15 min

3. UNDERSTANDING RUN QUEUE

The run queue shows how many processes are waiting for CPU time. A high run queue indicates CPU saturation.

Run Queue vs CPU Cores

Run Queue	Meaning	Action
~ equal to cores	Normal	Monitor
Slightly higher	High load	Investigate
Much higher	CPU saturated	Take action

Check run queue and CPU cores

```
$ cat /proc/loadavg
12.45 11.32 10.11 6/789 23456
$ nproc
4
```

In this example, run queue (12.45) is much higher than 4 CPU cores, indicating saturation.

4. FIND HOT PROCESSES

Identify which processes are using the most CPU.

Find top CPU processes

```
$ top
$ htop
$ ps aux --sort=-%cpu | head -n 10
$ pidstat -u 1 5
$ sar -u 1 5
```

Example: Top CPU processes

```
$ ps aux --sort=-%cpu | head -n 5
USER      PID    %CPU    %MEM    COMMAND
app       2345   398.5    2.1    java
nginx     1234   120.3    1.5    nginx
root      5678    95.1    0.8    python
db        9012    80.4    4.2    mysqld
app       3456    60.2    1.1    node
```

5. ANALYZE THREADS

A single process can have many threads using CPU.

Check threads in a process

```
$ ps -T -p <PID>
$ top -H -p <PID>
$ htop # press H for threads
```

Example: View threads for a process

```
$ ps -T -p 2345 | head -n 5
PID      SPID    %CPU    COMMAND
2345     2345   150.2    java
2345     2346   98.5     java
2345     2347   85.1     java
2345     2348   40.3     java
2345     2349   25.0     java
```

6. CHECK CPU THROTTLING

Containers and cgroups can limit CPU using quotas. Throttling causes slower performance.

Check throttling (cgroups)

```
$ cat /sys/fs/cgroup/cpu/cpu.stat
nr_periods      10000
nr_throttled    5234
throttled_time  1234567890
$ cat /sys/fs/cgroup/cpu/cpu.cfs_quota_us
50000 # 50 ms quota
$ cat /sys/fs/cgroup/cpu/cpu.cfs_period_us
100000 # 100 ms period (0.5 CPU)
```

High nr_throttled and throttled_time indicates the process is hitting its CPU limit.

7. COLLECT EVIDENCE

Gather data to understand the issue and confirm the root cause.

Important Evidence

- ✓ CPU usage (top, htop, mpstat)
- ✓ Load average (/proc/loadavg)
- ✓ Running processes (ps, top)
- ✓ Thread details (ps -T, top -H)
- ✓ System logs (/var/log/*)
- ✓ Application logs
- ✓ cgroup throttling (if using containers)
- ✓ Recent changes (deployments, config)

8. COMMON CAUSES

CPU saturation can be caused by many factors.

Cause	Description
High traffic	Sudden increase in user requests
Infinite loop	Bug in application code
Resource limits	Too many processes or threads
Poor queries	Expensive database queries
Background jobs	Cron or batch jobs using CPU
Container throttling	cgroup CPU limits too low
Thundering herd	Too many workers/processes
Malware	Unexpected high CPU usage
System updates	Package upgrades running
Misconfiguration	Inefficient application settings

9. RECOVERY STEPS

Take action to reduce CPU usage and restore service.

- 1 Identify and stop or restart the problematic process (if safe).
- 2 Scale up resources (add CPU or instances).
- 3 Apply or adjust resource limits (cgroups, ulimit).
- 4 Optimize application code or queries.
- 5 Restart affected services.
- 6 Clear stuck or zombie processes.
- 7 Monitor to confirm CPU usage is back to normal.
- 8 Implement preventive measures.

10. REAL EXAMPLE: HIGH CPU INCIDENT

A web server becomes slow. Investigation shows one Java process consuming 400% CPU across multiple threads.

Investigation Steps

- Check CPU usage with top (CPU at 100%).
- Identify Java process using 400% CPU.
- Check threads (many threads running).
- Review application logs (infinite loop found).
- Restart the service after fixing the code.
- Monitor to confirm CPU usage is back to normal.

11. USEFUL COMMANDS REFERENCE

Task	Command
Check CPU usage	top, htop
Check load average	cat /proc/loadavg
Check per-core usage	mpstat -P ALL 1
Find top processes	ps aux --sort=-%cpu
Check threads	ps -T -p <PID>
Check throttling	cat cpu.stat (cgroups)
Check system info	lscpu, nproc
View system logs	journalctl -xe
Monitor over time	sar -u 1 5
Check application logs	tail -f /var/log/app.log

12. KEY TAKEAWAYS

- ✓ High run queue and load average indicate CPU saturation.
- ✓ Identify hot processes and threads.
- ✓ Check for cgroup throttling in containers.
- ✓ Collect system, application and log evidence.
- ✓ Fix the root cause and reduce CPU usage.
- ✓ Monitor after recovery to confirm stability.
- ✓ Implement limits and monitoring to prevent recurrence.

Leak versus cache, swap, reclaim, OOM events, mitigation and prevention

1. OVERVIEW: MEMORY EXHAUSTION

Memory exhaustion happens when the system runs out of available memory. It can be caused by real memory leaks, high application usage, kernel memory pressure, or too much cache. This leads to slow performance, swapping, OOM kills and service failures.

Common Symptoms

- High memory usage (close to 100%)
- System becomes slow or unresponsive
- Applications getting killed (OOM)
- Increased swapping
- High load average
- Services failing or restarting
- Users reporting timeouts

2. CHECK MEMORY USAGE

Start by checking overall memory usage, swap usage and real-time stats.

Key Commands

```
$ free -h
$ top
$ htop
$ vmstat 1 5
$ cat /proc/meminfo
$ sar -r 1 5
$ dmesg | tail -n 50
```

Example: free -h

```
$ free -h
              total        used        free      shared  buff/cache   available
Mem:    7.7G       7.2G       200M       300M       300M       400M
Swap:   2.0G       1.8G       200M
```

High used memory with low available memory indicates memory pressure.

3. LEAK VERSUS CACHE

Not all high memory usage is a memory leak. Linux uses available memory for cache to improve performance. You must determine if memory is truly leaking or just being used as cache.

Normal Cache Usage

- ✓ buff/cache is high
- ✓ available memory is enough
- ✓ memory can be reclaimed
- ✓ system is responsive
- ✓ this is normal behavior

Possible Memory Leak

- used memory keeps growing over time
- available memory is very low
- swap usage increases
- application memory grows continuously
- system becomes slow
- OOM events may occur

4. CHECK DETAILED MEMORY INFO

Use these commands to understand what is using memory.

Useful Commands

```
$ cat /proc/meminfo
$ top
$ htop
$ smem -r -k
$ ps aux --sort=-%mem | head -n 10
$ pmap -x <PID>
$ cat /proc/<PID>/status
$ cat /proc/<PID>/smaps | head -n 20
```

5. CHECK SWAP AND RECLAIM

Swapping slows down the system. Check swap usage and memory reclaim activity.

Swap Commands

```
$ swapon -s
$ free -h
$ vmstat 1 5
$ cat /proc/vmstat | grep -E "pswp|pgscan|pgsteal"
```

Example: vmstat output

```
$ vmstat 1 5
procs -----memory----- --swap--
r  b   swpd   free   buff   cache   si   so
2  0  102400  204800  50000  1200000  10  20
6  1  512000  10240   30000   900000  120 300
```

High si and so values indicate heavy swapping.

6. CHECK OOM EVENTS

The Out Of Memory (OOM) killer terminates processes when memory is exhausted.

Check OOM Logs

```
$ dmesg | grep -i oom
$ dmesg -T | grep -i "killed process"
$ journalctl -k | grep -i oom
$ cat /var/log/messages | grep -i oom
```

Example: OOM log

```
Out of memory: Killed process 1234
(java) total-vm:4096000kB, anon-rss:
2048000kB, file-rss:0kB, shmem-rss:0kB,
UID:1000 pgtables:4000kB oom_score_adj:0
```

OOM events indicate the system ran out of memory and killed a process.

7. FIND MEMORY HUNGRY PROCESSES

Identify which processes are using the most memory.

Top Memory Processes

```
$ ps aux --sort=-%mem | head -n 10
$ top
$ htop
$ smem -r -k | head -n 10
$ pmap -x <PID> | tail -n 20
```

Example: top output

PID	USER	%MEM	COMMAND
1234	app	25.0	java
2345	root	10.2	nginx
3456	app	8.1	python
4567	mysql	6.5	mysqld

8. MITIGATION STEPS

Take immediate action to reduce memory pressure and stabilize the system.

- 1 Identify and stop non-essential processes.
- 2 Restart leaking or misbehaving services.
- 3 Clear cache (if safe): `echo 3 > /proc/sys/vm/drop_caches`.
- 4 Increase swap (temporary fix).
- 5 Tune application memory settings.
- 6 Adjust system limits (ulimit, cgroups).
- 7 Consider scaling up resources.
- 8 Monitor after changes to confirm stability.

9. PREVENTION AND BEST PRACTICES

Follow these practices to prevent memory exhaustion in production.

- ✓ Monitor memory usage and set up alerts.
- ✓ Understand application memory requirements.
- ✓ Use cgroups to limit memory usage per service.
- ✓ Set appropriate ulimit values.
- ✓ Configure swap if needed.
- ✓ Identify and fix memory leaks in applications.
- ✓ Keep applications and libraries updated.
- ✓ Use monitoring tools (Prometheus, Grafana).
- ✓ Regularly review logs for OOM events.
- ✓ Perform load testing to validate memory behavior.

10. USEFUL FILES AND CONFIGURATION

Important files and settings related to memory management.

File / Setting	Purpose
/proc/meminfo	Memory usage details
/proc/vmstat	Memory reclaim and swap stats
/proc/swaps	Swap information
/etc/fstab	Swap configuration
/proc/sys/vm/*	Kernel memory parameters
/etc/sysctl.conf	Persistent kernel settings
/sys/fs/cgroup/	Memory limits (sgroups)

11. COMMON MEMORY PARAMETERS

Useful sysctl parameters for memory tuning.

Parameter	Description
vm.swappiness	Controls swap usage (0-100)
vm.overcommit_memory	Memory allocation behavior
vm.overcommit_ratio	Percentage of memory for overcommit
vm.min_free_kbytes	Minimum free memory
vm.dirty_ratio	Dirty page writeback threshold
vm.dirty_background_ratio	Background writeback threshold
vm.vfs_cache_pressure	Reclaim behavior for inode/dentry cache

12. KEY TAKEAWAYS

- 1 High memory usage can be a leak or just cache.
- 2 Always check available memory, not just used.
- 3 Monitor swap and reclaim activity.
- 4 Check for OOM events in logs.
- 5 Identify memory hungry processes.
- 6 Take action to reduce memory pressure.
- 7 Implement monitoring and alerts.
- 8 Use best practices to prevent future issues.



Full filesystem, inode exhaustion, I/O latency, deleted-open files, recovery

1. OVERVIEW: DISK & FILESYSTEM FAILURE

Disk and filesystem failures can cause service outages, application errors, data loss and slow performance. Common issues include full filesystem, inode exhaustion, high I/O latency, deleted-open files and disk hardware problems.

Common Symptoms

- No space left on device
- Inodes exhausted
- Slow disk I/O and high latency
- Applications failing to write files
- Deleted files still using space
- Read-only filesystem
- File system corruption
- Services crashing or unresponsive

2. CHECK DISK SPACE AND INODES

Check overall disk space usage and inode usage for all mounted filesystems.

Check disk space

```
$ df -h
$ df -Th
```

Check inode usage

```
$ df -i
$ df -ih
```

Example output: df -h

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	48G	1.2G	98%	/
tmpfs	2.0G	12M	1.9G	1%	/dev/shm
/dev/sdb1	200G	50G	150G	25%	/data

Example output: df -i

Filesystem	Inodes	IUsed	IFree	IUse%	Mounted on
/dev/sda1	3276800	3270000	6800	99%	/
/dev/sdb1	13107200	102400	13004800	1%	/data

3. IDENTIFY LARGE FILES AND DIRECTORIES

Find what is consuming disk space.

Find large directories

```
$ du -sh /* 2>/dev/null | sort -h
$ du -sh /var/* 2>/dev/null | sort -h
```

Find large files

```
$ find / -type f -size +100M 2>/dev/null
$ du -ah /var | sort -h | tail -n 20
```

Visualize disk usage (if available)

```
$ ncdu /
$ du -xhd 1 / | sort -h
```

Example output: large files

1.2G	/var/log/app/application.log
800M	/var/lib/docker/overlay2/...
600M	/var/log/nginx/access.log

4. CHECK I/O LATENCY AND DISK HEALTH

High I/O latency can make the system slow even if disk space is available.

Check disk I/O and latency

```
$ iostat -x 1 5
$ sar -d 1 5
$ dmesg | grep -i error
$ smartctl -a /dev/sda
```

Key metrics to check

Metric	What it means
await	Average I/O wait time (ms)
%util	Disk utilization (100% is saturated)
svctm	Average service time (ms)
r/s, w/s	Read and write requests per second
smartctl	Check for disk errors and health

5. FIND DELETED-OPEN FILES

A file that has been deleted can still use disk space if a process has it open.

Find deleted files still in use

```
$ lsof +L1
$ lsof | grep deleted
$ df -h # check space usage again
```

Example output: lsof +L1

COMMAND	PID	USER	FD	TYPE	SIZE	NAME
nginx	1234	www	7u	REG		/var/log/nginx/access.log (deleted)
java	5678	app	12u	REG		/tmp/data.tmp (deleted)

The file is deleted from the filesystem but still open by a process. Restart the process or truncate the file to free the space.

6. COMMON CAUSES AND INVESTIGATION

Understand why the disk or filesystem issue happened.

Cause	How to Investigate
Full filesystem	df -h, find large files/directories
Inode exhaustion	df -i, find many small files
High I/O latency	iostat, sar, dmesg, smartctl
Deleted-open files	lsof +L1, lsof grep deleted
Log files growing	Check /var/log, logrotate config
Application data	Check app directories (DB, cache)
Docker/container	docker system df, overlay2
Disk hardware issue	smartctl -a, dmesg, system logs
File system corruption	fsck (offline), check dmesg
Too many files	find count, check application

7. RECOVERY STEPS

Take action to free space and restore normal operation.

- 1 Identify what is using the space (df, du, lsof).
- 2 Remove or archive unnecessary files.
- 3 Truncate or rotate large log files.
- 4 Restart processes holding deleted files.
- 5 Clean up Docker images, containers and volumes.
- 6 Free inodes by removing unused small files.
- 7 Check and fix disk errors (fsck if needed).
- 8 Monitor after recovery to confirm stability.

8. LOG MANAGEMENT EXAMPLE

Log files are a common cause of disk space issues.

Check log file size

```
$ du -sh /var/log/* | sort -h
```

Truncate a large log file (safe)

```
$ sudo truncate -s 0 /var/log/app/app.log
```

Configure logrotate (example)

```
/var/log/app/*.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
}
```

10. USEFUL COMMANDS REFERENCE

Task	Command
Check disk space	df -h
Check inode usage	df -i
Find large directories	du -sh /* sort -h
Find large files	find / -type f -size +100M
Check disk I/O	iostat -x 1 5
Check disk health	smartctl -a /dev/sda
Find deleted-open files	lsof +L1
Check file system	fsck -f /dev/sda1 (offline)
View kernel messages	dmesg grep -i error

11. REAL EXAMPLE: FULL /VAR PARTITION

Scenario: /var is 100% full and the application is failing.

- 1 Check disk space: df -h
- 2 Find large files: du -sh /var/* | sort -h
- 3 Find deleted files: lsof +L1
- 4 Truncate or rotate logs: truncate -s 0 /var/log/app.log
- 5 Restart the application (if holding deleted files).
- 6 Verify space is freed: df -h
- 7 Set up logrotate to prevent recurrence.

9. PREVENTION AND BEST PRACTICES

- ✓ Monitor disk space and inode usage.
- ✓ Set up alerts before disk is full.
- ✓ Use logrotate for application logs.
- ✓ Implement log retention policies.
- ✓ Clean up old files and unused data.
- ✓ Monitor container and Docker storage.
- ✓ Check disk health regularly (smartctl).
- ✓ Use appropriate filesystem and mount options.
- ✓ Plan for growth and capacity.
- ✓ Document recovery procedures.

12. KEY TAKEAWAYS

- 1 Disk and filesystem issues can cause outages.
- 2 Always check both disk space and inode usage.
- 3 Use the right tools to find large files and processes.
- 4 Deleted-open files can still use disk space.
- 5 High I/O latency indicates disk or storage problems.
- 6 Take immediate action to free space and recover.
- 7 Implement monitoring, alerts and log management.
- 8 Regularly check disk health and plan for capacity.

DNS, routes, firewall, listening sockets, packets, dependency isolation

1. OVERVIEW: LINUX NETWORK FAILURE

Network failures can make applications unreachable, cause timeouts and break dependencies. The root cause can be DNS, routing, firewall rules, listening services, packet loss or an external dependency.

Common Symptoms

- Cannot reach a website or service
- Connection timeout or refused
- DNS resolution failures
- Intermittent connectivity
- Application works locally but not remotely
- Slow network or high latency
- Service is not listening
- Firewall blocking traffic
- Only some servers are affected
- Dependent service is down

2. CHECK BASIC CONNECTIVITY

Start with simple checks to confirm the problem and narrow the scope.

Basic connectivity commands

```
$ ping -c 4 google.com
$ ping -c 4 <server_ip>
$ curl -I https://google.com
$ curl -v http://<service_ip>:80
$ nc -zv <server_ip> <port>
$ telnet <server_ip> <port>
```

Example output: ping

```
$ ping -c 4 google.com
64 bytes from 142.250.190.78: icmp_seq=1 ttl=115
...
```

If ping fails, check DNS, routes, firewall and network configuration.

3. DNS RESOLUTION ISSUES

If DNS is failing, the server cannot resolve hostnames to IP addresses.

Check DNS resolution

```
$ nslookup google.com
$ dig google.com
$ host google.com
$ cat /etc/resolv.conf
$ systemd-resolve --status
```

Example output: nslookup

```
$ nslookup google.com
Server:      8.8.8.8
Address:     8.8.8.8#53
Name:        google.com
Address:     142.250.190.78
```

If DNS fails, check /etc/resolv.conf, DNS server reachability and network connectivity.

4. CHECK ROUTING AND GATEWAY

Verify routes, default gateway and reachability to other networks.

Routing commands

```
$ ip route
$ ip route get <ip>
$ ip -4 addr show
$ ip -6 addr show
$ netstat -rn
$ traceroute <ip>
$ mtr <ip>
```

Example output: ip route

```
$ ip route
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel
10.0.0.0/16 via 192.168.1.1 dev eth0
```

Ensure the default route is correct and packets can reach the destination.

5. CHECK FIREWALL AND SECURITY RULES

Firewall rules can block incoming or outgoing traffic.

Firewall commands

```
$ sudo ufw status verbose
$ sudo iptables -L -n -v
$ sudo firewall-cmd --list-all
$ sudo nft list ruleset
```

Example output: ufw status

```
$ sudo ufw status verbose
Status: active

To Action From
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
```

Check both local firewall and cloud security groups (if using cloud).

6. CHECK LISTENING SOCKETS

Confirm that the service is listening on the expected port and interface.

Socket commands

```
$ ss -tulpn
$ netstat -tulpn
$ lsof -i -P -n | grep LISTEN
$ systemctl status <service>
```

Example output: ss -tulpn

Netid	State	Local Address:Port	Process
tcp	LISTEN	0.0.0.0:22	sshd
tcp	LISTEN	127.0.0.1:3306	mysqld
tcp	LISTEN	0.0.0.0:80	nginx

If the service is not listening, check the application configuration and service status.

7. CHECK PACKET FLOW

Analyze network traffic to see if packets are reaching the server and how it responds.

Packet analysis commands

```
$ tcpdump -i any host <ip>
$ tcpdump -i any port <port>
$ sudo nmap -sV <server_ip>
$ mtr -rw <ip>
```

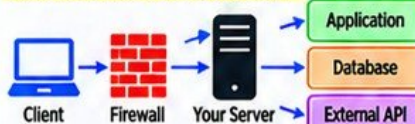
Example output: tcpdump

```
$ sudo tcpdump -i eth0 port 80
12:10:15.123 IP 192.168.1.10.54321 >
12:10:15.124 IP 192.168.1.20.80: Flags [S],
```

Use tcpdump or nmap to confirm if packets arrive and how the server responds.

8. DEPENDENCY ISOLATION

Identify if the issue is with a local service, remote service or a dependency.



Isolation steps

- 1 Test locally on the server (localhost).
- 2 Use multiple tools to confirm the issue.
- 3 Test from different locations and networks.
- 4 Test from an external network.
- 5 Check each dependency (database, API, etc).
- 6 Identify where the failure starts.
- 7 Fix or escalate based on findings.

9. COMMON CAUSES AND FIXES

Network issues can be caused by many factors. Use this list to guide your investigation.

Cause	What to Check	Common Fix
DNS failure	resolv.conf, DNS server	Fix DNS config
No route	ip route, gateway	Add or correct route
Firewall block	ufw, iptables, security group	Allow required port
Service not listening	ss, netstat, service status	Start service, fix config
Packet loss	tcpdump, mtr	Fix network issue
High latency	ping, mtr, application logs	Investigate network path
External dependency down	Test API, database, DNS	Fix or wait for provider
Misconfiguration	Network config files	Correct configuration

10. RECOVERY STEPS

Take action to restore connectivity and service.

- 1 Identify the root cause from your investigation.
- 2 Apply the appropriate fix (DNS, route, firewall, service).
- 3 Restart affected services if needed.
- 4 Validate that the service is working.
- 5 Monitor for recurrence.
- 6 Communicate the resolution.

11. BEST PRACTICES

- ✓ Always check DNS, routes and firewall first.
- ✓ Use multiple tools to confirm the issue.
- ✓ Test from different locations and networks.
- ✓ Monitor network latency and packet loss.
- ✓ Keep firewall rules documented.
- ✓ Implement monitoring and alerts.
- ✓ Use redundancy for critical dependencies.
- ✓ Regularly test disaster recovery procedures.

12. KEY TAKEAWAYS

- 1 Network failures can be caused by DNS, routes, firewall, services or external dependencies.
- 2 Always start with basic connectivity checks.
- 3 Use the right tools to inspect DNS, routing, firewall, sockets and packets.
- 4 Isolate the problem by testing each dependency.
- 5 Fix the root cause and validate the solution.
- 6 Implement monitoring and best practices to prevent future incidents.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Senior diagnostic workflow and command map for real incidents

1. OVERVIEW: PRODUCTION COMMAND CENTER

A centralized workflow and command map to investigate, diagnose and resolve real Linux production incidents. Use this structured approach to stay focused, collect the right evidence and reduce troubleshooting time.

Key Benefits

- ✓ Structured investigation flow
- ✓ Quick access to essential commands
- ✓ Covers all major incident types
- ✓ Helps with root cause analysis
- ✓ Useful for on-call and SRE teams

2. SENIOR DIAGNOSTIC WORKFLOW

Follow this flow for any production incident.

- 1 Detect & Understand**
Identify the symptom and scope.
- 2 Assess Blast Radius**
Check what is affected.
- 3 Collect Evidence**
Gather metrics, logs, configs.
- 4 Form Hypothesis**
Identify possible root cause(s).
- 5 Test and Verify**
Run commands and validate.
- 6 Mitigate and Recover**
Apply fix or workaround.
- 7 Prevent Recurrence**
Document and implement improvements.

3. INCIDENT TRIAGE CHECKLIST

Run these checks first to understand the issue.

Area	Key Commands
System health	uptime, top, free -h
Recent changes	last, rpm -qa --last, journalctl
Affected services	systemctl status <service>
Error messages	journalctl -xe, dmesg tail
Resource usage	top, htop, vmstat 1 5, iostat
Disk space	df -h, df -i, du -sh
Network connectivity	ip a, ip r, ping, ss -tulnp
Open files	lsof head, lsof +L1
Current users	who, w, last
Running processes	ps aux --sort=-%cpu head
System load	cat /proc/loadavg

Get a quick view of the system before going deeper.

4. SYSTEM AND RESOURCE COMMANDS

Check CPU, memory, disk and overall system health.

Purpose	Command
CPU usage	top, htop, mpstat 1 5
Memory usage	free -h, vmstat 1 5
Load average	uptime, cat /proc/loadavg
Disk usage	df -h, df -i, du -sh /path
Disk I/O	iostat, iostat -x 1 5
Process list	ps aux, pstree -p
System info	uname -a, lscpu, lsblk
Kernel messages	dmesg tail -n 50
System logs	journalctl -xe, journalctl -u <service>

Use these commands to quickly assess system health.

5. NETWORK AND CONNECTIVITY COMMANDS

Check network configuration, connectivity and sockets.

Purpose	Command
Network interfaces	ip a, ip link show
Routing table	ip r, route -n
Connectivity test	ping, ping -c 4 <host>
Port connectivity	nc -zv <host> <port>
DNS lookup	nslookup <domain>, dig <domain>
Listening sockets	ss -tulnp, netstat -tulnp
Firewall rules	sudo iptables -L -n -v sudo nft list ruleset
Packet capture	tcpdump -i any port 80
Network stats	sar -n DEV 1 5

Use these commands to diagnose network related issues.

6. SERVICE AND APPLICATION COMMANDS

Check service status, application logs and behavior.

Purpose	Command
Service status	systemctl status <service>
Start/stop service	systemctl start stop <service>
Service logs	journalctl -u <service> -xe
Follow live logs	journalctl -u <service> -f
Application logs	tail -f /var/log/<app>.log
Process details	ps aux grep <app>
Check listening port	ss -tulnp grep <port>
Restart service	systemctl restart <service>
Service configuration	cat /etc/<service>/<config>

Use these commands to check and manage services.

7. LOGS AND DEBUGGING COMMANDS

Collect and analyze logs for evidence.

Purpose	Command
View recent logs	journalctl -xe
Filter logs by time	journalctl --since "1 hour ago"
Search for errors	journalctl grep -i error
Follow logs live	tail -f /var/log/syslog
Kernel logs	dmesg tail -n 50
Application logs	tail -n 100 /var/log/<app>.log
Search log files	grep -r "pattern" /var/log/
Log rotation status	logrotate -d /etc/logrotate.conf

Logs provide critical evidence for root cause analysis.

8. DISK AND FILESYSTEM COMMANDS

Check disk space, inodes and file system health.

Purpose	Command
Disk usage	df -h
Inode usage	df -i
Directory size	du -sh /path
Find large files	find / -type f -size +100M
Deleted open files	lsof +L1
File system check	fsck -t /dev/sdX
Disk I/O stats	iostat -x 1 5
Mount information	mount column -t
Identify file types	file /path/to/file

Use these commands to diagnose disk related issues.

9. SECURITY AND USER INVESTIGATION

Check users, authentication and security events.

Purpose	Command
Current users	who, w, last
Failed login attempts	grep "Failed" /var/log/auth.log
Active SSH sessions	ss -tnp grep :22
User sudo activity	grep sudo /var/log/auth.log
Check file permissions	ls -l /path
Check SUID/SGID files	find / -perm -4000 -o -perm -2000
Audit system changes	ausearch -m USER -ts recent
Check open ports	ss -tulnp
Firewall status	sudo ufw status / iptables -L

Security issues can be a root cause of incidents.

10. INCIDENT COMMAND MAP

Quick reference for common incident types.

Incident Type	Key Commands
High CPU	top, ps aux --sort=-%cpu, mpstat
High memory	free -h, vmstat, ps aux --sort=-%mem
Disk full	df -h, df -i, du -sh, lsof +L1
Network issue	ping, ip r, ss -tulnp, tcpdump
DNS failure	nslookup, dig, cat /etc/resolv.conf
Service down	systemctl status, journalctl -u
Slow application	top, iostat, journalctl, logs
Permission issue	ls -l, id, sudo -l, grep auth.log
Too many open files	ulimit -n, lsof wc -l, cat /proc/sys/fs/file-nr

Use this map to quickly find the right commands.

11. REAL EXAMPLE: WEBSITE IS DOWN

Follow the workflow for a real incident.

- 1** Symptom: Users cannot access website.
- 2** Check scope: Is it local or global?
- 3** Check service: systemctl status nginx
- 4** Check logs: journalctl -u nginx -xe
- 5** Check network: ss -tulnp | grep :80
- 6** Check disk: df -h (is disk full?)
- 7** Check config: nginx -t
- 8** Mitigate: Restart service or apply fix.
- 9** Verify: curl http://localhost
- 10** Prevent: Set monitoring and alerts.

Use the workflow and command map together.

12. KEY TAKEAWAYS

- ✓ Use a structured workflow for every incident.
- ✓ Always check system, network, disk and services.
- ✓ Collect and analyze logs for evidence.
- ✓ Use the right commands for each incident type.
- ✓ Check the blast radius and dependencies.
- ✓ Verify your findings before applying fixes.
- ✓ Document the root cause and solution.
- ✓ Implement monitoring and alerts.
- ✓ Use this command center as a quick reference.
- ✓ Practice these commands regularly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

40